CineRankX

Complete Project Study and Viva Guide

An end-to-end technical learning resource: history, mathematics, code walkthrough, evaluation, demonstration script and 150+ viva questions

Department of Artificial Intelligence and Machine Learning

Walchand College of Engineering, Sangli

Mini Project, Academic Year 2026-27

Document built from repository commit 3938f34

Metrics verified against saved artifacts in data/processed/ and reports/

Every statement in this guide is grounded in the repository, its git history and its saved evaluation artifacts. Where something could not be verified it is marked UNVERIFIED.

Table of Contents

Executive Summary 7
What exists today (verified against the repository) 7
The headline numbers, honestly stated 7
Chapter 1. Project History and Origin 9
1.1 Why the project exists 9
1.2 Questions this chapter answers 9
1.3 Chronological timeline (from git log) 9
1.4 The dataset migration (a major early decision) 10
1.5 Problems found and fixed along the way 10
1.6 How the project evolved after the first four modules 11
1.7 Status ledger 11
1.8 Weekly progress reports (supporting evidence) 11
Chapter 2. Problem Statement and Motivation 13
2.1 The general problem: too many movies, too little attention 13
2.2 How conventional recommenders work, and where each one fails 13
2.3 Why one strategy cannot suit every user 13
2.4 Why single-objective ranking is not enough 13
2.5 Formal problem statement 14
2.6 Objectives, scope and research gap 14
2.7 Expected versus achieved outcomes 14
Chapter 3. Complete Project Overview 15
3.1 What CineRankX does, in plain words 15
3.2 The user journey in the application 15
3.3 The three engines and how they cooperate 15
3.4 Strategy selection versus hybrid blending (what is actually implemented) 16
3.5 System architecture 16
3.6 End-to-end data flow 17
Chapter 4. Technology Stack 19
4.1 Stack at a glance 19
4.2 Technology by technology 19
4.3 What is not used 21
Chapter 5. Datasets and Data Engineering 22
5.1 MovieLens 22
5.2 TMDB 22
5.3 Joining MovieLens and TMDB 23

5.4 Text cleaning 23
5.5 From movies to model-ready features 23
5.6 Splitting data (three different splits in the project) 23
5.7 Cold-start data and why it is simulated 24
5.7b Missing values and duplicates 24
5.8 Processed artifacts 24
Chapter 6. The Recommendation Algorithms, Explained From Scratch 25
A. Collaborative Filtering (CF) 25
B. Content-Based Filtering 26
C. Popularity-Based Recommendation 28
D. Predicted Rating (the bias baseline) 28
E. Hybrid recommendation: what is and is not implemented 28
F. Summary table 29
G. Likely viva questions for this chapter 29
Chapter 7. The Six Recommendation Objectives 30
7.1 How the ranker combines them 30
7.2 Objective 1: Relevance 30
7.3 Objective 2: Diversity 30
7.4 Objective 3: Novelty 31
7.5 Objective 4: Coverage 31
7.6 Objective 5: Popularity / Quality 31
7.7 Objective 6: Predicted Rating 31
7.8 A worked greedy example (illustrative numbers) 32
7.9 Why not maximise relevance alone? 32
7.10 Tuning the weights (benchmark) 32
7.11 Genre-overlap relevance gate (not a seventh objective) 32
Chapter 8. The Strategy Classifier: Deep Technical Explanation 33
8.1 Why a classifier at all? 33
8.2 What it predicts 33
8.3 How the training labels are made (the most important idea) 33
8.4 The features (47 after removing 2 redundant ones) 34
8.5 Training, validation and testing protocol 34
8.6 LightGBM explained from scratch 35
8.7 Prediction path 35
8.8 Other approaches genuinely tested 35
8.9 Confusion matrix and feature importance 36
Chapter 9. The Classifier Accuracy Investigation 37

9.1 Reproducing the headline numbers 37
9.2 Why the accuracy is low: the ranked root causes 37
9.3 Why the majority baseline can beat the classifier on accuracy 39
9.4 Why changing algorithms did not help 39
9.5 Further experiments on the labelled data (selector_phase2_base.json) 40
9.6 What the selector is actually worth 40
9.7 Did a better label definition help? 41
9.8 Metric glossary for this chapter 41
9.9 Improving the content-based strategy (content_experiments.json) 42
9.10 End-to-end effect of the improved content strategy (selector_phase2_rq_t30_h0.3.json) 42
9.10b The best experiment so far: improved content + larger label budget (selector_phase2_rq_t50_h0.4.json) 43
9.11 Status of the more advanced strategy-selection ideas 43
9.12 How to defend the result in one paragraph 44
Chapter 10. Model Evaluation and the Mathematics Behind It 45
10.1 Classification metrics 45
10.2 Ranking and recommendation metrics 46
10.3 Rating prediction metrics 47
10.4 System metrics 47
10.5 Evaluation methodology concepts 47
10.6 The frozen benchmark protocol (how recommendation accuracy is measured fairly) 48
Chapter 11. Module-by-Module Codebase Walkthrough 50
11.1 Directory structure 50
11.2 eval_config.yaml: the single configuration file 50
11.3 Package src/ingestion (Module 1) 50
11.4 Package src/ir_engine (Module 2) 51
11.5 Package src/ranking_engine (Module 4) 51
11.6 Package src/strategy_selector (Module 3) 52
11.7 Package src/evaluation (the 23-KPI suite) 53
11.8 Package src/benchmark (frozen comparison) 54
11.9 Package src/storage (SQLite) 55
11.10 Package src/ui and app.py (the web application) 55
11.11 scripts/ 56
11.12 Where the saved results come from 56
Chapter 12. End-to-End Execution Flow 57
12.1 The sequence in the actual implementation 57
12.2 Stage by stage 57

12.3 The offline training sequence 59
Chapter 13. Project Differentiation and Competitive Analysis 60
13.1 Comparison table 60
13.2 Modern platforms 61
13.3 What is our actual innovation? 61
13.4 Why it is not "just another movie recommendation website" 61
13.5 Why six objectives, and why per-user selection 61
13.6 Limitations of the approach (say them first) 61
13.7 What could make this publishable 61
Chapter 14. Testing, Debugging and Software Engineering 63
14.1 How to run the tests 63
14.2 What each test file validates 63
14.3 Types of testing in the project 64
14.4 Error handling and logging 64
14.5 Dependencies, serialisation, reproducibility 64
14.6 Realistic debugging scenarios 64
Chapter 15. Limitations and Future Scope 66
15.1 Real limitations (current implementation) 66
15.2 Future work (not implemented) 66
Chapter 16. Viva Preparation: 164 Questions with Model Answers 68
A. Project introduction and motivation 68
B. Problem statement and research gap 69
C. Architecture and system design 69
D. Dataset and preprocessing 70
E. Collaborative filtering 71
F. Content-based filtering 72
G. Popularity-based recommendation 73
H. Hybrid recommendation 73
I. TF-IDF and cosine similarity 74
J. Multi-objective recommendation 74
K. The strategy classifier 75
L. LightGBM and machine learning 76
M. Evaluation metrics 77
N. Accuracy and label noise 78
O. Software architecture and implementation 79
P. Testing and debugging 80
Q. Project differentiation 81

R. Limitations and future scope 81
S. Advanced research questions 82
Chapter 17. Practical Demonstration Preparation 84
17.1 Before the examiner arrives (10-minute checklist) 84
17.2 Demonstration script (about 10 minutes) 84
17.3 Questions an examiner may ask while the app is running 85
17.4 Troubleshooting during the demo 85
Chapter 18. Ten-Day Study Roadmap 86
Appendix A. Glossary 87
Appendix B. Quick Revision Notes (one page) 89
Appendix C. References and Source Paths 90
C.1 Repository sources used for this guide 90
C.2 Literature (recalled from memory; verify details before citing in a report) 90
Appendix D. Complete Code Inventory (auto-extracted) 91

(If page numbers are missing, right-click the table in Word and choose Update Field.)

Executive Summary

CineRankX is a movie recommendation system built for the Department of Artificial Intelligence and Machine Learning at Walchand College of Engineering, Sangli (mini project, AY 2026-27). Its central idea, taken directly from the project README, is that *existing recommender systems fix one strategy and rank by a single similarity score, ignoring diversity and per-user variation*. CineRankX therefore does two things that a conventional recommender does not:

1. It **predicts, for each user, which of three recommendation strategies is likely to work best** (content-based, collaborative filtering, or popularity) using a machine-learning classifier.
2. It **re-ranks candidate movies on six objectives at once** (relevance, diversity, novelty, coverage, popularity/quality and predicted rating) instead of sorting by one similarity number.

The system runs on **MovieLens 20M** (20,000,263 ratings from 138,493 users on 27,278 movies) enriched with **TMDB 2023** metadata (overviews, keywords, genres, posters). It is built in Python with pandas, NumPy, SciPy, scikit-learn, NLTK, LightGBM, SQLite and a Streamlit web application.

What exists today (verified against the repository)

Area	State	Evidence
Data pipeline	Complete: MovieLens + TMDB join (26,743 of 27,278 movies matched, 98.04%), NLTK text cleaning	`src/ingestion/`, `data/processed/movies_merged.csv`
Module 2: IR engine	Complete: TF-IDF (5,000 terms, bigrams) + cosine search + Trie autocomplete	`src/ir_engine/`
Module 3: strategy selector	Complete but **modest accuracy**: LightGBM multiclass, held-out accuracy 0.617 vs majority class 0.644	`src/strategy_selector/`, `data/processed/strategy_eval.json`
Module 4: ranking engine	Complete: three candidate sources, six-objective greedy re-ranking, tunable weights	`src/ranking_engine/`
Evaluation suite	Complete: 23 KPIs named in `eval_config.yaml`	`src/evaluation/`, `reports/kpi_report.json`
Web application	Complete: Overview, Engines, Run Engine, User Profile (User 360), KPI Benchmarks	`app.py`, `src/ui/`
Frozen benchmark	Complete: deterministic split, frozen popularity baseline, EASE-based CineRankX relevance	`src/benchmark/`, `reports/strategy_comparison.json`
Tests	272 pytest tests passing at last full run	`tests/`

The headline numbers, honestly stated

- **Recommendation quality** (frozen benchmark, 2,000 test users): CineRankX reaches **Hit Rate@10 = 0.877**, against **0.601** for the frozen most-popular baseline and **0.751** for the earlier CineRankX pipeline.
- **Strategy classifier** (999 held-out users, none removed): **accuracy 0.617**, **macro-F1 0.472**, **balanced accuracy 0.469**, **ROC-AUC 0.735**. Always predicting "popularity" would score 0.644 accuracy, so *the classifier does not beat the majority class on plain accuracy*.
- **What strategy choice is worth**: the selector's chosen strategy achieves a mean hit-rate of **0.128** against **0.112** for always using popularity and **0.148** for an oracle (an optimistic upper bound). The gain over popularity is **+0.0155, 95% CI [+0.011, +0.020]**, so it is statistically real even though classification accuracy looks weak.

- **Best experiment so far (not deployed)**: with a more reliable label definition (50 recommendations, 40% holdout) and an improved content-based strategy, held-out accuracy rises to **0.677 against a 0.564 majority rate**, macro-F1 to **0.586**, ROC-AUC to **0.807**, and the selector's gain over always-popularity doubles (**+0.037, 95% CI [+0.030, +0.044]**). The two settings use different label definitions, so compare each with its own baseline (section 9.10b).
- **Why accuracy is capped (deployed setting)**: two independent re-labellings of the same users agree on only about **65-73%** of users, and a perfect model that knew each user's true hit-rates would reach only about **0.76**. The labels, not the algorithm, are the bottleneck. This is a central viva point (Chapter 9).

> **How to use this guide.** Read Chapters 1-3 for the story and the system, 5-8 for data and algorithms, 9-10 for evaluation maths, 11-12 for the code, 16 for viva questions and 17 for the demo script. Chapter 18 is a 10-day study plan.

Chapter 1. Project History and Origin

This chapter reconstructs the story from evidence: the git history (21 commits at the time of writing), `README.md`, `DATASET_MIGRATION.md`, `CLAUDE.md`, `eval_config.yaml` and the code itself. Nothing here is invented; where the repository is silent the text says so.

1.1 Why the project exists

The README states the motivation in one sentence: *"Existing recommender systems fix one strategy and rank by a single similarity score, ignoring diversity and per-user variation. CineRankX predicts the best strategy per user via ML classifiers and ranks candidates on six weighted objectives, for relevant, diverse, explainable recommendations."*

`CLAUDE.md` (the project context file) adds the structure of the original plan: **three engines built in a fixed order**:

1. A classical information-retrieval engine (TF-IDF + Trie).
2. An adaptive machine-learning strategy-selection engine.
3. A multi-objective hybrid ranking engine.

The same file records the working conventions the team adopted: one module at a time, pytest tests before moving on, commit after each module passes, ranking weights exposed as configuration (never hard-coded), and no deep learning ("this stays explainable and rule-based (TF-IDF + classical ML classifiers only)").

1.2 Questions this chapter answers

Question	Short answer (details below)
What problem were we solving?	Single-strategy, single-score recommenders ignore per-user differences and list diversity
What was the original idea?	Predict the best strategy per user, then rank on six objectives
What did we implement?	All three engines, an evaluation suite, a web UI, a SQLite user store, a frozen benchmark
What changed along the way?	Datasets were migrated; popularity definition was corrected; the classifier and its labels were re-engineered; the UI was redesigned
What is incomplete?	Content-based strategy is weak; classifier accuracy is label-limited; no live user feedback; no contextual bandit

1.3 Chronological timeline (from git log)

Date	Commit (author)	What happened
2026-08-23	`eac5871` Initial commit; `f1f836e` Module 1 finished (Shivam Bangade)	Repository created; first data-ingestion module with `ml-latest-small` and TMDB 5000
2026-08-23	`ee397bd` Module 1: migrate to TMDB and MovieLens datasets	**Dataset migration** to MovieLens 20M + TMDB 2023 (details in 1.4)
2026-08-23	`9e393c1` Module 2: Classical IR Engine (18 tests)	TF-IDF + Trie engine
2026-08-23	`85a586e` Module 3: Adaptive ML Strategy Selection Engine (31 tests)	First classifier (Random Forest) with backtest labels
2026-08-25	`ce108d5` Module 3: SMOTE investigation (41 tests)	SMOTE tested, **shipped disabled**
2026-08-25	`11effea` Module 4: Multi-Objective Hybrid Ranking Engine (72 tests)	Six-objective ranker
2026-08-25	`3583413` Evaluation suite: all 23 KPIs (71	Evaluation module

	tests)	
2026-08-25	`c273ba3` Rank popularity by observed rating counts, not TMDB popularity	Important bug fix (1.5)
2026-08-25	`e536505`, `9acaca7` README with evaluation suite and final KPI report	Documentation milestone
2026-09-30	`116f21a` Recommendation results dashboard (Shardul Funde, PR #1)	First Streamlit results page
2026-10-01	`74e14c5` (prathmesh0077)	Comment in `src/ui/results.py`
2026-10-15	`8ebab03`, `47d423b` (Aditya Patil)	README updates
2026-10-15	`ef395a9` Redesign UI and add User 360 profile with SQLite user store	Analytics UI, user profiles, run history
2026-11-04	`d2c3d22` Improve strategy selector: leak-free labels, richer features, CV model selection	Classifier re-engineering
2026-11-05	`3938f34` Frozen benchmark, EASE relevance, utility selection and selector diagnostics	Benchmark and diagnostics

> **Note on dates.** Dates are copied from `git log --date=short`. Contributor names are the git author names. Which team member did which part of the later work is not recorded beyond these author fields.

1.4 The dataset migration (a major early decision)

`DATASET_MIGRATION.md` documents three stages:

Stage	Datasets	Join result	Verdict
Before	MovieLens `ml-latest-small` (100,836 ratings, 9,742 movies) + TMDB 5000	3,537 of 9,742 joined (**36.3%**)	TMDB 5000 is a fixed 5,000-movie slice, so most MovieLens films are absent
Considered, not adopted	MovieLens `ml-32m` + TMDB 5000	projected **5.3%** (4,634 / 87,585)	More MovieLens movies make the *rate* worse
After (current)	MovieLens 20M + TMDB 2023 (near-full TMDB export)	26,743 of 27,278 joined (**98.04%**)	Solved by upgrading the *bottleneck* dataset

Two consequences recorded in the same document: the TMDB export has **no cast/crew columns**, so `text_chunk` is overview + genres + keywords only (a deliberate scope decision, not a bug), and TMDB genres/keywords are plain comma-separated strings (not JSON), so the joiner uses a simple split.

1.5 Problems found and fixed along the way

These are real engineering events from the history and migration notes:

1. **Popularity defined the wrong way (commit c273ba3).** Both the strategy backtest and the candidate generator ranked "popular" by TMDB's `popularity` column. That metric is live and recency-weighted, while MovieLens ratings stop in 2015, so the TMDB top-10 was touched by only 39% of users (87% for the most-rated top-10). Switching to observed rating counts changed the label mix from collaborative 84% / popularity 9% / content 7% to roughly 52% / 45% / 3%, and raised classifier lift over the majority baseline from -0.045 to +0.144.
2. **A flattering cold-start baseline.** The evaluation's popularity baseline used the same TMDB metric, making CineRankX look +0.35 better than it was. With the correct baseline the cold-start lift was -0.03 and, after the popularity fix in both engines, +0.02.
3. **SMOTE investigated and rejected.** Over 10 random seeds SMOTE raised content-based F1 by about 0.038 but left ROC-AUC flat (0.605-0.627) and cost accuracy in 10 of 10 seeds. Conclusion in the migration notes: rebalancing moves the decision threshold; it does not create information.

4. **Label noise.** Multi-split averaging was added to the labeller because single-split winners flipped by chance (popularity "won" for 37.7% of users on one split but 8.9% when averaged over five).

1.6 How the project evolved after the first four modules

- **UI (Sept-Oct 2026):** a simple Streamlit results page (`app.py`) was added by a team member; later restyled into a multi-section analytics dashboard with Overview, Engines, Run Engine, User Profile and KPI pages, and a local SQLite store (`data/processed/cinerankx.db`) holding all 20M ratings and every recommendation run.
- **Evaluation integrity (Nov 2026):** the strategy classifier's labels were rebuilt so features can never see the ratings that decide a label (a "profile / holdout" split), and a **frozen benchmark** with an independent baseline was added so improvements cannot move the reference.
- **Better relevance model:** an EASE item-item linear model (`src/ranking_engine/ease.py`) was added as a candidate source in the benchmark pipeline, raising Hit Rate@10 from 0.60 (baseline) to 0.877.
- **Diagnostics:** scripts such as `scripts/analyze_selector.py`, `label_experiments.py`, `content_experiments.py` and `selector_phase2.py` measure *why* the classifier is limited, rather than only trying new algorithms.

1.7 Status ledger

Category	Items
Initially proposed and implemented	IR engine (TF-IDF + Trie); strategy classifier; six-objective ranker; 23-KPI evaluation suite
Implemented later / improved	Dataset migration; popularity definition; leak-free labels; richer features; frozen benchmark; EASE relevance; web dashboard; SQLite store; user profiles
Implemented but disabled / not adopted	SMOTE (`use_smote=False`); utility-based selector with fallback (evaluated, ties with the simple classifier); improved content model and larger-budget labels (evaluated end to end, clearly better in the experiment of section 9.10b, but **not deployed**)
Incomplete or weak	Content-based strategy hit-rate (0.016); classifier accuracy ceiling; cold-start gain is small
Future possibilities (not built)	Contextual bandits, real user feedback loop, deep/embedding models, hybrid fusion of strategy lists, cast/crew metadata

1.8 Weekly progress reports (supporting evidence)

The untracked file `demo/weekly_reports_week7-12.md` (and its `.docx` twin) contains the project's weekly sheets for weeks 7 to 12, addressed to the mentor, **Prof. Sahil Nadaf**. It states its own assumptions (the split into Team A data/IR, Team B ML selector, Team C ranking/UI, and that weeks 3 to 6 covered TF-IDF, classifier and ranker are *assumed*, not recorded), so treat team labels as UNVERIFIED. Its content is consistent with the git history:

Week	Focus recorded in the report
7	Evaluation suite and the 23 KPIs; cold-start simulated; SMOTE shipped off; popularity switched to observed counts
8	Streamlit results dashboard merged by pull request; ranked-CSV validation
9	UI redesign (design tokens, Feather icons), runner with six weight sliders, reset and validation
10	Full MovieLens 20M + TMDB 2023 load (26,743 movies matched), classifier retrained (accuracy 0.52 at that time, equal to the majority baseline), SQLite store with 20,000,263 ratings

11	User 360 profile, run history logging, no-authentication note
12	End-to-end test, KPI report saved as JSON, demo script; **cloud deployment blocked** (no admin access to the GitHub repository; the 1.4 GB data is not in git); plan: demo locally or fork and bundle a small sample

> **Caution for the viva:** the sheets contain "Mentor response" lines (for example "keep the numbers honest"). These are as written in the report; they are not independently verifiable from the repository, so quote them only if you know they match what your mentor actually said.

Chapter 2. Problem Statement and Motivation

2.1 The general problem: too many movies, too little attention

A streaming catalog can contain tens of thousands of titles. The MovieLens catalog used here has 27,278 movies; the TMDB export we joined has about 1.5 million. A user cannot browse that. A **recommender system** is software that selects a short list of items the user is likely to value. The difficulty is that "likely to value" is not directly observable. We only observe indirect evidence: ratings, tags, what other users rated, and item descriptions.

2.2 How conventional recommenders work, and where each one fails

Approach	Idea	Typical weakness (and what it means for CineRankX)
Popularity-based	Recommend what most people rated	Not personalised; reinforces *popularity bias* (the rich get richer); but surprisingly strong on offline accuracy, which is why it is our fallback and our frozen baseline
Collaborative filtering (CF)	"Users like you liked X"	**Cold start** (new users/items have no history); the user-item matrix is extremely sparse (MovieLens 20M: 20M ratings among 138,493 x 27,278 = 3.8 billion possible pairs, about 0.53% filled)
Content-based filtering	"X is similar to what you liked" using item text/metadata	Over-specialisation (always similar items); depends on metadata quality; in our data it is the weakest individual strategy (hit-rate 0.016)
Hybrid	Combine several of the above	Usually one *fixed* blend for everybody
Single-objective ranking	Sort by one score	The top of the list is often ten near-duplicates; no notion of diversity, novelty or coverage

2.3 Why one strategy cannot suit every user

Different users give the different methods different amounts of evidence:

- A user with 5 ratings gives collaborative filtering almost nothing to work with, but popularity needs no history.
- A user whose ratings cluster in one genre gives content-based filtering a clear profile; a user who rates everything gives it a blurry one.
- A user whose taste matches a large neighbourhood benefits from CF; an idiosyncratic user does not.

CineRankX's data confirms this is not just theory. Over 4,993 sampled users the best strategy on a hidden holdout was popularity for 64% of users, collaborative for 33% and content-based for 2% (Chapter 9). The mix differs per user, which is exactly what a per-user strategy selector tries to exploit.

2.4 Why single-objective ranking is not enough

If we rank only by relevance, the top ten for a *Toy Story* fan are nine animated sequels. A good list also needs:

- **Diversity**: items should not be near-copies of each other.
- **Novelty**: some items the user is unlikely to already know.
- **Coverage**: across all users, the system should not only ever show the same few titles.

- **Quality / popularity**: well-regarded items are safer bets.
- **Predicted rating**: how much we expect this specific user to like it.

These objectives conflict. Raising diversity lowers average relevance (the project measured diversity 0.952 -> 0.972 at the price of relevance 0.814 -> 0.657 when diversity weight was raised from 0.15 to 0.60). The weights therefore expose the trade-off.

2.5 Formal problem statement

> **Given** a user *u* with an observed rating history *R_u* (possibly very short), a movie catalog *M* with text and metadata, and a population of other users' ratings, **select** a strategy *s(u)* from {content-based, collaborative, popularity} predicted to perform best for *u*, **generate** candidate movies from the strategies weighted by that prediction, and **rank** them on six weighted objectives (relevance, diversity, novelty, coverage, popularity/quality, predicted rating) to produce a top-K list that is accurate on held-out interactions while remaining diverse, novel and explainable.

2.6 Objectives, scope and research gap

Objectives (from the README and the code):

1. Build a classical IR engine (TF-IDF + Trie) over movie text.
2. Learn a per-user strategy selector from backtested strategy performance.
3. Re-rank candidates on six tunable objectives.
4. Evaluate everything with a fixed set of 23 named KPIs.
5. Provide an interactive application for inspection (later work).

Scope: offline evaluation on MovieLens 20M with TMDB enrichment; explainable classical machine learning; no deep learning; no live user feedback.

Research gap addressed: most hybrid recommenders blend strategies with one global weighting, and most re-rankers optimise one or two objectives. CineRankX studies (a) *user-specific* strategy choice and (b) a six-objective re-ranker in one pipeline, and, importantly, measures honestly how reliable the strategy labels are. This is a combination of established techniques, not a claim of a brand-new algorithm (Chapter 13).

2.7 Expected versus achieved outcomes

Expected (from README/CLAUDE.md)	Achieved (verified)
Classifier that predicts best strategy per user	Built; deployed LightGBM has accuracy 0.617, macro-F1 0.472, ROC-AUC 0.735; it does *not* beat the majority class on accuracy but does beat it on balanced accuracy (0.469 vs 0.333 chance) and recommendation value
Ranking on six objectives with tunable weights	Built and tested (72 ranking tests); weights in `eval_config.yaml`
23-KPI evaluation suite	Built; `scripts/generate_kpi_report.py` writes `reports/kpi_report.json`
Diverse, relevant, explainable lists	Diversity 0.837, MAP 0.114 in the KPI report; every result card shows a score breakdown and reason
(Later) accurate recommendations versus a fixed baseline	Hit Rate@10 0.877 vs 0.601 frozen baseline on the benchmark
(Implicit hope) high classifier accuracy	**Not achieved** (label noise ceiling about 0.76, Chapter 9)

Chapter 3. Complete Project Overview

3.1 What CineRankX does, in plain words

Open the web app, enter a MovieLens user ID, choose how many movies you want and how much you care about each of the six objectives, and press **Generate recommendations**. CineRankX:

1. looks at that user's rating history,
2. computes a feature vector describing the user,
3. asks a trained classifier which strategy suits them (and how confident it is),
4. pulls candidate movies from three sources, weighting the sources by that prediction,
5. re-ranks the candidates greedily on the six objectives, and
6. shows ranked movie cards with a score breakdown, lets you export a CSV, and records the run in a database.

3.2 The user journey in the application

Step	Screen	What happens
1	Top navigation	Five sections: Overview, Recommendation Engines, Run Engine, User Profile, KPI Benchmarks; the URL keeps the section (`?view=run`)
2	Overview	Catalog size, ratings in sample, strategy model status, search MRR; candidate funnel; objective weights; strategy routing; telemetry; surfaced candidates
3	Recommendation Engines	Three cards describing the IR engine, the strategy selector and the ranker, with real metrics from the saved report
4	Run Engine	Choose "Live engine" or "Upload ranked CSV"; set user ID, K and the six weights; press Generate; see movie cards and export CSV
5	User Profile	A 360-degree view: summary, paginated rating history, genre/decade/language preferences, past recommendation runs, current recommendations
6	KPI Benchmarks	Classification quality, the strategy comparison (frozen baseline vs CineRankX), top-K charts, system performance

3.3 The three engines and how they cooperate

- **Engine 1: IR engine** (`src/ir_engine`). Builds a TF-IDF vector for every movie and answers "which movies are most similar to this text or this movie?". A Trie provides instant title autocomplete.
- **Engine 2: Strategy selector** (`src/strategy_selector`). Turns a user's history into features and predicts which of three strategies fits best, with a confidence.
- **Engine 3: Ranking engine** (`src/ranking_engine`). Builds a candidate pool from IR similarity, collaborative filtering and popularity, then selects the final list one movie at a time using six objectives.

An **evaluation suite** (`src/evaluation`) and a **benchmark package** (`src/benchmark`) measure the whole thing, and a **UI layer** (`app.py`, `src/ui`) presents it.

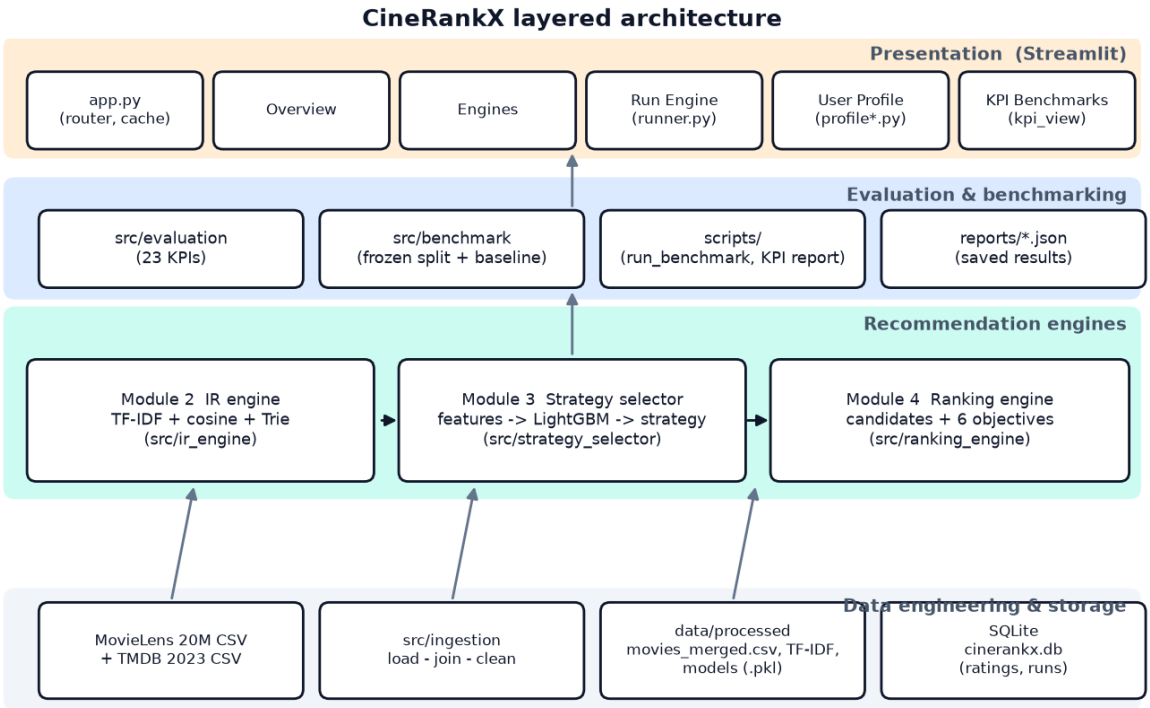
3.4 Strategy selection versus hybrid blending (what is actually implemented)

This distinction is a common viva trap, so be precise:

Term	Meaning	In CineRankX
Strategy selection	Pick *one* strategy for a user	The classifier predicts a strategy and a confidence
Strategy switching	Change the strategy at run time by a rule	Not used as a hard switch
Weighted score combination (hybrid)	Blend several strategies' scores	**Yes**: `strategy_source_weights(strategy, confidence)` turns the prediction into blending weights over the three sources
Hybrid ranking	Combine objectives	**Yes**: six weighted objectives in the ranker

So the predicted strategy is a **weighted prior**, not a hard route: at chance confidence (1/3) the blend equals a fixed prior (collaborative 0.60, content 0.20, popularity 0.20); at confidence 1.0 it is fully the predicted strategy; in between it interpolates linearly.

3.5 System architecture



Layered architecture of CineRankX

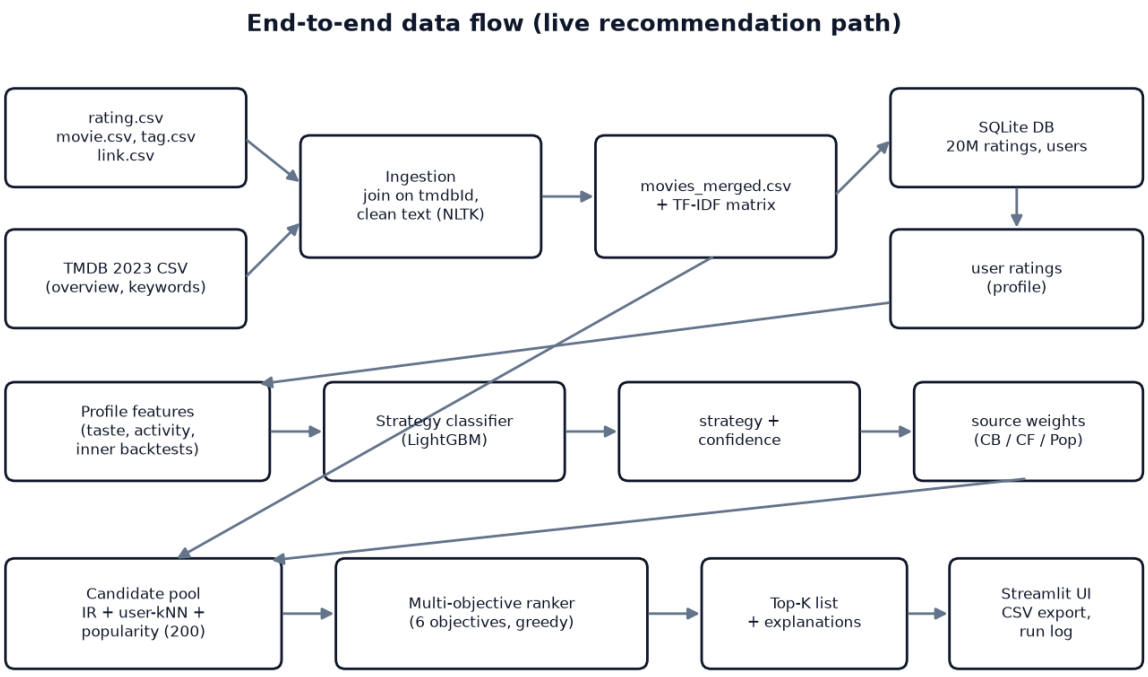
```
flowchart TB
  subgraph Data["Data engineering and storage"]
    A[MovieLens 20M CSV] --> C[src/ingestion: load, join, clean]
    B[TMDB 2023 CSV] --> C
    C --> D[data/processed/movies_merged.csv]
    A --> E[(SQLite cinerankx.db: ratings, user_stats, runs)]
  end
  subgraph Engines["Recommendation engines"]
```

```
    D --> F[Module 2: IR engine TF-IDF + Trie]
    E --> G[Module 3: Strategy selector]
    F --> H[Module 4: Candidate generator + six-objective ranker]
    G --> H
  end
  subgraph Eval["Evaluation"]
    H --> I[src/evaluation 23 KPIs]
    H --> J[src/benchmark frozen split and baseline]
  end
  subgraph UI["Presentation: Streamlit"]
    H --> K[Run Engine / User Profile]
    I --> L[KPI Benchmarks]
    J --> L
  end
```

Component explanations

- *Data layer*: raw CSVs stay in `newdata/` (git-ignored); `src/ingestion` produces `movies_merged.csv`; `scripts/build_user_db.py` streams the 20M ratings into SQLite so a single user's history can be read quickly.
- *Engines*: the three modules are separate packages with their own tests.
- *Evaluation*: two complementary systems. `src/evaluation` is the original 23-KPI suite; `src/benchmark` is a stricter, frozen, hash-split benchmark used for the baseline comparison.
- *Presentation*: `app.py` is the Streamlit entry point and router; `src/ui/*` holds the screens.

3.6 End-to-end data flow



End-to-end data flow

```
flowchart LR
  R[rating.csv, movie.csv, link.csv, tag.csv] --> J[Join with TMDB on tmdbId]
  T[TMDB CSV] --> J
  J --> M[movies_merged.csv: text_chunk_clean]
  M --> V[TF-IDF matrix 5000 terms]
  R --> DB[(SQLite ratings)]
  DB --> UR[One user's ratings]
  UR --> FE[Profile features + inner backtests]
  FE --> CL[LightGBM strategy classifier]
  CL --> SW[Strategy + confidence -> source weights]
  V --> CG[Candidate generator]
  SW --> CG
```

```
  CG --> RK[Greedy six-objective ranker]
  RK --> OUT[Top-K movie cards + CSV + run log]
```

Read it as two phases. **Offline** (done once, or when data changes): ingestion, TF-IDF fitting, labelling users, training the classifier. **Online** (per request): load that user's ratings, build features, predict strategy, generate candidates, rank, display.

Chapter 4. Technology Stack

Only technologies that the repository actually imports or configures are listed. Versions are those installed in the development environment when this guide was written.

4.1 Stack at a glance

Layer	Technology (version)	Where it is used
Language	Python 3.14	Everything
Data handling	pandas 3.0, NumPy 2.5, SciPy 1.18	`src/ingestion`, features, sparse matrices (`csr_matrix`), statistics (`scipy.stats.binomtest`)
Classical ML	scikit-learn 1.9	TF-IDF (`TfidfVectorizer`), cosine similarity, Random Forest/Extra Trees, cross-validation, metrics
Gradient boosting	LightGBM 4.7 (deployed); XGBoost, CatBoost (experiments only)	`src/strategy_selector/estimators.py`, `scripts/selector_experiments.py`
Hyperparameter search	Optuna	`scripts/selector_experiments.py` (tuning only)
Imbalance handling	imbalanced-learn (SMOTE)	`src/strategy_selector/classifier.py` (disabled by default)
NLP	NLTK 3.10 (stop-words, Porter stemmer)	`src/ingestion/text_cleaner.py`
Configuration	PyYAML	`eval_config.yaml`, loaded by `ranking_engine/config.py`, `strategy_selector/config.py`, `benchmark/config.py`
Database	SQLite (Python `sqlite3`)	`src/storage/` (ratings, user stats, recommendation runs)
Web UI	Streamlit 1.64	`app.py`, `src/ui/`
System measurement	psutil	`src/evaluation/system_metrics.py`
Testing	pytest	`tests/`
Icons / fonts	Feather icons (inline SVG), Google Fonts via CSS `@import`	`src/ui/icons.py`, `src/ui/theme.py`

`requirements.txt` also lists `matplotlib` and `joblib`; no application module imports them (Matplotlib was part of the original plan in `CLAUDE.md`; this guide's figures use it). There is no separate JavaScript frontend, no REST API server and no cloud deployment in the repository: the "backend" is the Python code Streamlit runs.

4.2 Technology by technology

Python

What: a general-purpose language with a rich data-science ecosystem. *Why chosen:* the entire team workflow (pandas, scikit-learn, NLTK) is Python. *Alternatives:* R (strong statistics, weaker engineering/UI), Julia (fast, smaller ecosystem). *Trade-off:* Python is slow for per-row loops, which is why many functions here are vectorised.

pandas and NumPy

What: tabular data frames and n-dimensional arrays. *Where:* every pipeline stage. *How it works:* columnar storage with vectorised operations (the 20M-row ratings file is read in 1M-row chunks into SQLite). *Alternatives:* Polars or Spark (faster/bigger scale), Dask. *Why not here:* the working sets (400k ratings in memory for the app, 5,000 users for labelling) fit pandas comfortably.

SciPy sparse matrices

Ratings and TF-IDF are overwhelmingly zeros. A `csr_matrix` (compressed sparse row) stores only non-zero entries and supports fast row slicing and matrix-vector products. The TF-IDF matrix is saved with `scipy.sparse.save_npz`.

scikit-learn

Provides `TfidfVectorizer`, `cosine_similarity`, `StratifiedKFold`, `RandomForestClassifier`, `ExtraTreesClassifier`, `HistGradientBoostingClassifier`, metrics and calibration. *Why:* standard, well-tested, consistent API. *Alternatives:* PyTorch/TensorFlow (deep models; the project explicitly excluded them to remain explainable).

LightGBM

A gradient-boosted decision-tree library with leaf-wise tree growth and histogram binning; chapter 8 explains it from scratch. *Why selected:* among 14 candidate models/formulations evaluated with the same protocol, multiclass LightGBM had the best cross-validated macro-F1 (0.467) in the full comparison, though several models were within noise of it (Chapter 9). *Alternatives tested:* XGBoost, CatBoost, Random Forest, Extra Trees, HistGradientBoosting, stacking, voting.

NLTK

Used only for English stop-word removal and Porter stemming in `clean_text`. The stop-word list is downloaded on first import if absent. *Alternative:* spaCy (lemmatisation), which is heavier.

SQLite

A serverless SQL database stored in one file. *Why:* the app needs to read **one user's** ratings out of 20 million rows in milliseconds (an index on `(userId, ts)`), paginate and filter histories server-side, and store recommendation runs. Reading `rating.csv` per request would take many seconds. *Alternatives:* PostgreSQL/MySQL (better for multi-user deployments), DuckDB (fast analytics). *Limitation:* single-writer, local file; no authentication layer (the project has none, so none was added).

Streamlit

A Python framework that turns scripts into web apps; each user interaction re-runs the script top to bottom. State lives in `st.session_state`; expensive objects use `@st.cache_resource` (engine, SQLite connection, strategy context) and `@st.cache_data` (saved reports). *Alternatives:* Flask/FastAPI + React (more flexible, more code), Dash. *Trade-off:* simplest path from Python models to a UI; limited control over layout (the project injects custom CSS in `src/ui/theme.py`).

Optuna, XGBoost, CatBoost

Installed for the model-comparison experiments (`scripts/selector_experiments.py`). Only LightGBM's model is deployed.

psutil

Used by `measure()` in `src/evaluation/system_metrics.py` to read process memory and CPU time while timing operations.

4.3 What is not used

No deep-learning framework, no embedding model, no large language model, no message queue, no containerisation, no REST API. If asked "why not a neural network / LLM?" see Chapter 16, questions K12-K13.

Chapter 5. Datasets and Data Engineering

5.1 MovieLens

MovieLens is a research dataset collected by the GroupLens lab at the University of Minnesota: real people rating real movies on a movie-recommendation website. It is the standard benchmark for recommender systems, so results are comparable to the literature.

Why MovieLens 20M (documented in `DATASET_MIGRATION.md`): 20M ratings give enough users and overlap for collaborative filtering and classifier training; the whole file still fits on a laptop; ~27k movies is a manageable catalog; it has `link.csv` mapping to TMDB.

File	Rows	Columns
`rating.csv`	20,000,263	`userId, movieId, rating, timestamp` (rating 0.5-5.0 in half steps; timestamp is a datetime string such as `2005-04-02 23:53:47`)
`movie.csv`	27,278	`movieId, title, genres` (genres pipe-separated: `Adventure|Animation|Children|Comedy|Fantasy`; title contains the year: `Toy Story (1995)`)
`link.csv`	27,278	`movieId, imdbId, tmdbId` (252 movies have no `tmdbId`)
`tag.csv`	465,564	`userId, movieId, tag, timestamp` (free-text user tags such as "computer animation")

Observed in the SQLite copy: 138,493 distinct users, ratings from 0.5 to 5.0, every user has at least 20 ratings (a MovieLens rule; this is why there are **no natural cold-start users**, see 5.7). The files `genome_scores.csv` and `genome_tags.csv` ship with MovieLens but are not used.

A taste of the raw data (first rows of `rating.csv`):

```
userId,movieId,rating,timestamp
1,2,3.5,2005-04-02 23:53:47
1,29,3.5,2005-04-02 23:31:16
1,32,3.5,2005-04-02 23:33:39
```

5.2 TMDB

TMDB (The Movie Database) is a community-maintained movie database. The project uses the Kaggle export "TMDB Movies Dataset 2023", file `TMDB_movie_dataset_v11.csv` (in the copy used here: 1,515,783 rows; the migration notes quote 1,480,412 for the earlier download, an unexplained size difference between two downloads of the same dataset family: UNVERIFIED cause). Relevant columns: `id, title, overview, genres, keywords, vote_average, vote_count, popularity, release_date, original_language, poster_path`. There is no cast/crew data; the team decided not to backfill it from another file because only a minority of movies would have it, which would make the text feature inconsistent across the catalog.

Raw TMDB row for Toy Story (id 862), abbreviated:

```
title: Toy Story        vote_average: 7.971     vote_count: 17152
release_date: 1995-10-30   original_language: en   popularity: 78.404
genres: Animation, Adventure, Family, Comedy
keywords: rescue, friendship, mission, martial arts, jealousy, bullying, ...
overview: Led by Woody, Andy's toys live happily in his room until ...
```

5.3 Joining MovieLens and TMDB

`src/ingestion/joiner.py::join_movielens_tmdb(ml, tmdb)`:

1. Left-join MovieLens movies with `link.csv` on `movieId` to get `tmdbId` (converted to numeric).
2. **Inner-join** with the TMDB table on `tmdbId == id`. Movies without a match are dropped.
3. Parse TMDB genres and keywords (comma-separated text) into lists with `_split_names`.
4. Fill missing overviews with an empty string.
5. Build the raw text: `text_chunk = overview + " " + genres + " " + keywords`.
6. Keep columns: `movieId, tmdbId, title, genres, genre_names, keyword_names, overview, vote_average, vote_count, popularity, text_chunk` plus (added later for the UI) `release_date, original_language, poster_path`.

Join quality (verified from the pipeline output):

Count	Value
MovieLens movies	27,278
with a non-null `tmdbId`	27,026
successfully joined to TMDB	**26,743 (98.04%)**
movies with empty overview in the merged file	89
movies whose cleaned text is empty	11
duplicate `movieId` in the merged file	0

The 535 movies lost are those without a TMDB id (252) or whose id is not in the TMDB export. They never appear as candidates because everything downstream uses the merged catalog.

5.4 Text cleaning

`src/ingestion/text_cleaner.py::clean_text(text)`:

1. lowercase,
2. keep only alphabetic tokens with the regex `[a-zA-Z]+` (numbers and punctuation vanish),
3. remove English stop-words (NLTK list: *the, and, of...*),
4. **Porter-stem** each remaining token (*living -> live*, *happily -> happili*).

Result for Toy Story (`text_chunk_clean`, start): `led woodi andi toy live happili room andi birthday bring buzz lightyear onto scene afraid lose place andi heart woodi plot buzz ...`. Note the stems are not real words (*woodi*, *happili*): stemming chops suffixes by rule so that *toys / toy* and *plotting / plots* collapse together.

5.5 From movies to model-ready features

Two different "feature" worlds exist:

- **Movie features** (for retrieval): the TF-IDF vector of `text_chunk_clean` (Chapter 6, section B), plus genre sets, `vote_average`, `vote_count` and catalog popularity used by the objectives.
- **User features** (for the strategy classifier): numbers summarising one user's ratings (Chapter 8), such as average rating, number of ratings, genre concentration, popularity of what they watch, and "inner backtest" scores.

5.6 Splitting data (three different splits in the project)

Purpose	Where	Rule

Classifier labels	`holdout_labeler.split_profile_holdout`	Per user: 70% of ratings are the *profile* (features and strategy training), 30% are the hidden *holdout* (the labelling test)
Classifier evaluation	`scripts/selector_experiments.py`	Users (not ratings) are split 80/20, stratified by label; 5-fold CV inside the 80%; the 20% is scored once
Recommendation benchmark	`src/benchmark/split.py`	Each *rating* is assigned to train / validation / test by a hash of (userId, movieId, seed): 70% / 10% / 20%
Legacy KPI report	`recommendation_metrics.split_user_ratings`	Per-user random 80/20 split

The hash split is worth remembering for viva: because it uses a deterministic hash instead of random shuffling, the split is identical on every run, on every machine, regardless of row order or sample size, and needs no stored file.

5.7 Cold-start data and why it is simulated

MovieLens 20M admits only users with at least 20 ratings. The KPI "Cold-Start Accuracy" is defined for users with at most 5 ratings, which therefore **never occur naturally**. The evaluation code *simulates* cold users: real users whose profile is truncated to five ratings and who are scored on their untouched held-out items. The code records `mode: "simulated"` so a simulated number is never presented as natural.

5.7b Missing values and duplicates

- Missing overview: 89 movies (filled with empty string). Missing `tmdbId`: 252 movies (dropped by the inner join).
- Duplicate movie ids in the merged catalog: none. Exact duplicate user-feature rows: none (checked by `scripts/analyze_selector.py`: `exact_duplicate_rows = 0`, `missing_values = 0`).
- Highly correlated feature pairs ($|r| > 0.95$): 2; two redundant/constant features are dropped before training (`cf_cand_score_mean`, `rating_variance`).

5.8 Processed artifacts

File	Created by	Content
`data/processed/movies_merged.csv`	`python -m src.ingestion.build_dataset`	26,743 rows, 15 columns
`data/processed/ratings_clean.csv`	same	20,000,263 ratings (the app reads only the first 400,000 rows)
`data/processed/tfidf_*.pkl/.npz`	`scripts/verify_ir_engine.py`	vectorizer, sparse matrix, id/title metadata
`data/processed/cinerankx.db`	`scripts/build_user_db.py`	SQLite: `ratings`, `movies`, `user_stats`, `recommendation_runs`
`data/processed/strategy_classifier.pkl`, `strategy_context.pkl`, `strategy_eval.json`	selector scripts	model, reference population, evaluation results
`data/processed/ease_model.npz`, `cinerankx_model.json`	`scripts/run_benchmark.py`	EASE item-item weights and tuned blend
`reports/*.json`	KPI/benchmark scripts	saved results the dashboard reads

Chapter 6. The Recommendation Algorithms, Explained From Scratch

This chapter teaches each algorithm in four steps: intuition, mathematics, a small numerical example, and *what the CineRankX code actually does* (which sometimes differs from the textbook version). Code references are to the repository.

A. Collaborative Filtering (CF)

A.1 Intuition

People who agreed in the past tend to agree in the future. If you and another user rated ten movies similarly, then movies *they* loved and you have not seen are good candidates for you. CF needs **no information about the movies themselves**, only who rated what.

A.2 The user-item matrix and sparsity

Arrange ratings in a matrix *R*: one row per user, one column per movie, entry *R[u,i]* the rating (blank/0 if unrated). For MovieLens 20M the matrix is 138,493 x 27,278 = 3.78 billion cells but only 20,000,263 are filled: **0.53% density, 99.47% sparse**. Sparsity is the central difficulty of CF: two users rarely rate the same movies, so similarities are noisy.

A.3 Similarity: cosine

Treat each user as a vector of ratings and measure the angle between two vectors:

$$sim(u, v) = cos(r_u, r_v) = (r_u \cdot r_v) / (||r_u|| \, ||r_v||)$$

It is 1 when two users rate in proportion, 0 when they share no rated items. (Blanks counted as zero: a simplification the CineRankX code also makes.)

A.4 Worked example (also the textbook rating prediction)

Items A-E, four users; target user U1 has not rated D or E.

	A	B	C	D	E
U1	5	3	4	?	?
U2	4	0	5	3	0
U3	1	5	0	4	5
U4	0	4	1	5	4

Cosine similarities of U1 with the others: sim(U1,U2) = **0.800**, sim(U1,U3) = **0.346**, sim(U1,U4) = **0.297**.

Prediction by similarity-weighted average (user-based CF): for item D the neighbours who rated it are U2 (3), U3 (4), U4 (5):

$$r\hat{}(U1, D) = (0.800 \times 3 + 0.346 \times 4 + 0.297 \times 5) / (0.800 + 0.346 + 0.297) = 3.65$$

For item E: U3 (5) and U4 (4) give (0.346 x 5 + 0.297 x 4)/(0.346 + 0.297) = **4.54**. So E would be recommended before D.

Item-based CF does the same with item columns: cos(D, C) = 0.436 and cos(D, A) = 0.349, so items that co-occur with what U1 liked score higher.

A.5 What CineRankX actually implements

The CF source in `src/ranking_engine/candidates.py` (`CandidateGenerator._build_collaborative`, `_collaborative_candidates`) is a **user-user nearest-neighbour candidate generator**, not a rating predictor:

1. Take the first `_CF_MAX_USERS = 5,000` users in the ratings frame and their ratings of the `_CF_MAX_MOVIES = 2,000` most-rated movies (bounded so the similarity computation stays cheap).
2. Build the target user's vector from *their own ratings* (so users outside the pool still work), compute cosine against all pooled users, set a perfect self-match to -1.
3. Keep the `_CF_NEIGHBORS = 25` most similar users with positive similarity.
4. Each neighbour contributes their top `_CF_PER_NEIGHBOR = 20` highest-rated movies (from their *full* history, not just the 2,000-movie window). A candidate's score is the **sum of the similarities of the neighbours who propose it**.

$$score_CF(m) = sum\ over\ neighbours\ n\ with\ m\ in\ top20(n)\ of\ sim(u, n)$$

The strategy labeller has its own, slightly different, CF used only for labelling (`strategy_labeler._collaborative_hit_rate`: 5 neighbours x 5 movies, capped to the top-10 or top-30 by similarity so CF cannot win just by proposing more items than the others).

Advantages: simple, explainable, needs no metadata. **Disadvantages:** cold start; popularity-biased neighbours; bounded to a 5,000-user pool (a scalability compromise); no rating normalisation. **Potential viva questions:** why cosine and not Pearson? (Pearson removes each user's mean and is better for rating-scale bias; we kept cosine for sparse vectors and speed; mean-centring is a possible improvement.) Why a bounded pool? (the dense user-user matrix is O(n^2).)

A.6 EASE: the stronger collaborative model used in the benchmark

The benchmark pipeline (`src/ranking_engine/ease.py`) uses **EASE** (Embarrassingly Shallow Autoencoder, Steck 2019), a closed-form linear item-item model. Let X be the binary user-by-item matrix of training interactions (6,000 most-rated movies):

$$P = (X'X + lambda\ I)\text{^-1},\quad B = -P / diag(P),\quad diag(B) = 0,\quad score(u) = x_u\ B$$

Intuition: for each item, ridge-regress its column on all other columns (excluding itself); *B[j,i]* says how strongly owning item *j* predicts owning item *i*. The diagonal is forced to zero so an item cannot trivially predict itself. A user is scored from their own ratings alone (fold-in), including a cold-start user with five ratings. The tuned regulariser is lambda = 1,500. Scores are converted to per-user **percentile ranks** and blended with a popularity percentile with weight beta (1.0 overall, 0.8 cold-start) before the six-objective ranker runs.

An exploratory PureSVD (truncated SVD) run was tried while choosing a relevance model and discarded in favour of EASE; the repository contains no SVD implementation.

B. Content-Based Filtering

B.1 Intuition

"You liked *Toy Story*, so here are movies that *read* like it." It uses only item descriptions, so it works for new items and needs no other users, but cannot discover tastes outside what you already like.

B.2 TF-IDF

Turn each movie's text into a numeric vector. For term t in document d of a collection of N documents:

$$tf(t,d) = count\ of\ t\ in\ d$$

$$idf(t) = ln((1 + N) / (1 + df(t))) + 1$$

$$tfidf(t,d) = tf(t,d) \times idf(t), \quad \text{then L2-normalise the vector}$$

Term frequency rewards words used often in a document; *inverse document frequency* down-weights words that appear in many documents (low discriminating power). The +1 and smoothing are scikit-learn's defaults (`smooth_idf=True`).

Worked example. Three tiny documents: d1 = "toy story toy adventure", d2 = "space adventure laser", d3 = "toy robot space". Vocabulary: adventure, laser, robot, space, story, toy. Document frequencies: adventure 2, laser 1, robot 1, space 2, story 1, toy 2, so idf(adventure) = ln(4/3) + 1 = 1.288, idf(laser) = ln(4/2) + 1 = 1.693.

d1: toy 2 x 1.288 = 2.576, story 1.693, adventure 1.288 -> after L2 normalisation (length 3.34): toy **0.771**, story **0.507**, adventure **0.386**.

B.3 Cosine similarity between movies

After normalisation, the cosine similarity is just the dot product. Here cos(d1,d2) = **0.200** (shared word: adventure), cos(d1,d3) = **0.399** (shared word: toy, which d1 uses twice), cos(d2,d3) = **0.268** (shared word: space).

B.4 What CineRankX implements (Module 2)

`src/ir_engine/tfidf_vectorizer.py::TFIDFEngine` fits a `TfidfVectorizer(max_features=5000, min_df=2, max_df=0.8, ngram_range=(1, 2), strip_accents='unicode', lowercase=False)` on `text_chunk_clean`:

- `min_df=2` drops terms occurring in fewer than 2 movies (typos, one-off names); `max_df=0.8` drops terms in more than 80% of movies; `ngram_range=(1,2)` adds two-word phrases; `max_features=5000` keeps the most frequent 5,000 terms.
- `search(query)`: vectorise the query with the same vocabulary, cosine against every movie, return top-k with positive similarity.
- `similar_to(movie_id)`: use the movie's own row as the query, exclude itself.
- `src/ir_engine/trie.py::Trie`: a prefix tree over lower-cased titles for autocomplete; `search(prefix)` walks to the prefix node and does a depth-first collection of up to `limit` titles.

B.5 How content-based candidates are built for a user

`CandidateGenerator._content_candidates`: take the user's 10 highest-rated movies as **seeds** (`_IR_SEEDS = 10`); for each seed fetch 30 similar movies; weight each neighbour as `similarity x rating x gate`, where the **genre-overlap gate** is

$$gate = floor + (1 - floor) \times Jaccard(genres(candidate), genres(seed)), \quad floor = 0.25$$

(`genre_overlap_gate`). The gate fixes a real failure: *Toy Story*'s TF-IDF neighbours included *Silent Night, Deadly Night 5: The Toy Maker* purely because of the word "toy". The score of a candidate is the maximum over seeds.

B.6 The labeller's content strategy

In the backtest used to create classifier labels (`strategy_labeler._content_based_hit_rate`) a user's profile is the **rating-weighted sum of the TF-IDF rows of their 20 best-rated movies**, compared by cosine with every catalog movie; the top-N (10, later 30) unseen movies are the recommendation. This strategy's mean hit-rate is only **0.016**, against about 0.115 for collaborative and popularity (Chapter 9).

B.7 Experiments to improve it (Chapter 9, section 9.9)

A richer representation (bigram TF-IDF with 20,000 terms, `src/strategy_selector/content_model.py`) raised the held-out hit-rate from 0.0156 to 0.0246 and adding a mild quality prior raised it to 0.0452 (about 2.9 times better, 95% CI of the gain [+0.027, +0.033]); it remains far below popularity (0.119).

C. Popularity-Based Recommendation

C.1 Intuition

Recommend what most people rated. It needs no personal data, is hard to beat offline, and is therefore both our fallback strategy and the **frozen baseline** for fair comparison.

C.2 Raw popularity: rating counts

In the repository popularity means **number of observed ratings** (not TMDB's `popularity` column). `CandidateGenerator._build_popularity` keeps the 200 most-rated movies (`_POPULARITY_POOL = 200`); the labeller keeps the top 10 or 30; the frozen baseline ranks *all* movies by count (ties broken by `movieId`) and removes what the user has seen. This was a deliberate correction (commit `c273ba3`): TMDB's popularity is live and recency-weighted while MovieLens ends in 2015.

C.3 Quality-adjusted popularity (the "popularity/quality" objective)

Average rating alone is unreliable: a film with three 10/10 votes would beat one rated 8.4 by 20,000 people. `CatalogStats._build_popularity_quality` uses the IMDb-style **Bayesian weighted rating**:

$$WR = v/(v+m)\ x\ R + m/(v+m)\ x\ C$$

where *R* is the movie's average (`vote_average`), *v* its vote count, *C* the **unweighted mean** rating of movies that have votes, and *m* the **median** vote count among voted movies. The values are then min-max scaled to [0, 1] because they bunch in a narrow band.

Example (C = 6.5, m = 1,000): a classic with R = 8.4 and v = 20,000 gets WR = 8.31; an obscure film with R = 9.5 and v = 3 gets WR = 6.51 (pulled almost to the mean); a mid film R = 7.0, v = 1,000 gets 6.75.

C.4 Popularity bias

Popular items get more exposure, therefore more ratings, therefore appear more popular. A recommender that leans on popularity concentrates on a few titles (low catalog **coverage**). This is why novelty and coverage are explicit objectives. In our evaluation the frozen baseline covers only 0.38% of the catalog in top-10 lists, while CineRankX covers 5.7% (Chapter 10).

D. Predicted Rating (the bias baseline)

`src/ranking_engine/predictor.py::BaselinePredictor` predicts a user's rating of a movie as

$$r\hat{}(u,i) = mu + b_u + b_i, \quad \text{clipped to } [0.5, 5.0]$$

with damped biases, fitted in two steps: first the item bias *b_i = sum(r - mu) / (n_i + lambda)* then the user bias on the item-adjusted residual, with damping lambda = 10 to shrink sparse users/items toward 0. An unknown user falls back to mu + b_i; an unknown item to mu + b_u. In the benchmark this predictor has RMSE 0.846 / MAE 0.649 against 0.936 / 0.725 for a frozen item-mean baseline.

E. Hybrid recommendation: what is and is not implemented

Concept	Implemented?	Where
Weighted blend of three candidate *sources*	**Yes**	`CandidateGenerator.generate`: each source's scores are min-max normalised and summed with strategy-dependent weights
Strategy prediction steering the blend	**Yes**	`strategy_source_weights(strategy, confidence)`

Multi-objective re-ranking of the blended pool	**Yes**	`MultiObjectiveRanker.rank`
Hard switching to exactly one strategy	**No** (the prediction is a weighted prior)	
Fusion of complete ranked lists from strategies	No (identified as future work)	

Source blending detail. For each source, scores are min-max normalised to [0, 1] (otherwise popularity's raw counts would swamp cosine similarities), multiplied by that source's weight, and **summed**, so an item proposed by two sources outranks one proposed by a single source at the same strength. The pool is then normalised again and cut to 200 candidates.

F. Summary table

Algorithm	Needs history?	Needs metadata?	Strength	Weakness	Code
Popularity	no	no	strong offline, cold-start safe	no personalisation, popularity bias	`candidates._build_popularity`, `benchmark/baseline.py`
User-kNN CF	yes	no	personalised, serendipitous	sparse, cold start, bounded pool	`candidates._collaborative_candidates`
EASE	yes (any amount)	no	strongest accuracy in our benchmark	needs top-N item cut; no content	`ranking_engine/ease.py`
TF-IDF content	seeds only	yes	works for new items	over-specialised; weakest strategy here	`ir_engine`, `candidates._content_candidates`
Bias predictor	yes	no	gives rating estimate	not personalised beyond biases	`ranking_engine/predictor.py`

G. Likely viva questions for this chapter

Chapter 16 sections E-I contain model answers. Practise: *What is cosine similarity and why is it preferred over Euclidean distance for TF-IDF? What does IDF do? Why do you min-max normalise each source before blending? Why is a Bayesian average better than a plain average? Why is popularity a strong baseline?*

Chapter 7. The Six Recommendation Objectives

All six objectives are fully implemented in `src/ranking_engine/` (`objectives.py`, `predictor.py`, `ranker.py`, `config.py`). Each objective produces a score in **[0, 1]** for a candidate movie so that the six numbers can be combined with weights that mean what they say.

7.1 How the ranker combines them

The weights live in `eval_config.yaml` (`ranking_objectives.weights`) and are normalised to sum to 1 by `RankingWeights`:

Objective	Default weight
relevance	0.30
diversity	0.15
novelty	0.15
coverage	0.10
popularity_quality	0.15
predicted_rating	0.15

For a candidate *m*, given the list *S* selected so far:

$$Score(m \mid S) = \text{sum over } o \text{ in six objectives of } w_o \times obj_o(m, S)$$

Four objectives (relevance, novelty, popularity/quality, predicted rating) depend only on the movie and user, so they are computed once. **Two objectives depend on what is already in the list**: diversity and coverage. That is why `MultiObjectiveRanker.rank` is a **greedy sequential selection**: at every step it rescores all remaining candidates against the current list, picks the best, appends it, and repeats until K movies are chosen. Sorting once by a fixed score would be wrong for these two objectives. The per-objective values stored with each result are the ones that existed *at the moment of selection*, which makes the score breakdown shown in the UI auditable.

7.2 Objective 1: Relevance

- **Meaning:** how well the movie matches this user. **Why it matters:** the primary accuracy signal.
- **Calculation:** the candidate's blended score from `CandidateGenerator.generate`: each source's raw scores min-max normalised to [0, 1], multiplied by a strategy-dependent source weight and summed, then normalised again. In the benchmark pipeline the relevance is the min-max normalised EASE/popularity percentile within the 200-candidate pool.
- **Data required:** user ratings, TF-IDF matrix, neighbour ratings, rating counts.
- **Trade-off:** maximising relevance alone yields repetitive lists (nine sequels of the same franchise) and overrepresents popular titles.

7.3 Objective 2: Diversity

- **Meaning:** how unlike the movies already selected this one is. Defined by the **MMR** (maximal marginal relevance) idea:

$$diversity(m \mid S) = 1 - \max_{s \in S} sim(m, s), \quad diversity = 1 \text{ when } S \text{ is empty}$$

- **Why max, not mean:** one near-duplicate is redundant even if the movie differs from the other nine; a mean would dilute that.

- **Similarity source:** `MultiObjectiveRanker._similarity_lookup`: cosine similarity of the TF-IDF vectors of the candidates; for candidates missing from the matrix it falls back to **genre Jaccard** so unknown items are not treated as "maximally diverse".
- **Interaction:** pulls against relevance (the most relevant items resemble each other).

7.4 Objective 3: Novelty

- **Meaning:** how unexpected an item is, measured by **self-information**: rare items carry more information.

$$novelty(m) = minmax(-log2\ p(m)),\quad p(m) = observed(m) / sum\ of\ observed(all\ movies)$$

where `observed = MovieLens rating count + TMDB vote_count` (combining both keeps the measure defined for the whole catalog; counts below 1 are floored at 1). Min-max scaling maps the range to [0, 1].

Example. Three movies with 1,000, 100 and 10 observations: p = 0.901, 0.090, 0.009, so -log2 p = 0.151, 3.47, 6.79 and the scaled novelty is 0.00, 0.50, 1.00.

- **Why it matters:** counteracts popularity bias, supports discovery. **Tension:** deliberately opposed to popularity/quality.

7.5 Objective 4: Coverage

- **Meaning (per item):** how much of the catalog's *genre space* this movie opens up that the current list does not already cover:

$$coverage(m \mid S) = |genres(m)\ minus\ covered\ genres| / |genres(m)|$$

An item whose genres are all already covered scores 0; one with no genre metadata scores 0 (it cannot be shown to broaden coverage).

- **Honest note:** the catalog-level *Coverage* **KPI** (share of the whole catalog that ever appears in any user's list) cannot be a per-item score; the objective is a **per-item proxy** for it, as stated in the docstring of `coverage_score`. This is a *partial* implementation of the idea of catalog coverage and should be described that way in a viva.

7.6 Objective 5: Popularity / Quality

- **Meaning:** how well-regarded the movie is by many people, shrunk for reliability, using the Bayesian weighted rating of section 6.C.3, then min-max scaled.
- **Difference between popularity and quality:** *popularity* is how many people engaged (a blockbuster with mediocre reviews is popular); *quality* is how highly they rated (a small gem can be high quality, low popularity). The Bayesian weighted rating blends the two: a high average with many votes scores best; a high average from three votes is pulled toward the mean.
- **Interaction:** pulls against novelty; resolving that tension is exactly what the weights are for.

7.7 Objective 6: Predicted Rating

- **Meaning:** the rating we expect this user to give, from the bias model (section 6.D), rescaled to [0, 1] as

$$predicted(u, m) = (r\hat{}(u,m) - 0.5) / (5.0 - 0.5)$$

- **Why rescaled:** the other five objectives are already in [0, 1]; leaving the predictor on a 0.5-5 scale would silently let it dominate.
- **Limitation:** it only knows the global mean and user/item biases, so it cannot say "this user prefers thrillers"; that personalisation comes from relevance.

7.8 A worked greedy example (illustrative numbers)

Weights: relevance .30, diversity .15, novelty .15, coverage .10, popularity/quality .15, predicted rating .15. Three candidates (numbers are made up to show the mechanics):

Candidate	Relevance	Novelty	Pop/Quality	Pred. rating	Genres
A: *Toy Story 2*	0.95	0.20	0.70	0.85	animation, comedy
B: *Toy Story 3*	0.93	0.25	0.72	0.84	animation, comedy
C: *Heat*	0.60	0.45	0.65	0.75	action, crime

Step 1 (list empty: diversity = 1, coverage = 1 for everyone): A = 0.3(0.95) + 0.15(1) + 0.15(0.20) + 0.10(1) + 0.15(0.70) + 0.15(0.85) = **0.7975**; B = **0.8005**; C = 0.18 + 0.15 + 0.0675 + 0.10 + 0.0975 + 0.1125 = 0.7075. **B is picked.**

Step 2 (list = {B}; assume sim(A,B) = 0.8 and sim(C,B) = 0.05): A: diversity 0.2, coverage 0 (genres already covered) -> 0.285 + 0.03 + 0.03 + 0 + 0.105 + 0.1275 = **0.6175**. C: diversity 0.95, coverage 1 -> 0.18 + 0.1425 + 0.0675 + 0.10 + 0.0975 + 0.1125 = **0.7000**. **C is picked**, although A is far more relevant. That is the multi-objective ranker doing its job: the second slot goes to something different.

7.9 Why not maximise relevance alone?

Because the highest-relevance items are mutually similar and popular. The project measured the effect directly (README, "Weight sensitivity"): raising diversity 0.15 -> 0.60 and coverage 0.10 -> 0.40 lifted mean diversity 0.952 -> 0.972 and genre spread 13 -> 16 distinct genres, at the cost of mean relevance 0.814 -> 0.657. Because that trade-off is real, the weights are configurable and exposed as sliders in the Run Engine page.

7.10 Tuning the weights (benchmark)

For the benchmark pipeline the weights were *searched* on the validation split (`src/benchmark/tuning.py::tune_weights`): relevance weight sampled in [0.5, 0.95], the rest by a Dirichlet draw, maximising validation NDCG@10 subject to **diversity not falling below the frozen baseline's diversity**. Tuned overall weights: relevance 0.576, diversity 0.064, novelty 0.102, coverage 0.117, popularity/quality 0.110, predicted rating 0.032 (`data/benchmark/cinerankx_params.json`).

7.11 Genre-overlap relevance gate (not a seventh objective)

The gate of section 6.B.5 is a *pre-ranking filter on IR relevance*, tunable by `ranking_objectives.genre_relevance_gate.floor` (default 0.25). It exists because TF-IDF measures shared vocabulary, not shared tone.

Chapter 8. The Strategy Classifier: Deep Technical Explanation

8.1 Why a classifier at all?

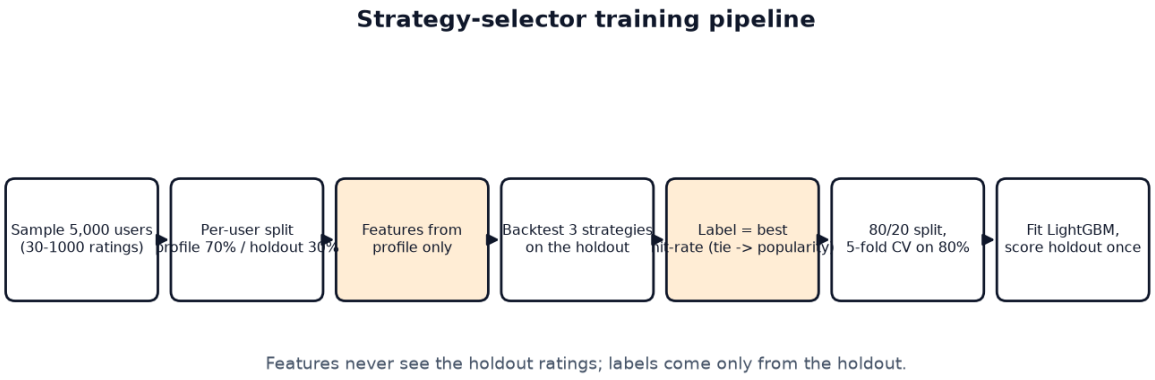
Because no strategy dominates. If the best strategy differed *randomly* per user a classifier could not help; if it differs *systematically* (with history size, taste concentration, neighbourhood strength) then features describing the user can predict it, and the right strategy can be favoured for that user. The classifier's job is to estimate that relationship.

8.2 What it predicts

Three classes, encoded alphabetically (`LabelEncoder`): **0 collaborative, 1 content_based, 2 popularity**. Output: a probability for each class. `confidence = max probability`, and the predicted strategy is the argmax. These two numbers feed `strategy_source_weights` (section 3.4).

8.3 How the training labels are made (the most important idea)

A label must say "which strategy actually worked best for this user". CineRankX answers by **backtesting** each strategy on the user's own hidden data:



Training pipeline

1. **Sample users.** `dataset.sample_user_ids`: 5,000 random users (seed 42) from `user_stats` with 30 to 1,000 ratings (`min_history`, `max_history`).
2. **Split each user's ratings** (`holdout_labeler.split_profile_holdout`): 70% **profile** (visible), 30% **holdout** (hidden); at least one rating on each side.
3. **Build the three strategies from the profile data only** (`strategy_labeler._BacktestContext`): the popularity list = the most-rated movies in the profile data; the CF similarity structures from the profile ratings; the TF-IDF matrix for content.
4. **Recommend N movies per strategy** (N = `label_top_n` = 30) for each user, excluding movies already in the profile.
5. **Score**: hit-rate = (number of recommended movies that appear in the user's holdout) / (number of holdout movies).
6. **Label** = the strategy with the highest hit-rate; ties go to popularity (`formulations.winner`), because a tie most often means all scores are 0 and popularity is the safe default.

> **The two disjoint worlds.** Features are computed from the *profile* ratings only; labels from the *holdout* only. If features could see the holdout the model could "cheat". Tests `test_features_use_only_the_ratings_given` and `test_holdout_labels_never_use_holdout_ratings` guard this.

8.4 The features (47 after removing 2 redundant ones)

Computed by `profile_features.extract_profile_features` (vectorised over users) and `strategy_context.build_features`. One example row from `strategy_labels_*_cbase.csv` is user 104 (698 ratings, average 2.84).

Group	Features	Intuition
Activity	`num_ratings`, `log_num_ratings`	more history => CF/content have evidence
Rating behaviour	`avg_rating`, `rating_std`, `rating_min`, `rating_max`, `rating_variance`*, `liked_share`, `user_bias`	harsh vs generous raters; `user_bias` = mean(user rating - item mean rating)
Genre structure	`num_genres`, `genre_diversity`, `top_genre_share`, `genre_entropy`, 19 x `affinity_<Genre>`	focused fans suit content; eclectic users do not. `genre_entropy` is Shannon entropy of the genre distribution scaled to [0,1]
Popularity of taste	`niche_fraction`, `mean_popularity_pct`, `top100_share`, `top1000_share`, `profile_surprisal`	users who watch only blockbusters are well served by popularity
Time	`log_active_span_days`, `recent_activity_share`	sustained vs one-sitting raters
Engine outputs	`cb_top_cos_mean`, `cb_coherence`, `cf_top_sim_mean`, `cf_top_sim_max`, `cf_cand_score_mean`*	how strongly the content and neighbourhood engines "pull" for this user
Strategy-aware ("inner backtest")	`inner_cb`, `inner_cf`, `inner_pop`, `inner_gap_cb_pop`, `inner_gap_cf_pop`	split the *visible* profile again (3 splits, 25% pseudo-holdout) and measure each strategy's hit-rate, an estimate of the same quantity the label measures

(* removed as redundant, |correlation| > 0.98.) The most influential features by mean absolute TreeSHAP value on the held-out users are `cf_top_sim_max`, `cf_top_sim_mean`, `top100_share`, `profile_surprisal`, `cb_coherence`, `inner_cf`, `top_genre_share`, `affinity_Sci-Fi`.

8.5 Training, validation and testing protocol

(`scripts/selector_experiments.py`, `scripts/selector_phase2.py`)

- **All users are kept** (4,993). The earlier practice of dropping "ambiguous" users was removed because it made accuracy look better than it was.
- **Stratified 80/20 split by label** (seed 42): 3,994 development users and 999 held-out users. Held-out users are *never* used to choose anything.
- **5-fold stratified cross-validation** on development users for every candidate; model choice by CV macro-F1.
- **Hyperparameters** tuned by Optuna on development users (TPE sampler, 3-fold macro-F1).
- **Final**: the winner is refit on development users and scored once on the held-out users; the *deployed* model is then refit on all 4,993 users (so the held-out numbers describe a model that never saw those users).

Deployed LightGBM hyperparameters (`selector_experiments_inner3.json`): 357 trees, learning rate 0.0219, 27 leaves, minimum 50 samples per leaf, row subsample 0.60 (every iteration), column subsample 0.463, L2 regularisation 0.0035, plus **sample weights** = balanced class weight x (1 + 3.50 x min(label margin / 0.1, 1)), which gives clearer labels more influence while still using every user.

8.6 LightGBM explained from scratch

Decision tree. A tree asks a sequence of yes/no questions about features ("is `top100_share` < 0.1?") and ends in leaves holding a value. It is easy to read but unstable and overfits.

Gradient boosting. Build an *ensemble* of small trees one at a time, each correcting the mistakes of those before it:

$$F_m(x) = F_{(m-1)}(x) + eta \times f_m(x)$$

eta is the **learning rate**; smaller values need more trees but generalise better.

Loss and gradient/Hessian (multiclass). The model outputs a raw score per class; probabilities are the **softmax**: p_k = exp(F_k) / sum_j exp(F_j). The loss is the log-loss (cross-entropy), -sum over classes of y_k log p_k. For each sample and class, boosting needs the first derivative (**gradient**) and second derivative (**Hessian**) of the loss with respect to the raw score:

$$g_k = p_k - y_k, \quad h_k = p_k (1 - p_k)$$

Intuition: the gradient is "how wrong, and in which direction"; the Hessian says "how confident/curved", and is used to scale the step.

Splitting and leaf value. For a set of samples in a leaf, with *G* = sum of gradients and *H* = sum of Hessians, the optimal leaf value and the quality of a split are

$$w^* = - G / (H + lambda)$$

$$Gain = 1/2 [G_L^2/(H_L + lambda) + G_R^2/(H_R + lambda) - (G_L + G_R)^2/(H_L + H_R + lambda)]$$

lambda is the L2 regularisation (`reg_lambda`), which shrinks leaf values and stops tiny leaves from dominating.

What makes LightGBM different. (1) **Histogram binning**: continuous features are bucketed into at most 255 bins, so searching for splits is fast. (2) **Leaf-wise growth**: instead of growing every level evenly, it always splits the single leaf with the largest gain, producing deeper, more accurate but more overfit-prone trees; `num_leaves` and `min_child_samples` are the main controls. (3) Optional GOSS/EFB speed-ups (not configured here).

Overfitting and regularisation. A model that memorises training noise has high training accuracy and lower test accuracy (here: train 0.77-0.88 vs test 0.61-0.63). Controls: small learning rate, limited leaves, minimum leaf size, row and column subsampling, L2 regularisation, cross-validation for model selection.

8.7 Prediction path

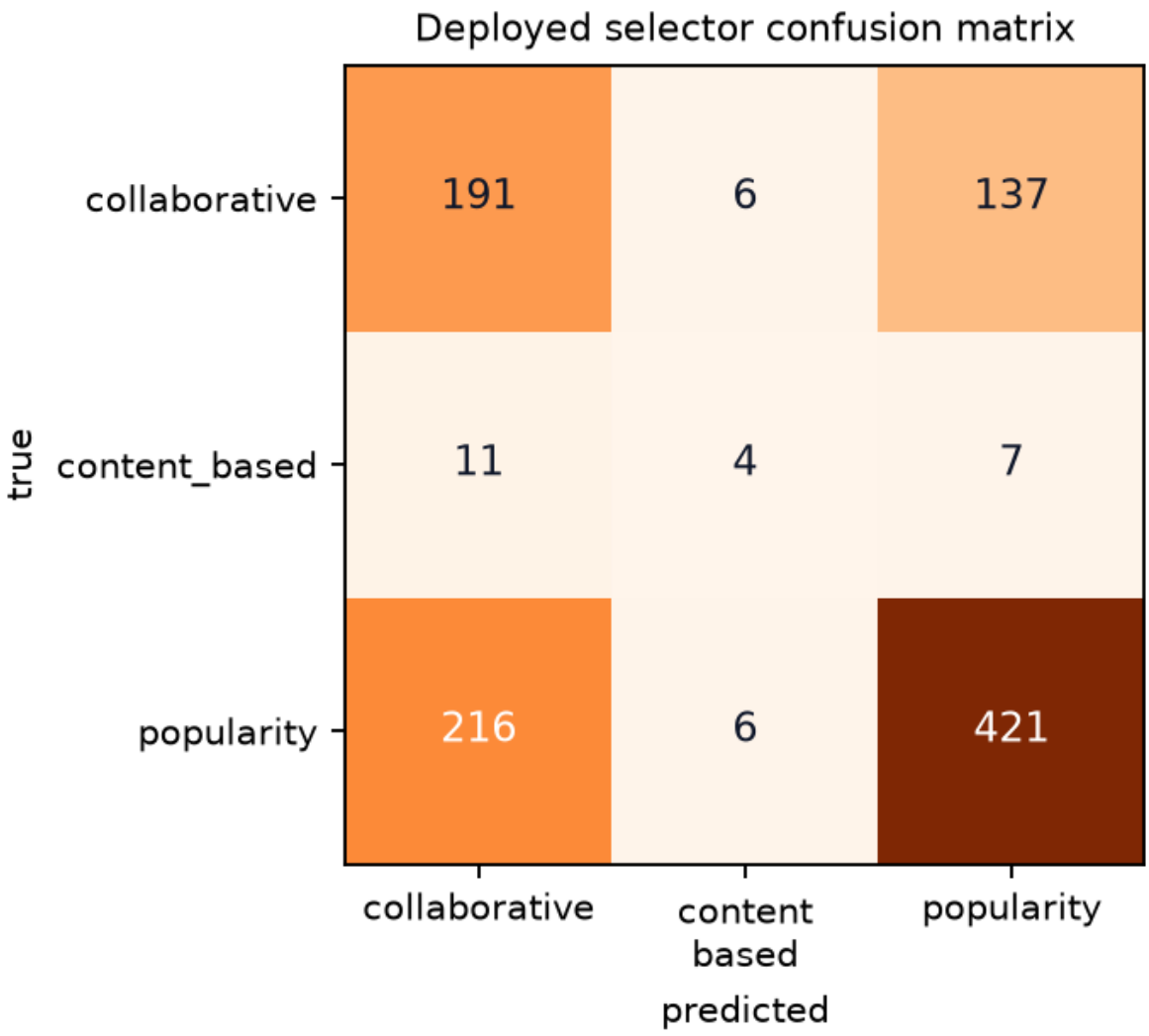
`app.py::recommend` -> `strategy_context()` (cached reference population) -> `build_features(user_ratings, movies, context, ...)` -> `classifier.predict(features)` returns `predicted_strategy` and `confidence`. `StrategyClassifier.predict` runs one forward pass (`predict_proba`), takes the argmax class and its probability. Latency: the saved evaluation (`strategy_eval.json`) records a mean of 79 ms per user for features plus prediction (p95 87 ms); an isolated measurement split it into roughly 72 ms of feature building and 17 ms of prediction.

8.8 Other approaches genuinely tested

(Chapter 9 has the tables.) Multiclass with six model families (LightGBM, XGBoost, CatBoost, Extra Trees, Random Forest, HistGradientBoosting); one-vs-rest; pairwise comparison (three binary classifiers); regression of expected hit-rate per strategy ("utility"); LambdaRank over the three strategies; soft-voting and stacking ensembles; sigmoid probability calibration; per-class threshold optimisation; SMOTE (early work, disabled); an uncertainty-aware utility selector with a popularity fallback margin (`src/strategy_selector/utility.py`).

8.9 Confusion matrix and feature importance

Confusion matrix of the deployed model on 999 held-out users (rows true, columns predicted):



Confusion matrix

true \ predicted	collaborative	content_based	popularity
collaborative	191	6	137
content_based	11	4	7
popularity	216	6	421

Per-class precision / recall / F1: collaborative 0.46 / 0.57 / 0.51 (334 users); content_based 0.25 / 0.18 / 0.21 (22 users); popularity 0.75 / 0.65 / 0.70 (643 users). Read it as: popularity and collaborative are partly separable; content_based is almost never the winner, so there is little to learn from.

Chapter 9. The Classifier Accuracy Investigation

This chapter explains why the strategy classifier scores modestly, what was tried, and how to talk about it honestly. Every number below was read from a saved artifact (file names in brackets) and can be regenerated with the scripts named.

9.1 Reproducing the headline numbers

Quantity	Value	Source artifact
Held-out accuracy (999 users)	**0.617** (0.6166)	`data/processed/strategy_eval.json`, `selector_phase2_base.json`
Majority-class baseline (always "popularity")	**0.644** (0.6436)	same
Macro-F1	**0.472**	same
Balanced accuracy (macro recall)	**0.469** (chance = 0.333)	same
ROC-AUC (one-vs-rest, macro)	**0.735**	same
Selector realised hit-rate	**0.128** (0.1276; 95% CI [0.121, 0.135])	`selector_phase2_base.json`
Always-popularity hit-rate	**0.112** (0.1125)	same
Oracle hit-rate (upper bound)	**0.148** (0.1482)	same
Training accuracy of the same model	0.765 (analysis run) to 0.876 (earlier run)	`selector_analysis.json`, `selector_experiments.json`

Discrepancies worth knowing (an examiner may compare reports):

1. *The accuracy of the same configuration has been reported as 0.617, 0.634 and 0.636 in different runs.* The held-out set has only 999 users and just 22 content-based users; run-to-run variation of about +/-0.02 in accuracy and +/-0.015 in macro-F1 is normal at this size. The exact internal source of the variation between runs is **UNVERIFIED**; the saved, deployed result is 0.617.
2. *The dashboard's "F1 = 0.623" is the weighted** F1, while the "macro-F1 = 0.472" is the unweighted mean.* They answer different questions (section 9.8).
3. *Label counts differ slightly between two scripts* (collaborative 1,668 / content 113 / popularity 3,212 in `formulations.winner`; 1,617 / 164 / 3,212 in the legacy `select_best_strategy`). The only difference is the tie rule when content-based and collaborative tie above popularity (`winner` prefers collaborative; the legacy function prefers the first tied key, content-based). Popularity is identical in both.
4. *The earlier accuracy of 0.636 versus a 0.550 baseline* was measured after dropping about 30% of users whose winner was ambiguous. That subset has a different class mix. It was replaced by the "no user dropped" evaluation above because dropping hard cases flatters accuracy.

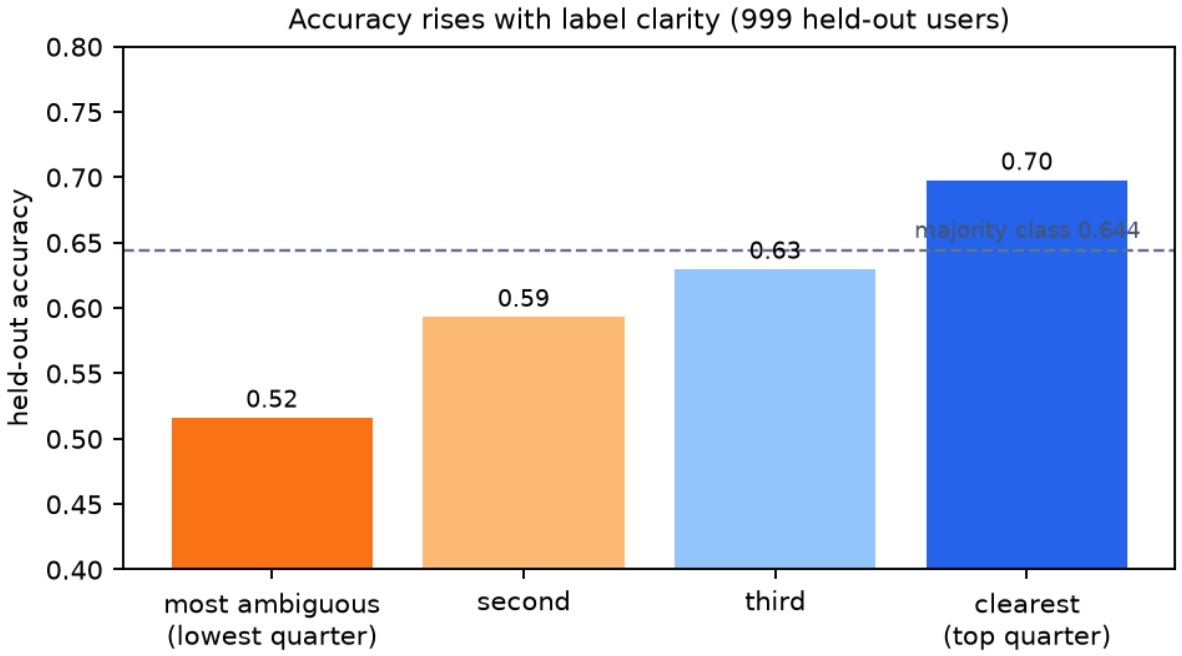
9.2 Why the accuracy is low: the ranked root causes

Cause 1 (largest): the labels are noisy

A user's label is the argmax of three **sample** hit-rates computed on their hidden holdout, which holds on average 44 movies (min 9, median 26, max 299). A hit-rate from so few items is a noisy estimate. Evidence:

- **Winner inside the noise.** Treat each hit-rate as a proportion with standard error *SE = sqrt(p(1-p)/m)* where *m* is the holdout size. For **60.3%** of users the gap between the best and second-best strategy is smaller than one combined standard error (`winner_within_1_standard_error`).
- **Label agreement.** Labelling the *same* 3,000 users twice with two independent random holdouts gives the same label only **65.3%** of the time (old 10-recommendation / 20% setting and current 30 / 30% setting) and **73.3%** with a larger 50-recommendation / 40% holdout (`label_experiments.json`).

- **Accuracy ceiling.** Simulate an oracle that knows each user's *true* hit-rates (taking the observed ones as truth) and is then scored against a freshly noisy label: its expected accuracy is only **0.761** (`noise_ceiling_full_population`; 0.825 / 0.768 / 0.793 for the three labelling settings).
- **Accuracy tracks label clarity** (figure below): the quarter of held-out users with the most ambiguous labels is classified correctly 52% of the time; the clearest quarter 70%.



Accuracy by label clarity

> **Plain-language version for the viva:** *"We ask the model to guess a label that is itself only about two-thirds reproducible. Even a perfect model would be marked wrong on the noisy third."*

Cause 2: the information gap

The distance between the ceiling (0.76) and what we reach (0.62-0.65) is the part the available features do not explain. Features describing the user's taste and history are only mildly informative about which backtest happens to win. Nearest-neighbour check: in the closest 10% of user pairs in feature space, the labels disagree 31% of the time against 48% expected by chance, so the features do carry real signal but far from enough.

Cause 3: content-based is a rare and weak strategy

Mean strategy hit-rates over 4,993 users: collaborative 0.115, popularity 0.116, content-based **0.016**; the oracle (best of three per user) 0.152. Content-based wins for only 2.3% of users (113), so the held-out set has just 22 of them. A class with so few examples is almost unlearnable; this was already noted in `DATASET_MIGRATION.md` (SMOTE could not fix it because it interpolates between the same few real points).

Cause 4: overfitting, which is visible

Training accuracy (0.77-0.88) is far above cross-validated (0.61-0.63) and held-out accuracy. Boosted trees can memorise label noise; regularisation limits but cannot remove it.

Cause 5: a strong majority class

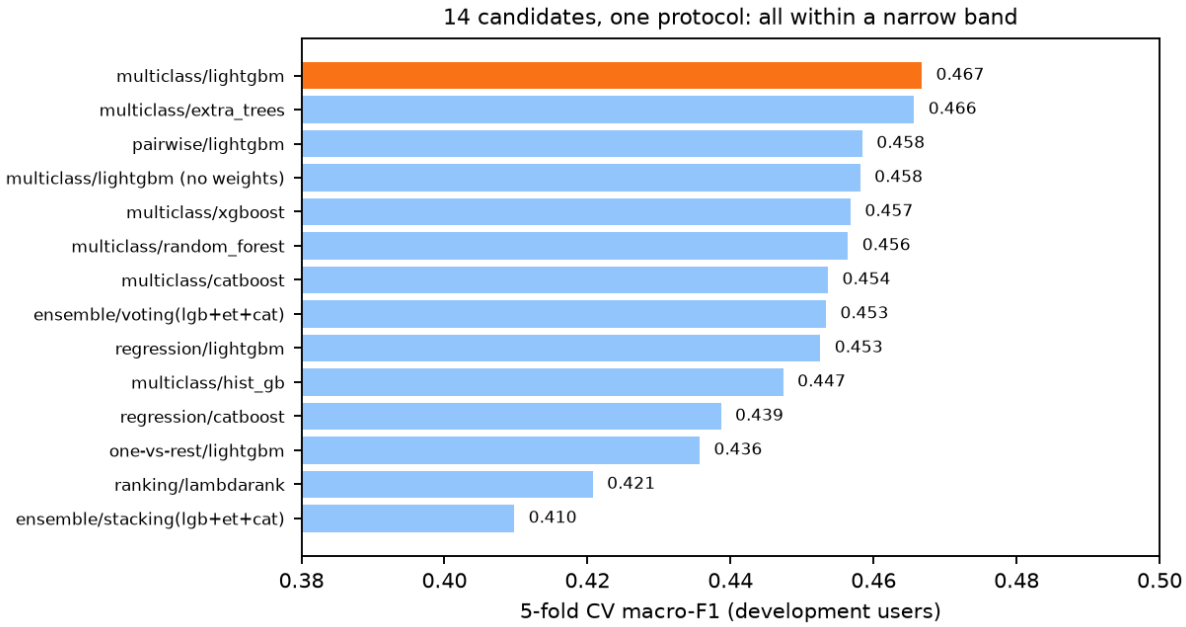
64.4% of users are labelled popularity. A classifier that learns anything useful about the minority classes must sacrifice some majority accuracy, so plain accuracy can fall below 0.644 even while macro-F1, balanced accuracy and AUC rise.

9.3 Why the majority baseline can beat the classifier on accuracy

Accuracy counts every user equally; if 64% of users are one class, "always predict that class" scores 64% while being useless for the other 36%. On the held-out users the deployed-style model predicts popularity for 67% of users, collaborative for 32% and content-based for 0.2%, while the true label mix is 64% / 33% / 2%. It recovers 57% of the collaborative users (recall 0.57) at the price of some wrong majority-class calls: a deliberate trade of plain accuracy for minority recall. Balanced accuracy (0.469 vs 0.333 for a majority guesser or random guesser) and ROC-AUC (0.735 vs 0.5) show that it does carry signal.

9.4 Why changing algorithms did not help

Fourteen candidates were compared under one protocol (5-fold CV on 3,994 development users, then once on the held-out users; `selector_experiments_inner3.json`):



Model comparison

Candidate	CV macro-F1	CV accuracy	CV realised hit-rate
multiclass LightGBM (deployed)	0.467	0.633	0.131
multiclass XGBoost	0.457	0.629	0.130
multiclass CatBoost	0.454	0.640	0.131
multiclass Extra Trees	0.466	0.620	0.130
multiclass Random Forest	0.456	0.613	0.130
multiclass HistGradientBoosting	0.447	0.630	0.130
one-vs-rest LightGBM	0.436	0.645	0.130
pairwise LightGBM	0.458	0.628	0.131
regression (expected utility) LightGBM	0.453	0.638	0.131
LambdaRank over strategies	0.421	0.636	0.129
soft voting (LGB + ET + Cat)	0.453	0.646	0.131
stacking (LGB + ET + Cat)	0.410	0.524	0.117

All sensible models land in a band of 0.43-0.47 macro-F1 and 0.61-0.65 accuracy; the differences between the top candidates are inside run-to-run noise (rerunning the same two candidates moved their CV macro-F1 by

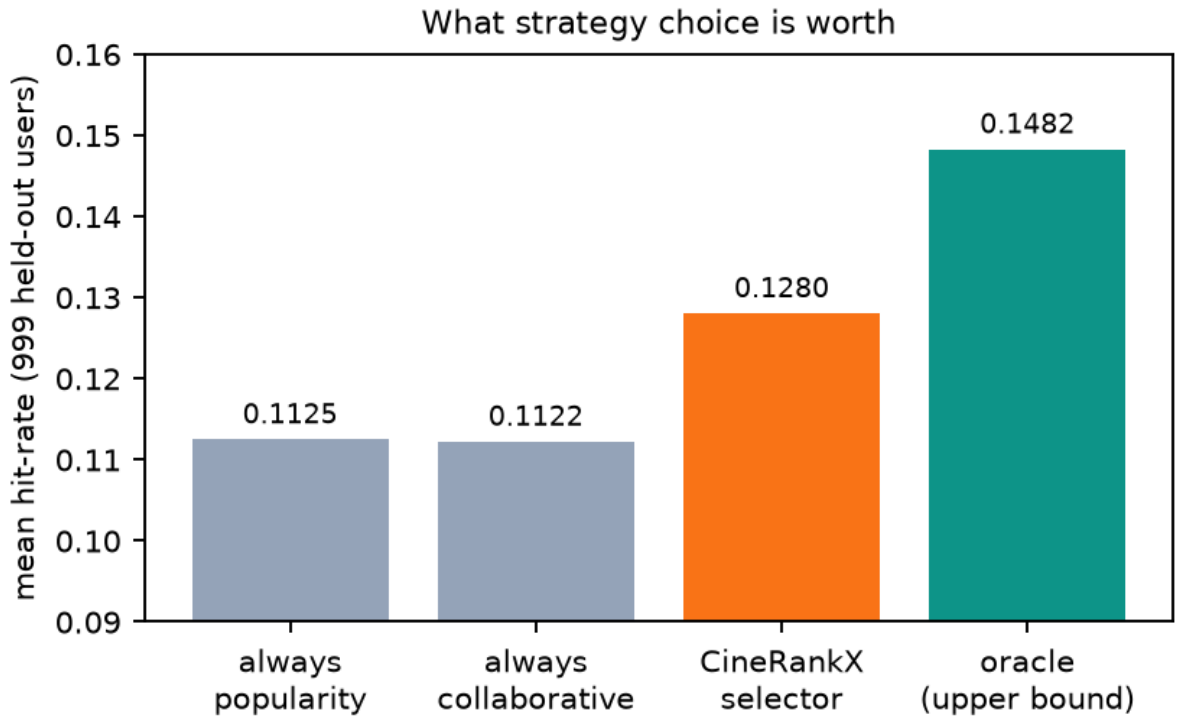
about 0.01). **Therefore the bottleneck is the data (labels and information), not the algorithm.** Deploying LightGBM is justified by simplicity, speed and being among the best, not by a decisive win.

9.5 Further experiments on the labelled data (selector_phase2_base.json)

Candidate	Held-out accuracy	Macro-F1	Balanced acc.	Hit-rate [95% CI]	Gain vs popularity [95% CI]
A. multiclass LightGBM	0.617	0.472	0.469	0.128 [0.121, 0.135]	+0.0152 [+0.010, +0.020]
B. + sigmoid calibration	0.495	0.401	0.571	0.113 [0.106, 0.120]	+0.0001 [-0.005, +0.005] (not significant)
B2. + tuned class thresholds	0.653	0.482	0.491	0.125 [0.118, 0.132]	+0.0125 [+0.009, +0.017]
D. pairwise	0.636	0.483	0.497	0.129 [0.122, 0.136]	+0.0166 [+0.012, +0.022]
E. LambdaRank	0.642	0.403	0.406	0.127 [0.120, 0.133]	+0.0143 [+0.010, +0.019]
C. utility regression	0.643	0.440	0.434	0.128 [0.121, 0.135]	+0.0156 [+0.011, +0.020]
F. utility + popularity fallback (margin 0.005)	0.658	0.443	0.433	0.128 [0.121, 0.135]	+0.0155 [+0.011, +0.020]

Observations: (1) probability calibration *hurt* here (it flattened class separation); (2) tuning per-class thresholds is the only change that lifted plain accuracy above the majority rate (0.653 vs 0.644) while keeping macro-F1 up, but it did not improve recommendation value; (3) all selectors achieve essentially the same hit-rate, differences far inside the confidence intervals.

9.6 What the selector is actually worth



What strategy choice is worth

The decision-quality view asks a different question: *regardless of whether the label is "right", does the chosen strategy produce more hits?* For each held-out user take the hit-rate of the strategy the selector picked. Mean 0.128 against 0.112 (always popularity) and 0.148 (oracle). The gain over always-popularity, +0.0155, has a 95% bootstrap CI [+0.011, +0.020] (user-level paired bootstrap), so it is statistically significant. The selector therefore captures roughly (0.128-0.112)/(0.148-0.112) = **43%** of the achievable gap. **Caution:** the oracle is optimistic by construction, because it takes the best of three *noisy* scores per user; no real selector can reach it.

By history size (held-out users in thirds by number of profile ratings): users with 21-40 ratings, selector 0.185 vs popularity 0.152; 41-89 ratings, 0.130 vs 0.117; 90+ ratings, 0.070 vs 0.069 (no meaningful gain). Strategy choice matters most for light users, which is where it should.

9.7 Did a better label definition help?

Three labelling settings (`label_experiments.json`, 3,000 users, two independent holdouts each):

Setting	Label agreement	Oracle ceiling	Class mix (collab / content / pop)
old: 10 recommendations, 20% holdout	0.653	0.825	21% / 3.1% / 75.6%
current: 30 recommendations, 30% holdout	0.653	0.768	31.6% / 2.3% / 66.1%
larger: 50 recommendations, 40% holdout	**0.733**	0.793	37.5% / 1.6% / 60.9%

An **uncertainty-aware** variant marks a user *indistinct* unless the winner beats the runner-up by a one-sided 95% margin. Result: 72% (old), 85% (current), 80% (larger) of users are indistinct; among the few users with a clear winner in both labellings, agreement is 92-99% (3-11% of users). This is the strongest evidence that most per-user "best strategy" labels are statistically indistinguishable. A larger evaluation budget gives more repeatable labels (0.653 -> 0.733) but squeezes the content-based class further (1.6%).

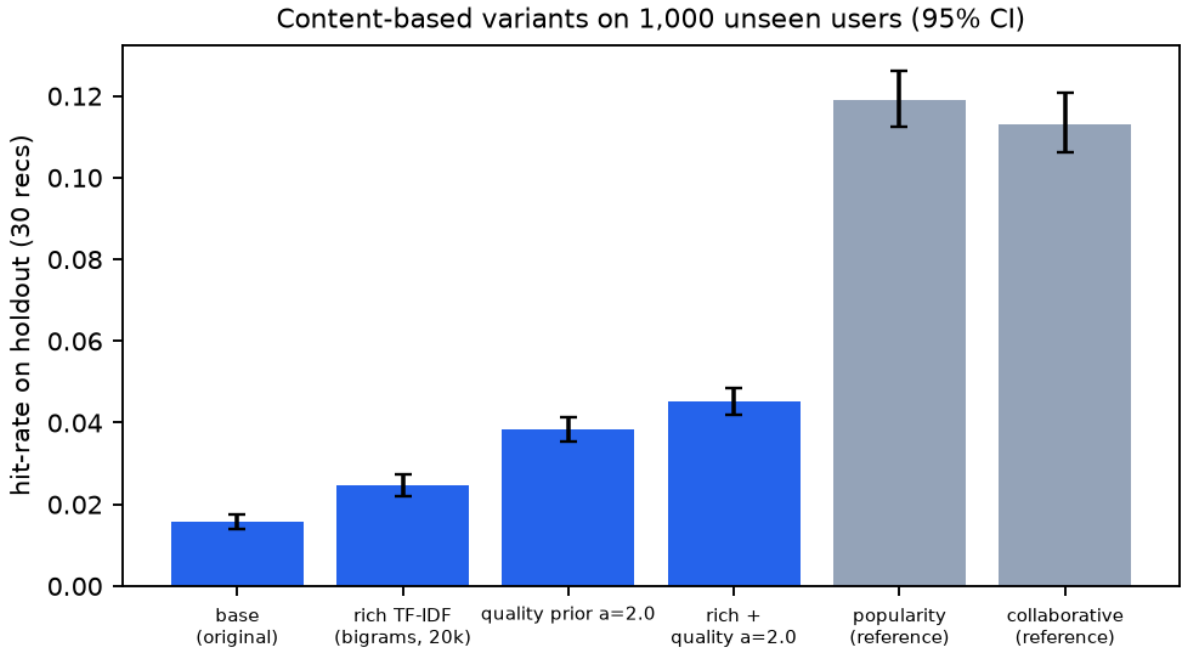
9.8 Metric glossary for this chapter

Term	What it measures	Where it can mislead
Classification accuracy	share of users whose predicted strategy equals the label	inflated by a dominant class; ignores label noise
Majority-class baseline	accuracy of always predicting the most common label (0.644)	the number a useful classifier should be compared with, not zero
Macro-F1	unweighted mean of per-class F1	very sensitive to the tiny content-based class (22 test users)
Weighted F1	per-class F1 weighted by class size	close to accuracy; dominated by the big classes
Balanced accuracy	mean per-class recall	chance level = 1/3 for three classes
ROC-AUC (OvR)	ability to rank users of a class above others	insensitive to calibration and to the decision threshold
Recommendation hit-rate	of the movies a strategy recommends, how many appear in the holdout (hits / holdout size), averaged over users	depends on holdout size and recommendation budget
Oracle hit-rate	mean of each user's best of three hit-rates	optimistic: selection from noisy numbers

Why improving classification accuracy does not necessarily improve recommendation quality. When two strategies perform almost the same for a user, mislabelling that user costs almost nothing in hit-rate; the costly mistakes are the rare cases where one strategy is clearly better. A classifier can therefore gain accuracy by getting many "ties" right without changing what users see, or lose accuracy yet improve hit-rate. That is why both objectives are reported separately.

9.9 Improving the content-based strategy (content_experiments.json)

Hypothesis: content-based is weak because its representation is poor. Protocol: label backtest (70/30 split, 30 recommendations), 2,000 users; variants chosen on a 1,000-user development half and confirmed on 1,000 *different* users.



Content variants

Variant (confirmation half)	Hit-rate [95% CI]	Gain vs original
original (2,000 terms, top-20-rated profile)	0.0156 [0.0138, 0.0174]	n/a
bigram TF-IDF, 20,000 terms ("rich")	0.0246 [0.0221, 0.0273]	+0.0090
quality prior only (alpha = 2)	0.0383	+0.0227
rich + quality prior (alpha = 2) (selected on dev)	**0.0452 [0.0420, 0.0484]**	**+0.0296 [+0.027, +0.033]**
mean-centred ("liked-centred") profile	0.0093	-0.0063 (worse)
genre block appended (weight 1.0)	0.0122	-0.0034 (worse)
reference: popularity	0.1188	
reference: collaborative	0.1130	

Improving the representation nearly **triples** content-based hit-rate, but it stays far below popularity. Two caveats: the quality prior makes content-based partly "popular among similar-looking movies", so part of the gain comes from a generic quality signal; and in MovieLens-style random-split evaluation popularity dominates any method not explicitly built on collaborative signals.

9.10 End-to-end effect of the improved content strategy (selector_phase2_rq_t30_h0.3.json)

Rebuilding labels and features with the improved content model changes the problem: the mean content-based hit-rate rises from 0.016 to 0.049 and the content-based class grows from 2.3% (113) to 10.6% (527 users). Results on the new labels (999 held-out users): CV macro-F1 about 0.49-0.50 (up from 0.47), held-out accuracy 0.546 against a *lower* majority rate of 0.585, macro-F1 0.47-0.48, selector hit-rate 0.130 vs always-popularity 0.116 (gain +0.0129 to +0.0149, CIs excluding zero) and an oracle of 0.158. Interpretation: the minority class

becomes more learnable, but the decision value of choosing a strategy does not change. **This variant is an experiment; it is not what is deployed.**

9.10b The best experiment so far: improved content + larger label budget (selector_phase2_rq_t50_h0.4.json)

Combining the improved content strategy with the more reliable label definition (50 recommendations per strategy, 40% holdout) changes the picture. All users are kept; evaluation is the same 80/20 user split with 999 held-out users:

Quantity	Deployed setting (30 recs, 30%, original content)	Experiment (50 recs, 40%, improved content)
Class mix (collab / content / popularity)	33% / 2% / 64%	36% / 7% / 56%
Majority-class accuracy	0.644	0.564
Held-out accuracy (multiclass LightGBM)	0.617	**0.677** (+11 points above the majority rate)
Macro-F1	0.472	**0.586**
Balanced accuracy	0.469	**0.606**
ROC-AUC	0.735	**0.807**
Selector hit-rate / always-popularity / oracle	0.128 / 0.112 / 0.148	0.199 / 0.162 / 0.218
Gain over always-popularity [95% CI]	+0.0152 [+0.010, +0.020]	**+0.0368 [+0.030, +0.044]**

By history size the experiment shows the same pattern, more strongly: users with 18-36 ratings: selector 0.297 vs popularity 0.210; 37-79 ratings: 0.196 vs 0.173; 80+ ratings: 0.107 vs 0.105.

How to read this honestly. (1) The two columns use *different label definitions* (budget, holdout size, content model) and different class mixes, so their hit-rates and accuracies are not directly comparable; compare each column with *its own* majority rate and popularity reference. (2) What does carry over is the conclusion: **better labels and a better content strategy make the classifier clearly beat the majority class** (accuracy +11 points, balanced accuracy 0.61, AUC 0.81) and double the gain over popularity (+0.037 vs +0.015). That supports the diagnosis that the earlier ceiling came from label quality. (3) This setting has **not been deployed**; the app still uses the deployed model of 9.1. Adopting it means rebuilding labels and features (`selector_phase2.py --top-n 50 --holdout 0.4` builds the frame; deployment would use the same deployment function as before) and then regenerating the KPI report.

9.11 Status of the more advanced strategy-selection ideas

Idea	Status in the repository
Utility-based selection (predict each strategy's expected hit-rate, pick the best)	Implemented and evaluated (`src/strategy_selector/utility.py`, candidates C and F above); statistically tied with the classifier; not deployed
Pairwise strategy comparison	Implemented and evaluated (`PairwiseForm`); tied
Learning-to-rank over strategies (LambdaRank)	Implemented and evaluated (`RankingForm`); lower macro-F1, equal hit-rate
Uncertainty-aware fallback	Implemented as the utility margin *tau* (0.005); did not improve over *tau* = 0
Contextual bandit	**Not implemented**; needs online user feedback that the project does not collect
Hybrid fusion of strategy lists	**Not implemented**
Improved content-based model	Implemented as `content_model.py`; evaluated; not yet the deployed content strategy

9.12 How to defend the result in one paragraph

"Our classifier's accuracy is 61.7%, below the 64.4% majority rate, and we report that openly. We showed that the cause is label reliability: two independent re-labellings of the same users agree only about 65-73% of the time, and a perfect model could reach only about 76%. Changing algorithms (fourteen of them) did not move the result, which confirms the bottleneck is the data. What matters for users is recommendation quality: following the selector's choice gives a hit-rate of 0.128 versus 0.112 for always-popularity, a gain that is statistically significant (95% CI +0.011 to +0.020)."

Chapter 10. Model Evaluation and the Mathematics Behind It

CineRankX evaluates three different things with different metric families: **search** (does the IR engine find on-topic movies?), **recommendation lists** (are the lists accurate, diverse, novel?) and **the strategy classifier** (does it predict the right label?). A fourth group measures **system** cost. The names are fixed in `eval_config.yaml` (23 KPIs) and checked by `build_report(strict=True)`, which refuses to finish if any is missing.

Group	KPIs (exact names)
Search (5)	Precision@10, Recall@10, MRR, NDCG, Search Latency
Recommendation (8)	Precision@K, Recall@K, F1, MAP, Coverage, Diversity, Novelty, Cold-Start Accuracy
Machine learning (6)	Accuracy, Precision, Recall, F1, ROC-AUC, Confusion Matrix
System (4)	Response Time, Memory Usage, CPU Usage, Throughput

10.1 Classification metrics

Use the real confusion matrix of the deployed strategy classifier on 999 held-out users (rows true, columns predicted):

	pred collab	pred content	pred popularity	row total
true collaborative	191	6	137	334
true content_based	11	4	7	22
true popularity	216	6	421	643
column total	418	16	565	999

Confusion matrix

A table counting, for every true class, how many users were predicted as each class. The diagonal is correct predictions. It shows *which* mistakes happen (here most true-popularity users are mistaken for collaborative: 216).

Accuracy

$$accuracy = (number\ of\ correct\ predictions) / (total) = (191 + 4 + 421) / 999 = 0.6166$$

Interpretation: 61.7% of users were assigned their labelled strategy. *Misleading when:* classes are imbalanced (always predicting popularity scores 643/999 = 0.644).

Precision and recall (per class)

$$precision_c = TP_c / (TP_c + FP_c), \quad recall_c = TP_c / (TP_c + FN_c)$$

For collaborative: TP = 191, FP = 11 + 216 = 227 (predicted collaborative but not), FN = 6 + 137 = 143. Precision = 191/418 = **0.457**; recall = 191/334 = **0.572**. *Precision* answers "when we say collaborative, how often are we right?"; *recall* answers "of the real collaborative users, how many did we find?".

F1-score

The harmonic mean of precision and recall, which punishes imbalance between them:

$$F1 = 2\,P\,R / (P + R) = 2 \times 0.457 \times 0.572 / (0.457 + 0.572) = 0.508$$

Content-based: P = 4/16 = 0.25, R = 4/22 = 0.182, F1 = **0.211**. Popularity: P = 421/565 = 0.745, R = 421/643 = 0.655, F1 = **0.697**.

Macro versus weighted averages

- **Macro**: plain mean over classes, treating each class equally: (0.508 + 0.211 + 0.697)/3 = **0.472**. Highlights poor minority-class performance.
- **Weighted**: mean weighted by class size: (334 x 0.508 + 22 x 0.211 + 643 x 0.697)/999 = **0.623** (this is the "F1" shown on the dashboard).
- **Balanced accuracy** = mean of per-class recall = (0.572 + 0.182 + 0.655)/3 = **0.469**.

ROC curve and AUC

For one class versus the rest, sweep a probability threshold from 1 to 0 and plot the **true-positive rate** (recall) against the **false-positive rate** at each threshold. The area under that curve (AUC) equals the probability that a randomly chosen positive user receives a higher predicted probability than a randomly chosen negative user.

Numerical example. Positive scores 0.9, 0.7, 0.4; negative scores 0.6, 0.3, 0.2. Of the 9 positive-negative pairs, the positive is higher in 3 + 3 + 2 = 8, so AUC = 8/9 = **0.889**. Chance = 0.5; perfect = 1.0. For three classes, **one-vs-rest** AUC is computed per class and averaged (macro): 0.735 for the deployed classifier. *Why it matters:* it measures *ranking* quality independent of the threshold, and the downstream ranker consumes probabilities, not hard labels. *Misleading when:* the classes are extremely imbalanced or probabilities are miscalibrated.

10.2 Ranking and recommendation metrics

Take one user, the recommended list **[A, B, C, D, E]** and the set of held-out movies the user liked **{B, D, F}** (3 relevant). Hits are at ranks 2 and 4.

Metric	Formula	Value
Precision@K	relevant in top-K / K	2/5 = **0.40**
Recall@K	relevant in top-K / total relevant	2/3 = **0.667**
Hit Rate@K	1 if at least one relevant in top-K, else 0 (averaged over users)	**1**
Average Precision@K	sum over hit ranks of precision at that rank, divided by min(K, #relevant)	(1/2 + 2/4) / 3 = **0.333**
MAP@K	mean of AP over users	mean of per-user values
NDCG@K	DCG / ideal DCG, with DCG = sum of 1/log2(rank + 1) over hits	DCG = 1/log2 3 + 1/log2 5 = 1.062; ideal = 1 + 1/log2 3 + 1/log2 4 = 2.131; NDCG = **0.498**

Interpretation and use in CineRankX. *Precision* is how clean the list is; *recall* is how much of what the user likes was found; *Hit Rate* is the most forgiving ("did we get anything right?"), and is used as the dashboard "accuracy"; *MAP* and *NDCG* reward putting hits early. A "relevant" movie is a held-out movie the user rated **>= 4.0** (`positive_threshold`).

When misleading. Recall@K is capped when a user has far more than K relevant items. Precision/Recall depend on K and on how many positives a user has. NDCG with binary relevance ignores how much more the user liked item A than item B. Hit Rate saturates near 1 for users with many liked items.

The project's definition of "accuracy" in the strategy-comparison dashboard is Hit Rate@10 on the frozen test split (`src/benchmark/metrics.py` docstring). It is *not* the classifier accuracy. This distinction is the most common viva confusion; keep both separate.

Coverage, diversity, novelty (list-level metrics)

- **Coverage** = number of distinct movies appearing in any user's top-K / catalog size. Baseline 0.38% vs CineRankX 5.7% on the benchmark.
- **Diversity** (`intra_list_diversity`) = 1 - mean pairwise genre Jaccard similarity inside a list. *Example:* two movies with genres {A,B} and {B,C}: Jaccard = 1/3, diversity = 0.667.
- **Novelty** = mean self-information (-log2 of item popularity), scaled; shared with the novelty objective.

Search metrics

- **MRR** (mean reciprocal rank) = mean over queries of 1 / (rank of first relevant result): first result relevant -> 1, third -> 0.333.
- **Precision@10**, **Recall@10**, **NDCG** as above, using the documented *proxy* relevance (a result is relevant if it shares at least one genre or keyword with the query). Because on average 1,812 of 26,743 movies count as "relevant" per query, **Recall@10 is bounded above by 10 divided by the number of relevant movies and measured only 0.0101**: it reflects the size of the relevant pool, not retrieval quality (this caveat is printed in the KPI report).
- **Search latency** in milliseconds.

10.3 Rating prediction metrics

Predicted ratings [4.0, 3.5, 2.0] versus actual [5.0, 3.0, 2.5]; errors [-1.0, +0.5, -0.5].

$$MAE = (1/n)\ sum\ |error| = (1 + 0.5 + 0.5) / 3 = 0.667$$

$$RMSE = sqrt(\ (1/n)\ sum\ error\text{^}2\ ) = sqrt(\ (1 + 0.25 + 0.25) / 3\ ) = 0.707$$

RMSE punishes large errors more than MAE. In the benchmark the bias predictor reaches RMSE **0.846** and MAE **0.649** on the test split against **0.936 / 0.725** for the frozen item-mean predictor (`reports/strategy_comparison.json`). *Misleading when:* a model that predicts everyone's rating accurately can still rank poorly.

10.4 System metrics

`src/evaluation/system_metrics.py::measure(operation, n_calls, warmup)` times repeated calls with `perf_counter` and uses `psutil` to read process memory and CPU time (derived from `cpu_times()` deltas, not `cpu_percent`). It reports response time in ms per call, memory (MB), CPU (% of one core) and throughput (operations per second) aggregated over the instrumented operations (IR search, IR similar_to, strategy predict, rank top-10). Latest saved: response time 23.4 ms, throughput 46.0 ops/s, memory 398 MB, CPU about 98% of one core.

10.5 Evaluation methodology concepts

Train/validation/test split

Train fits parameters; *validation* chooses hyperparameters and models; *test* is touched once for the final report. Reusing the test set to choose a model makes the reported number optimistic (it has been "tuned to"). In CineRankX the benchmark's tuning cohorts are scored on the **validation** split only, and the test split's ground truth is never read during training or tuning.

Cross-validation and stratified sampling

k-fold CV splits the training data into *k* parts, trains on *k-1*, scores on the remaining part, and averages; it estimates generalisation and its variance without spending the test set. *Stratified* folds keep the class

proportions the same in every fold, which matters when one class is 2%. The project uses 5-fold stratified CV (`StratifiedKFold`), reducing the fold count automatically when a class is rarer than the fold count (test `test_cross_validation_shrinks_folds_for_rare_classes...`).

Data leakage (and the concrete cases in this project)

Leakage is information from the evaluation target reaching the model. Cases found and fixed:

1. **Feature/label overlap.** Early features were computed on a user's *whole* history while labels came from backtests on random subsets of the same history; now features use only the profile split.
2. **Collaborative similarity including the test ratings.** The legacy backtest built neighbour structures from all ratings; the leak-free labeller builds everything from the profile ratings only.
3. **Rating predictor fitted on all ratings, including held-out ones.** The benchmark fits the predictor and catalog statistics on the training split only.
4. **SMOTE before splitting.** SMOTE must only see the training fold; the code resamples after the split and tests that no synthetic point is derived from test users.
5. **A user's own row as their neighbour** in CF: explicitly excluded (`sims[own] = -1`).

Generalisation, bias and variance

Bias is error from a model that is too simple; *variance* is error from a model too sensitive to the training sample. Boosted trees with many leaves have low bias and high variance: they fit training labels (accuracy up to 0.88) but generalise less (about 0.62). The gap between training and held-out accuracy is the symptom; regularisation, subsampling and cross-validation are the remedies.

Hyperparameter tuning

Hyperparameters (learning rate, leaves, ...) are not learned by training. They are chosen by search: Optuna's TPE sampler proposes values, 3-fold CV macro-F1 on development users scores them, the best are kept. Tuning uses only development users; the held-out set stays untouched.

Confidence intervals and statistical significance

A single metric is a point estimate from a sample of users. The project reports **user-level bootstrap intervals** (`src/strategy_selector/stats.py::bootstrap_ci`): resample users with replacement 2,000 times, recompute the mean, and take the 2.5th/97.5th percentiles. For *improvement* claims it uses a **paired bootstrap** on per-user differences (selector minus popularity); an interval that excludes 0 means the gain is unlikely to be chance. `mcnemar_p` is an exact test for "two classifiers have the same error rate on the same users", and `proportion_ci` is the Wilson interval for accuracy.

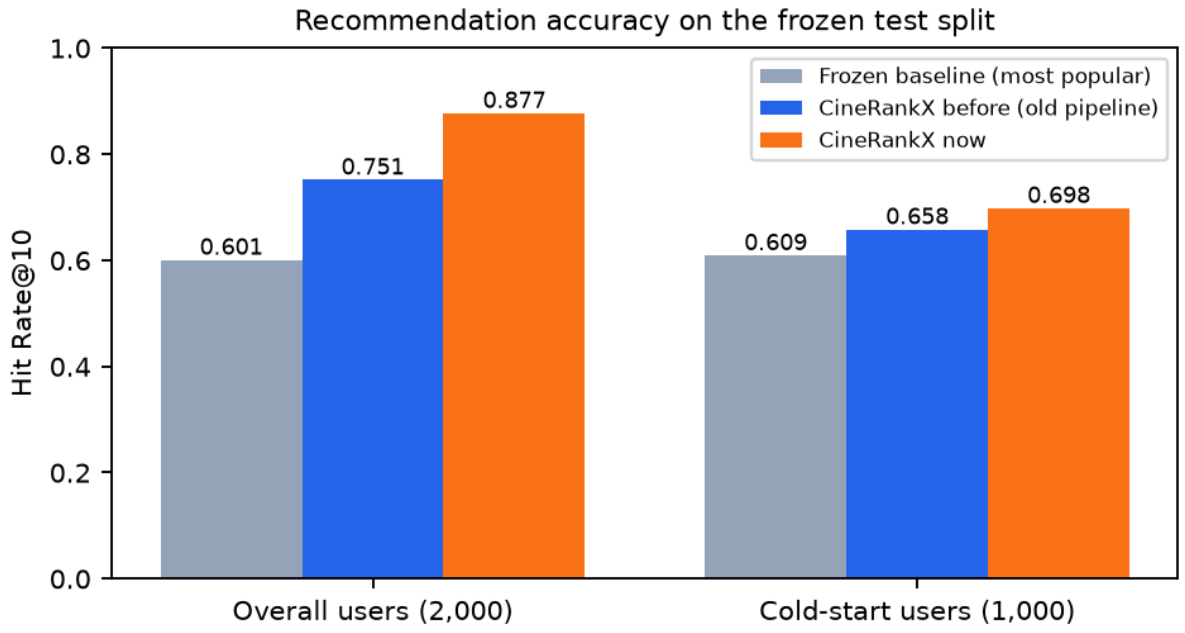
> **Worked reading:** selector hit-rate gain over popularity = +0.0155 with 95% CI [+0.011, +0.020]. The interval excludes zero, so the improvement is statistically significant, although small in absolute terms (about 1.5 hit-rate points).

10.6 The frozen benchmark protocol (how recommendation accuracy is measured fairly)

`src/benchmark/` implements the protocol that produced the Hit Rate@10 figures.

1. **Deterministic hash split** (`split.assign_splits`): each rating (user, movie) goes to train (70%), validation (10%) or test (20%) by a splitmix64 hash of (user, movie, seed 20260101). No stored file; identical everywhere.

2. **Fixed cohorts**: 2,000 overall test users, 1,000 cold-start test users (profile cut to the first 5 training ratings, ordered by a second hash), plus two disjoint 1,000-user tuning cohorts scored only on validation.
3. **Same ground truth for every strategy**: relevant = held-out ratings >= 4.0; items the user rated before the scored split are excluded from recommendations.
4. **Frozen baseline** (`baseline.py`): rank movies by number of training ratings; predict ratings by item mean with damping 10. The file imports nothing from the rest of the project (a unit test checks this) and `scripts/run_benchmark.py` stores the SHA-256 of the baseline, metrics and split code with the baseline's results and **aborts if the code, split or numbers drift**.
5. **Results** (test split): overall Hit Rate@10 0.601 (baseline) / 0.751 (old CineRankX pipeline) / 0.877 (current); cold-start 0.609 / 0.658 / 0.698; NDCG@10 0.163 -> 0.362; Precision@10 0.131 -> 0.291; RMSE 0.936 -> 0.846.



Strategy comparison

Why 90-95% accuracy is not realistic here. Hit Rate@10 asks for at least one liked movie among ten picks out of about 27,000. Hit Rate@20 is already 0.928 overall, but for the *classification* problem, where the label is itself noisy, the ceiling is about 0.76 (Chapter 9). A fairer way to judge the system is the set of ranking metrics plus significance tests, not a single accuracy percentage.

Chapter 11. Module-by-Module Codebase Walkthrough

The repository holds about 11,000 lines of Python in 89 files (counted by `docs/_build/extract_code.py`). This chapter walks through them package by package. Appendix D lists every class and function automatically extracted from the source, so nothing is missing even where this chapter summarises.

11.1 Directory structure

```
CineRankX/
  app.py                       Streamlit entry point: routing, caching, recommend()
  eval_config.yaml             exact KPI names + all tunable settings
  requirements.txt  CLAUDE.md  README.md  DATASET_MIGRATION.md
  .streamlit/config.toml       theme colours, upload limit
  src/
    ingestion/                 Module 1: load, join, clean
    ir_engine/                 Module 2: TF-IDF search + Trie
    strategy_selector/         Module 3: features, labels, classifiers, utility selection, stats
    ranking_engine/            Module 4: candidates, six objectives, ranker, EASE
    evaluation/                23-KPI suite (search, recommendation, ML, system)
    benchmark/                 frozen split, frozen baseline, CineRankX strategy, tuning
    storage/                   SQLite: ratings, user stats, recommendation runs
    ui/                        Streamlit screens, theme, icons, navigation
  scripts/                     verification, training, benchmark, experiment scripts
  tests/                       12 pytest files (272 test cases)
  data/processed/              generated artifacts (git-ignored)
  newdata/                     raw datasets (git-ignored)
  reports/                     generated reports (git-ignored)
```

11.2 eval_config.yaml: the single configuration file

Blocks: `search.metrics`, `recommendation.k_values: [5, 10, 20]` plus `metrics`, `machine_learning.metrics`, `system.metrics`, then `ranking_objectives` (genre gate floor 0.25 and the six `weights`), `strategy_selector` (`holdout_fraction`, `label_margin`, `label_top_n`, `inner_splits`, `inner_fraction`, `min_history`, `max_history`, `cv_folds`, `content_options`) and `benchmark` (seed, split fractions, cohort sizes, `ks`, `headline_k`). Loaders merge the block over safe defaults, so an older config still loads. *Design decision:* anything tunable lives here, never hard-coded in logic (a convention in `CLAUDE.md`).

11.3 Package src/ingestion (Module 1)

File	Purpose and logic
`loaders.py`	`load_movielens(base_dir)` reads `rating.csv, movie.csv, link.csv, tag.csv` into a dict of DataFrames; `load_tmdb` reads `TMDB_movie_dataset_v11.csv`. Default dirs `newdata/movielens`, `newdata/tmdb`
`joiner.py`	`join_movielens_tmdb(ml, tmdb)` returns `(merged_frame, report)`. Steps in 5.3. `_split_names` turns `"Action, Adventure"` into a list. The report dict (`movielens_movies`, `movielens_with_tmdb_id`, `tmdb_movies`, `joined_rows`) is printed by `build_dataset`
`text_cleaner.py`	`clean_text` (lowercase, regex letters, NLTK stop-words, Porter stem); `_ensure_nltk_data` downloads stop-words once
`build_dataset.py`	`build()`: load, join, clean `text_chunk` -> `text_chunk_clean`, write `movies_merged.csv` and `ratings_clean.csv`, print the join report. Run as `python -m src.ingestion.build_dataset`

11.4 Package src/ir_engine (Module 2)

- `tfidf_vectorizer.py::TFIDFEngine`: `fit(df)` builds the TF-IDF matrix over `text_chunk_clean`; `save/load` pickle the vectorizer, write the sparse matrix with `save_npz`, and store movie ids/titles; `search(query, limit)` transforms the query and computes `cosine_similarity` against all rows, `np.argsort(...)[::-1]` for the top-k, keeping only positive similarities; `similar_to(movie_id, limit)` uses the movie's row as the query.
- `trie.py::Trie`: nodes keyed by character; `insert(title, movie_id)` lower-cases titles and stores `(movie_id, title)` at the final node; `search(prefix, limit)` descends to the prefix node then runs a depth-first `collect_movies` until `limit` results. Complexity: O(length of prefix + results).
- `ir_engine.py::IREngine`: a façade that owns a `TFIDFEngine` and a `Trie`. **Design quirk:** the Trie is rebuilt in `fit` and cannot be persisted, so `load()` restores only TF-IDF; autocomplete after `load` needs a re-fit. The ranking engine uses only `similar_to` and the TF-IDF matrix.

11.5 Package src/ranking_engine (Module 4)

config.py

`OBJECTIVES = ("relevance", "diversity", "novelty", "coverage", "popularity_quality", "predicted_rating")` is the canonical order. `RankingWeights` validates (all six present, none unknown, none negative, sum > 0) and **normalises to sum 1**; `.items()` returns canonical order; `.with_overrides(**kw)` returns a new object (no mutation, used for sensitivity experiments); `.from_config()` reads the YAML. `load_genre_gate_floor` reads the gate floor (default 0.25).

objectives.py

`CatalogStats(movies_df, ratings_df)` precomputes per-item values once: `_build_genres` (genre sets; understands stringified lists, comma strings and pipe strings), `_build_popularity_quality` (Bayesian weighted rating, min-max), `_build_novelty` (self-information of observed engagement). Accessors `genres()`, `popularity_quality()`, `novelty()` return neutral values for unknown ids. Module functions `diversity_score(movie_id, selected_ids, similarity)` and `coverage_score(movie_id, covered_genres, stats)` implement objectives 2 and 4.

predictor.py

`BaselinePredictor(damping=10)`: `fit(ratings_df)` (global mean, damped item bias, then damped user bias on the item-adjusted residual), `predict(user, movie)` (clipped to [0.5, 5]), `predict_normalized` (rescaled to [0, 1]).

candidates.py

- `strategy_source_weights(strategy, confidence)`: `_FALLBACK_PRIOR = {collaborative 0.60, content_based 0.20, popularity 0.20}`; `strength = clip((confidence - 1/3) / (2/3), 0, 1)`; result `(1 - strength) x prior + strength x one_hot(strategy)`. Unknown/missing strategy returns the prior. Weights always sum to 1 (tested).
- `genre_overlap_gate`, `_normalize` (min-max per source), `CandidateGenerator`: constructor builds `_popular` (200 most-rated) and the CF structures; `_content_candidates`, `_collaborative_candidates`, and `generate(user_id, strategy, confidence, pool_size=200)` which excludes already-rated movies, blends sources (sum of weight x normalised score), records which sources proposed each movie in a `sources` string such as `"content_based+collaborative"` and returns `[movieId, relevance, sources]` sorted by relevance. `candidates_for_seed` serves "similar to this one movie" lookups through the same gate.

ranker.py::MultiObjectiveRanker

Constructor builds `CatalogStats`, a `BaselinePredictor` (or accepts one), a `CandidateGenerator` (skippable with `build_generator=False`, used by the benchmark), and a title lookup. `_similarity_lookup(movie_ids)` builds pairwise similarity for the pool (TF-IDF cosine, genre-Jaccard fallback). **rank(user_id, k, strategy, confidence, candidates, weights, pool_size)** is the heart of the project:

1. obtain candidates (given, or generated);
2. precompute the four item/user-only objectives per candidate (`fixed`);
3. loop: for every remaining candidate compute `diversity_score` against the selected list and `coverage_score` against the covered genres, form `total = sum(weight x score)`, pick the maximum, remove it from `remaining`, add to `selected`, update `covered_genres`;
4. return a DataFrame `[rank, movieId, title, score, sources, relevance, diversity, novelty, coverage, popularity_quality, predicted_rating]`.

Complexity O(K x P) objective evaluations for pool size P (200) and list length K. `recommend()` is a thin wrapper that returns just movie ids.

ease.py::EaseModel

`fit(train, n_items=6000, l2=500)` (see 6.A.6), `user_matrix(profiles, users)` (binary sparse rows), `blended_scores(user_rows, beta)`, `save/load` (`data/processed/ease_model.npz`).

11.6 Package src/strategy_selector (Module 3)

File	Role
`profile_features.py`	`extract_profile_features(ratings_df, movies_df)`: vectorised user feature table (section 8.4). `_catalog_tables` caches per-catalog genre maps and popularity percentiles (a speed fix: rebuilding them per call dominated inference time). `_genre_features` computes `num_genres, genre_diversity, top_genre_share, genre_entropy` and 19 `affinity_*` columns; `_time_features` the span and recency features
`strategy_labeler.py`	The *backtest*. `_BacktestContext` builds shared structures once per batch: popularity top-N, a TF-IDF matrix, a user-item matrix with dense user-user cosine similarity (bounded to 5,000 users x 2,000 movies). `_content_based_hit_rate`, `_collaborative_hit_rate`, `_popularity_hit_rate`; `select_best_strategy(cb, cf, pop)` (tie goes to popularity); `label_users_with_best_strategy` (the original multi-split labeller, kept for tests and legacy scripts)
`holdout_labeler.py`	`split_profile_holdout(ratings, fraction, seed)` (per user, at least one rating each side); `label_holdout(profile, holdout, movies, top_n, content_model)` uses the same hit-rate functions but builds everything from the profile only; `clean_labels(labels, margin)` is the (now optional) ambiguity filter
`dataset.py`	`sample_user_ids` (random users from `user_stats`), `load_ratings` (SQL `WHERE userId IN (...)`, restricted to catalog movies), `cache_path`, `build_labelled_frame` (sample, split, label, build features, cache CSV and context pickle)
`strategy_context.py`	`LiteContext`: a compact reference population (popularity set, TF-IDF row-normalised matrix, sparse CF matrix, item means, item surprisal). `inner_scores(ctx, user, ratings)` runs the inner backtest (3 seeded splits, 25% pseudo-holdout). `LiteContext.engine_scores` computes similarity strengths. `build_features(ratings, movies, ctx)` = profile features + head shares + engine scores + bias/liked share/surprisal + inner

	scores, **the single function used at training and at serving**
classifier.py	StrategyClassifier: original Random Forest fit with optional SMOTE (train fold only), fit_full (refit a chosen estimator on all users), predict (single pass, argmax + max probability), save/load (pickle model and (LabelEncoder, feature_names); sets n_jobs=1 on load for cheap single-user inference)
formulations.py	winner, margin, five formulation classes sharing fit(X, scores) / predict_proba, decision_quality
model_selection.py	CANDIDATES registry, TUNING search spaces, cross_validate, compare_candidates, evaluate_holdout
estimators.py	make(family, **params): importable factory (so pickled models reference a real module path instead of a script)
utility.py	UtilitySelector (per-strategy regressors, margin tau against the fallback) and choose_tau
content_model.py	ContentModel with options vectorizer, genre_weight, profile, quality_alpha
stats.py	bootstrap_ci, paired_bootstrap, mcnemar_p, proportion_ci
config.py	load_selector_config merging the YAML block over defaults

Walk-through: how one user's label is created (label_holdout)

```
ctx = _BacktestContext(profile, movies_df, top_n=top_n)        # structures from PROFILE data only
hold_by_user = holdout.groupby("userId")["movieId"].apply(lambda s: s.values)
for user_id, train in profile.groupby("userId"):
    test_ids = hold_by_user.get(user_id, np.array([]))
    cb  = content_model.hit_rate(...) or _content_based_hit_rate(ctx, train, test_ids)
    cf  = _collaborative_hit_rate(ctx, user_id, test_ids)
    pop = _popularity_hit_rate(ctx, test_ids)
    rows.append({... "best_strategy": select_best_strategy(cb, cf, pop), ...})
```

In words: build the strategies from visible data, recommend, and count how many of the recommended movies the user later rated (hits divided by holdout size).

Walk-through: serving one prediction (app.py::recommend)

```
ratings_of_user = ratings[ratings["userId"] == user_id]
profile = build_features(ratings_of_user, movies, context, cfg["inner_splits"], cfg["inner_fraction"])
prediction = classifier.predict(profile).iloc[0]       # predicted_strategy, confidence
ranked = ranker.rank(user_id, k=k, strategy=strategy, confidence=confidence, weights=weights)
```

If the classifier or context artifact is missing the strategy stays None and the default prior blend is used, so the app degrades gracefully.

11.7 Package src/evaluation (the 23-KPI suite)

File	Purpose and key logic
metric_names.py	One constant per KPI (PRECISION_AT_10, MRR, COLD_START_ACCURACY, ...) and group tuples SEARCH_METRICS, RECOMMENDATION_METRICS, MACHINE_LEARNING_METRICS, SYSTEM_METRICS. load_configured_metrics reads eval_config.yaml; missing_from(report, path) lists configured KPIs that the report lacks, so a metric cannot silently disappear
search_metrics.py	evaluate_search(engine, queries, movies, k=10, relevance_index): relevance proxy (shares at least one genre or keyword), precision_at_k, recall_at_k, reciprocal_rank, ndcg_at_k, latency in ms. build_relevance_index parses genre/keyword cells in three formats
recommendation_metrics.py	Per-user precision_at_k, recall_at_k, f1_score_at_k,

	`average_precision` (denominator `min(k, #relevant)`), `coverage`, `intra_list_diversity`, `novelty`; `split_user_ratings` (per-user random 80/20); `popularity_baseline_top_k` (observed counts); `evaluate_cold_start` (natural or simulated cohort; returns `mode`); `evaluate_recommendations` ties them together
`ml_metrics.py`	`classification_metrics(y_test, y_pred, proba, class_names, y_train)` returns accuracy, weighted precision/recall/F1, per-class table, confusion matrix with labels, one-vs-rest ROC-AUC (returns 0.0 if a class is absent from the test set), majority baseline; `to_report_names` renames to the exact KPI strings
`system_metrics.py`	`measure(operation, n_calls, warmup, label)` and `aggregate(measurements)` (see 10.4)
`report.py`	`build_report(search, recommendation, ml, system, strict=True)` (raises if any configured KPI is missing) and `render(report)` for the printed table

`scripts/generate_kpi_report.py` is the driver: it loads the catalog and 400,000 ratings, the IR engine, the classifier and the ranking engine, runs the four groups, writes `reports/kpi_report.json` (plus a text copy when redirected), and reads the classifier results from `data/processed/strategy_eval.json` rather than retraining.

11.8 Package src/benchmark (frozen comparison)

File	Purpose and key logic
`config.py`	`load_benchmark_config`: seed 20260101, 70/10/20 split, positive threshold 4.0, cohort sizes 2,000 / 1,000 / 1,000, `cold_history` 5, `ks`, `headline_k = 10`
`split.py`	`uniform_hash(users, items, seed)` (splitmix64 mapped to [0, 1)), `assign_splits`, `load_interactions` (read `rating.csv` with narrow dtypes, keep catalog movies, add `split` and an ordering key), `build_benchmark` (eligible users, seeded permutation into four disjoint cohorts, cold users' training rows truncated to their first 5 by the second hash, ground-truth dictionaries), `fingerprint` (SHA-256 of cohorts and test truth), `get_benchmark` (pickle cache, verifies the fingerprint)
`baseline.py`	`FrozenBaseline(train)`: `ranking` (movies sorted by training rating count, tie by id), `recommend(seen, k)`, `predict(movie)` (damped item mean). Version tag `popularity-itemmean-v1`. Imports only numpy and pandas
`metrics.py`	`hit_rate`, `precision`, `recall`, `ndcg`, `average_precision`, `rating_errors`, `summarize` (mean over users with at least one positive, plus coverage/diversity/novelty hooks)
`evaluate.py`	`exclusions(bench, phase, users)` (items rated before the scored split are not recommendable), `evaluate_strategy(bench, phase, cohort, recommend_many, predict, ...)`: the *same function* scores any strategy on identical users and truth
`cinerankx.py`	`CineRankXStrategy`: EASE pools (`pools`), optional six-objective re-ranking (`recommend_many`), the bias predictor, and the ranker with `build_generator=False`
`tuning.py`	`tune_relevance` (grid over lambda and beta by validation NDCG@10) and `tune_weights` (random weight vectors under the diversity floor)

`scripts/run_benchmark.py` orchestrates: build or load the benchmark, score and verify the frozen baseline (`freeze_baseline` compares code hash, split fingerprint and results with

data/benchmark/baseline_frozen.json), tune (or reuse cinerankx_params.json), evaluate old pipeline (before_pipeline) and new pipeline on the test split, write reports/strategy_comparison.json, and refit EASE on all ratings for the app.

11.9 Package src/storage (SQLite)

- db.py: connect, db_ready (true when user_stats has a row), build_database(path, ratings_chunks, movies, merged). Tables: movies(movieId, title, year, genres, original_language, release_date, poster_path), ratings(userId, movieId, rating, ts), user_stats(userId, n_ratings, avg_rating, first_ts, last_ts), recommendation_runs(id, userId, created_at, source, strategy, confidence, k, weights, status, error, elapsed_ms, results). The index idx_ratings_user on (userId, ts) makes one user's history a millisecond lookup.
- users.py: user_summary, history_page (server-side search, genre filter, rating range, sort and LIMIT/OFFSET; the sort column is chosen from a whitelist so user input can never be injected into SQL), rating_distribution, genre_preferences, user_timeline (monthly activity, decade mix, language mix), active_users (prefix search over user ids).
- runs.py: record_run (writes strategy, confidence, normalised weights as JSON, status, error text, elapsed time and the ranked results as JSON), list_runs.

11.10 Package src/ui and app.py (the web application)

app.py (157 lines): st.set_page_config, theme and header; **navigation** is a st.segmented_control whose key nav is synchronised with the URL query parameter view (overview | engines | run | profile | kpi) through nav.sync_view, so refresh and deep links work; load_engine() is a @st.cache_resource that reads the catalog, the first 400,000 ratings, the IR engine and the classifier and builds the ranker; recommend(user_id, k, weights) validates, builds features, predicts the strategy, ranks, logs the run (_log_run, best-effort) and returns (ranked, strategy, confidence); load_report is a @st.cache_data JSON reader keyed by file modification time so the dashboard always shows the latest saved results.

File	Purpose
theme.py	Design tokens as CSS variables (porcelain canvas, orange #F97316, cobalt, teal), card() (bordered container with title), kpi_row(), meter() (progress bar row)
icons.py	icon(name, size): Feather icons as inline SVG strings (no icon font, no emoji)
nav.py	VIEWS, selected_user, set_user, open_profile, sync_view
pages.py	render_overview (KPI tiles, candidate funnel, objective harmony, strategy routing, telemetry, surfaced candidates) and render_engines
runner.py	The runner: render_runner, _weights_panel (six sliders, normalisation indicator, reset callback), _live_controls (user ID validation), show_results (movie cards with a reason and six mini-bars), _csv_source (upload validation, 25 MB limit)
results.py	prepare_results (validates and sorts a ranked frame), output_kpis, split_title, reason_for (explanation from the two highest objective scores), genres_lookup
profile.py, profile_history.py	User 360 tabs: overview, history (paginated), preferences, past runs, current recommendations
kpi_view.py, comparison_view.py	KPI Benchmarks page: summary cards, top-K chart, selector quality vs chance, strategy comparison panels (frozen baseline vs CineRankX, overall and cold-start), classifier confusion matrix and per-class table, system performance. **Everything shown is read from the saved JSON files** (reports/kpi_report.json, reports/strategy_comparison.json)

11.11 scripts/

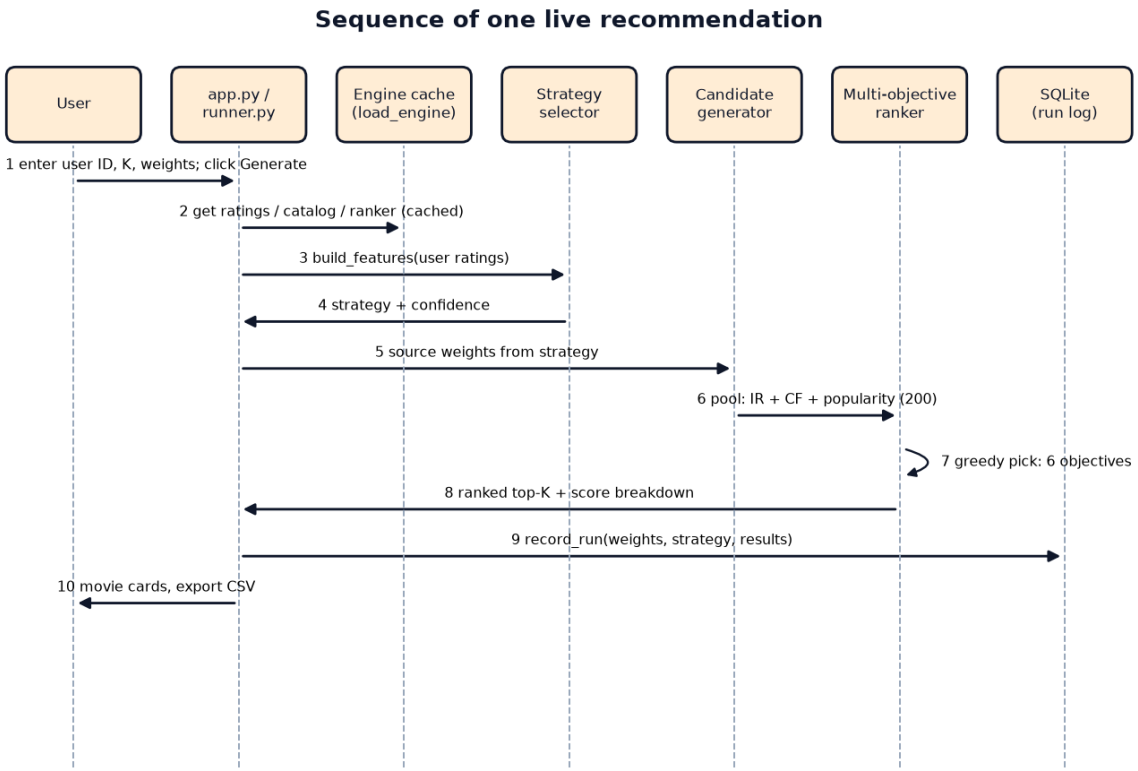
Script	What it does
verify_ir_engine.py, verify_strategy_selector.py, verify_ranking_engine.py	Original per-module demonstrations and artifact builders (the IR script saves the TF-IDF files; the selector script trains and saves the early classifier)
generate_kpi_report.py	The 23-KPI report driver (above)
build_user_db.py	Streams rating.csv in 1M-row chunks into SQLite
build_movielens_only.py	Builds a demo catalog from MovieLens alone (no TMDB)
train_strategy_selector.py	Earlier full retraining script (cleaned-label protocol, tuned CV candidates); still runnable
selector_experiments.py	Fourteen-candidate comparison with Optuna; --deploy-saved NAME refits the chosen model on all users and writes data/processed/strategy_eval.json and the model files that the app and KPI report use (**this produced the currently deployed selector**)
selector_phase2.py	Two-objective evaluation: classification and recommendation value, bootstrap CIs, cold-start-by-history thirds, TreeSHAP importances
analyze_selector.py, label_experiments.py, content_experiments.py	Diagnostics: label reliability and ceiling; label-definition comparison; content-model variants
run_benchmark.py	Frozen-benchmark comparison (11.8)
compare_smote.py, smote_seed_robustness.py, smote_vs_class_weight.py	The SMOTE investigation (Module 3)

11.12 Where the saved results come from

Artifact	Created by	Read by
reports/kpi_report.json	generate_kpi_report.py	app.py -> kpi_view.render_report
reports/strategy_comparison.json	run_benchmark.py	comparison_view.render_comparison
data/processed/strategy_eval.json	selector_experiments.py --deploy-saved	generate_kpi_report.py (classifier KPIs)
data/processed/strategy_classifier.pkl (+ strategy_label_encoder.pkl, strategy_context.pkl)	same	app.recommend
data/benchmark/baseline_frozen.json	run_benchmark.py (first run)	run_benchmark.py (drift check)

Chapter 12. End-to-End Execution Flow

12.1 The sequence in the actual implementation



Sequence of one live recommendation

```
sequenceDiagram
  actor U as User
  participant A as app.py / runner.py
  participant E as load_engine (cached)
  participant S as Strategy selector
  participant G as CandidateGenerator
  participant R as MultiObjectiveRanker
  participant D as SQLite run log
  U->>A: enter user ID, K, weights; click Generate
  A->>E: catalog, ratings sample, ranker (cached)
  A->>S: build_features(user ratings) then predict
  S-->>A: strategy and confidence
  A->>G: source weights from strategy_source_weights
  G-->>R: pool of 200 candidates (IR, CF, popularity)
  R->>R: greedy pick, six objectives
  R-->>A: ranked top-K with score breakdown
  A->>D: record_run(weights, strategy, results)
  A-->>U: movie cards and CSV export
```

12.2 Stage by stage

#	Stage	What happens internally (function)
1	**Application initialisation**	`streamlit run app.py` executes the script on every interaction. It applies the theme, draws the header and the section tabs, reads `?view=` from the URL into `st.session_state["nav"]`, and decides `data_ready` (catalog and ratings files exist)

		and `has_db` (SQLite built)
2	**Data loading**	`load_engine()` (cached with `@st.cache_resource`, so it runs once per server process): reads `movies_merged.csv`, the first 400,000 rows of the ratings file, loads the IR artifacts, builds `MultiObjectiveRanker` (which builds `CatalogStats`, the bias predictor and the candidate generator's popularity and CF structures), loads the classifier
3	**Dataset preprocessing**	Already done offline by `src.ingestion.build_dataset` (join, clean). The online path only reads the results
4	**Feature generation**	`build_features(user_ratings, movies, context)` computes the profile features, popularity-head shares, engine-strength scores and three inner backtests for that one user
5	**Model loading**	`StrategyClassifier.load()` restores the model and encoder; `strategy_context()` (cached) restores the reference population
6	**User profile construction**	The user's ratings are selected from the in-memory sample (`ratings[ratings.userId == user_id]`); a user outside the 400k-rating sample cannot be served (the UI explains this)
7	**Strategy selection**	`classifier.predict(features)` returns `predicted_strategy` and `confidence` (maximum class probability)
8	**Candidate generation**	`CandidateGenerator.generate(user, strategy, confidence)`: `strategy_source_weights` gives the three source weights; IR neighbours of the user's top-10 rated movies (genre-gated), user-kNN neighbours' favourites, and the 200 most-rated movies are normalised, weighted and summed; rated movies removed; top 200 kept
9	**Recommendation scoring**	For each candidate the four user/item objectives are computed once (`fixed` dict)
10	**Multi-objective ranking**	The greedy loop in `MultiObjectiveRanker.rank` selects K movies using diversity and coverage relative to the selected list
11	**Result presentation**	`runner.show_results` renders movie cards (rank, title, year, genre chips, reason, six objective bars, rank score), summary tiles and a CSV export; `_log_run` writes the run to `recommendation_runs`
12	**Evaluation and monitoring**	Offline: `generate_kpi_report.py`, `run_benchmark.py` write JSON read by the KPI page. Online: the run's elapsed time is shown in the status line and stored; the Past runs tab lists every run

12.3 The offline training sequence

1. `python -m src.ingestion.build_dataset` -> `movies_merged.csv`.
2. `PYTHONPATH=. python scripts/verify_ir_engine.py` -> TF-IDF artifacts.
3. `PYTHONPATH=. python scripts/build_user_db.py` -> SQLite.
4. `PYTHONPATH=. python scripts/selector_experiments.py 5000 --deploy-saved multiclass/lightgbm` -> classifier, context, `strategy_eval.json` (an earlier full run of the same script with `--trials 20` created the saved comparison `selector_experiments_inner3.json` that `--deploy-saved` reads).
5. `PYTHONPATH=. python scripts/run_benchmark.py` -> `strategy_comparison.json`, EASE model.
6. `PYTHONPATH=. python scripts/generate_kpi_report.py 400000 1000` -> `kpi_report.json`.
7. `streamlit run app.py`.

Chapter 13. Project Differentiation and Competitive Analysis

> **Rule for this chapter:** we do not claim CineRankX invented any single technique. TF-IDF, cosine similarity, user-kNN, popularity baselines, MMR-style diversity, Bayesian ratings, gradient boosting and algorithm selection are all established. The contribution is in the *combination*, the *honest evaluation*, and the *engineering*.

13.1 Comparison table

Property	Popularity	Collaborative	Content-based	Hybrid (fixed blend)	Single-objective ranker	CineRankX (as implemented)
Personalisation	none	high (needs history)	medium (needs item text)	medium-high	depends on score	high: neighbours, content seeds, bias predictor
Strategy selection per user	no	no	no	no (one blend for all)	no	**Yes**: classifier predicts strategy and confidence; used as a weighted prior
Multi-objective ranking	no	no	no	rarely	no (one score)	**Yes**: six weighted objectives
Diversity	low	low	low (over-specialised)	low	low	explicit objective (MMR-style)
Novelty	none	low	medium	low	low	explicit objective (self-information)
Coverage	very low (0.38% in our benchmark)	medium	medium	medium	low	genre-coverage objective; catalog coverage 5.7% in benchmark (partial: per-item proxy)
Cold-start handling	**good**	poor	good for new *items*	partial	partial	**partial**: popularity prior, EASE fold-in from 5 ratings (Hit Rate@10 0.698 vs 0.609 baseline, simulated cohort)
Explainability	high	medium	high	low	low	high: score breakdown per movie plus a plain-language reason
Adaptability	none	retrain	retrain	retrain	retrain	weights adjustable live; classifier retrained offline; **no online learning**

13.2 Modern platforms

Public descriptions of large streaming recommenders (for example Gomez-Uribe and Hunt, "The Netflix Recommender System", ACM TMIS 2015) describe *many* algorithms combined and a personalised ranking stage, plus attention to diversity and context. We do not know their internal details and make no claim of equivalence. Relevant comparison points: such systems learn from implicit feedback at scale, run online experiments and optimise measured engagement, which CineRankX (offline, 20M ratings, no live users) does not.

13.3 What is our actual innovation?

1. **A per-user strategy selector trained on backtested strategy performance**, with the prediction used as a *weighted prior* over candidate sources (not a hard switch). Related literature on predicting when a recommender fails and choosing algorithms per user exists (for example Ekstrand and Riedl, "When Recommenders Fail", RecSys 2012); our implementation and findings are specific to this stack.
2. **Six explainable objectives in one greedy re-ranker** with tunable weights, exposed as sliders, and a score breakdown shown for every movie.
3. **A rigorous, reproducible evaluation culture**: leak-free labels, a hash-based frozen split, a self-contained frozen baseline with a drift check, bootstrap confidence intervals and paired tests, and a systematic diagnosis of why the classifier is limited.
4. **An honest negative result**: with this label definition, per-user "best strategy" is only weakly identifiable (agreement 65-73%, oracle ceiling about 0.76), yet following the selector still gives a statistically significant hit-rate gain over always-popularity.

13.4 Why it is not "just another movie recommendation website"

A typical demo returns a list ranked by one score. CineRankX (a) chooses and weights methods per user, (b) shows *why* each movie was ranked (six scores plus a reason), (c) lets the operator change trade-offs live, (d) logs every run with its weights and strategy, (e) provides a User 360 view for each user's history, and (f) reports its own measurement limits.

13.5 Why six objectives, and why per-user selection

Accuracy-only lists are repetitive and popular. Each added objective addresses a documented failure: diversity (repetition), novelty (filter bubble), coverage (concentration on few titles), popularity/quality (reliability), predicted rating (the user's likely score). Per-user selection is useful because evidence differs by user: light users gain most (selector hit-rate 0.185 vs 0.152 for popularity among users with 21-40 ratings), heavy users hardly at all (0.070 vs 0.069).

13.6 Limitations of the approach (say them first)

Content-based hit-rate is low (0.016); labels are noisy; classifier accuracy is below the majority rate; the strategies in the backtest are simplified versions of the production sources; evaluation is offline (no user feedback); one dataset.

13.7 What could make this publishable

(Suggestions, not claims.) Evaluate on several datasets; compare with learned hybrids and published algorithm-selection methods; add online or counterfactual evaluation; formalise the uncertainty-aware label definition and

release the labelling protocol; study the trade-off curves among objectives; and test the improved content model and larger-budget labels end to end. A paper-quality claim would rest on the **evaluation methodology and the label-reliability finding**, not on high classifier accuracy.

Chapter 14. Testing, Debugging and Software Engineering

14.1 How to run the tests

```
cd CineRankX
python -m pytest -q            # 272 test cases in 12 files (last full run: all passed)
python -m pytest tests/test_ranking_engine.py -q
```

14.2 What each test file validates

File (cases)	What it protects
`test_ingestion.py` (6)	loaders read the right files/schemas; the join builds `text_chunk`; unmatched movies are dropped; `clean_text` lowercases, removes stop-words, stems, handles empty input
`test_ir_engine.py` (18)	Trie insert/search is case-insensitive, respects limit, ignores empty titles; TF-IDF `fit/search/similar_to/save/load`; unfitted engine raises; the façade `IREngine` combines both
`test_strategy_selector.py` (43)	feature extraction (genre entropy and top-genre share distinguish focused from eclectic users); `select_best_strategy` tie rules; popularity from observed counts, not TMDB; CF candidate budget capped; `_build_cb_profile_vector` caps and weights; classifier fit/predict/save/load, per-class metrics, absent classes flagged; SMOTE only on the training fold and never produces held-out rows
`test_ranking_engine.py` (72)	weights normalise and reject invalid input; each objective in [0, 1] and behaves as defined (first pick maximally diverse, near-duplicate scores low, max-not-mean similarity, coverage fractions, Bayesian shrinkage, novelty rewards rare items); predictor fallbacks; `strategy_source_weights` interpolation; candidates exclude rated movies and respect pool size; ranker returns K results with all objective columns
`test_evaluation.py` (71)	KPI names equal config; every metric function on hand-computed cases (P@K divides by K, reciprocal rank, NDCG ideal, AP denominator capped at K, coverage, diversity, novelty); the split gives every user train and test; cold-start modes; `build_report(strict)` refuses missing KPIs
`test_selector_pipeline.py` (11)	richer features; profile/holdout split is disjoint; holdout labels never use holdout ratings; label cleaning margin from config; cross-validation beats chance on separable data and not on noise; fold shrinkage for rare classes
`test_strategy_context.py` (6)	fast content score equals the reference implementation; inner scores deterministic and bounded; a user's own profile is never their neighbour; training and serving produce identical columns
`test_formulations.py` (6, 11 with parametrisation)	`winner` tie rule; every formulation learns a planted signal and returns valid probabilities; pairwise ignores ties; ranking expansion keeps three rows per user; decision quality by hand; pickling
`test_utility_stats.py` (6)	bootstrap CIs, paired bootstrap, McNemar, Wilson; utility selector fallback; `choose_tau`; content model
`test_benchmark.py` (8)	split deterministic and order-independent; cohorts disjoint; cold users truncated; exclusions never use test items; metrics by hand; **baseline is frozen, independent (AST check that it imports only numpy/pandas) and unchanged when CineRankX is retrained**; EASE beats popularity on structured data

`test_ui_results.py` (9)	ranked-CSV validation (missing columns, duplicates, invalid ranks), KPI summary, title/year split, explanation text, genre lookup
`test_user_store.py` (6)	user summary and unknown users; history pagination/filter/sort with SQL-injection-safe sorting; preferences/timelines; user search; run history round trip

14.3 Types of testing in the project

- **Unit tests** (all of the above), mostly on small synthetic frames so they run in seconds.
- **Integration checks**: Streamlit's `AppTest` was used during development to run each page and a full recommendation without a browser; the verification scripts (`verify_*.py`) run each engine on real data.
- **Model testing**: planted-signal tests (the model must find a known pattern) and noise tests (it must not beat chance on random labels).
- **Data validation**: ranked-CSV validation (`prepare_results`), KPI completeness (`missing_from`), benchmark fingerprint check.
- **API testing / frontend testing**: there is no HTTP API; the UI is tested through `AppTest` and by manual use. **No automated browser test suite exists** (UNVERIFIED claims should not be made about it).

14.4 Error handling and logging

- Missing artifacts degrade gracefully: no classifier or context -> strategy `None` and the default prior blend; no database -> the profile page shows the build command; no KPI report -> an info message.
- Invalid input is rejected before running: user ID not in the sample disables the Generate button; all-zero weights raise a clear error; uploaded CSVs are size-limited and column-checked.
- Runs are logged in SQLite (success or error with message). The Python `logging` module is not used; scripts print progress.

14.5 Dependencies, serialisation, reproducibility

- `requirements.txt` lists unpinned packages (a reproducibility weakness; exact versions used while writing this guide are in Chapter 4).
- Models are saved with `pickle`; sparse matrices with `save_npz`; EASE with `np.savez`. *Security note:* never unpickle files from untrusted sources. A saved model refers to classes by module path, so factories must live in importable modules (`estimators.py`), not in scripts (a bug found while deploying).
- Seeds: 42 for model training and sampling; 20260101 for the benchmark hash split. Caches (labelled frames, benchmark pickle) are keyed by their settings.

14.6 Realistic debugging scenarios

Symptom	Likely cause	Fix
Page shows an old behaviour after a code edit	Streamlit caches imported modules; the running process has the old code	Restart `streamlit run app.py`
`StreamlitDuplicateElementKey: card_topk`	the same UI block was accidentally rendered twice (a duplicated code section)	Remove the duplicate; keys must be unique per element
`ModuleNotFoundError: yaml`	dependencies not installed	`pip install -r requirements.txt`
`ValueError: pandas dtypes must be int, float or bool ... niche_fraction: object`	a boolean column produced by `.where` had object dtype	cast with `.astype(float)` (fixed in `profile_features.py`)
Classifier cannot be loaded after training (`load()` returns False)	the model was pickled with a factory defined in a script (`__main__`)	move the factory into an importable module (`estimators.py`)
"Strategy selector not trained" when	`strategy_eval.json` or the	run `selector_experiments.py ... --`

generating the KPI report	classifier/context is missing	`deploy-saved`
Profile page says the user store is not built	`cinerankx.db` missing	`PYTHONPATH=. python scripts/build_user_db.py`
"User N is not in the loaded ratings sample"	live recommendations use only the first 400,000 ratings (about 2,900 users)	choose a smaller user ID, or raise `RATINGS_LIMIT` (more memory and load time)
Cold-start KPI is missing	MovieLens has no natural cold users	the evaluation simulates them; check `cold_start_mode`
Site stops responding after idling	the preview server was stopped	restart `streamlit run app.py`

Chapter 15. Limitations and Future Scope

15.1 Real limitations (current implementation)

Area	Limitation	Evidence
Labels	per-user best strategy is only about 65-73% reproducible; 60% of winners lie within one standard error	Chapter 9
Classifier	below the majority rate on plain accuracy (0.617 vs 0.644); content-based class only 2.3% of users	`strategy_eval.json`
Content-based	hit-rate 0.016; improved to 0.045 in an experiment not yet deployed	`content_experiments.json`
Cold start	MovieLens has no natural cold users; evaluation is simulated; gain over popularity is moderate (0.698 vs 0.609)	`strategy_comparison.json`
Data sparsity	0.53% density; CF uses a 5,000-user x 2,000-movie window	`candidates.py`
Popularity bias	frozen baseline covers 0.38% of the catalog; CineRankX 5.7% (old pipeline reached 7.9%, so the newer pipeline gave up some coverage)	`strategy_comparison.json`
Scalability	the live app loads 400,000 ratings; user-user matrices are quadratic; the in-memory ranker is single-process	`app.py`
Evaluation	offline, one dataset; relevance proxy for search; hit-rate depends on holdout size and K	`search_metrics.py` docstring
User feedback	no clicks, watches or explicit feedback are collected	repository
Security	no authentication layer; pickle persistence	repository
Weak search relevance proxy	Recall@10 near zero by construction	KPI report note
Reproducibility	dependency versions unpinned; run-to-run variation of about 0.02 accuracy on 999 users	Chapter 9

15.2 Future work (not implemented)

Idea	Why it might help	Status
Utility-based selection with calibrated uncertainty	optimise the quantity users care about (hit-rate)	prototype exists and ties the classifier
Contextual bandits / online learning	learn strategy value from real clicks	needs live feedback; **not built**
Reinforcement learning	sequential session optimisation	only reasonable with interaction logs; **not built**
Fusion of strategy lists (hybrid beyond one prior)	combine strengths per item	**not built**
Better content embeddings (sentence embeddings, cast/crew, plot summaries)	stronger content strategy	would relax the "no deep learning" scope
Deep recommenders (neural CF, sequence models)	larger accuracy	out of the explainable scope
Real-time feedback loop	adapt to the current session	not built
Explainable recommendations from neighbours ("because you liked X")	richer explanations	partly present (per-objective reasons)
Larger labelling budget and uncertainty-aware labels	more reliable targets	measured (agreement 0.733), not yet deployed

Larger evaluation (more users, several datasets, temporal splits)	tighter intervals, generality	not done

Chapter 16. Viva Preparation: 164 Questions with Model Answers

How to use this chapter. Each question has a difficulty tag: **[Basic]** what is it; **[Intermediate]** how does it work; **[Advanced]** why did we choose it; **[Expert]** what if we change it; **[Critical]** what is the weakness. Answers are written to be *spoken*, in first person plural, and tied to CineRankX facts. Numbers come from saved artifacts (Chapter 9-10). Do not memorise answers word for word: understand the reasoning, then say it in your own words.

A. Project introduction and motivation

A1 [Basic] What is CineRankX in one minute?
CineRankX is a movie recommender built on MovieLens 20M and TMDB metadata. It does two things a normal recommender does not: it predicts for each user which recommendation strategy (content-based, collaborative or popularity) is likely to work best, and it ranks candidate movies on six objectives (relevance, diversity, novelty, coverage, popularity/quality, predicted rating) instead of a single similarity score. A Streamlit web app lets you run it, inspect any user's history and view the evaluation results.

A2 [Basic] What problem does it solve?
Most recommenders use one fixed method and rank by one score. That ignores that different users give different methods different amounts of evidence (a user with five ratings versus one with five hundred), and it produces repetitive lists. CineRankX addresses per-user variation and list quality.

A3 [Basic] What are the three engines?
Module 2, the classical IR engine (TF-IDF with cosine similarity plus a Trie for title autocomplete); Module 3, the adaptive strategy selector (a classifier on user features); Module 4, the multi-objective ranking engine (candidate generation and greedy six-objective re-ranking). Module 1 is data ingestion. An evaluation suite and a UI sit around them.

A4 [Intermediate] How did the project start and evolve?
It began in August 2026 with data ingestion and a decision to move from a small MovieLens plus TMDB 5000 pairing (36% join rate) to MovieLens 20M plus TMDB 2023 (98% join). Modules 2, 3 and 4 and the 23-KPI evaluation suite followed within days, then a Streamlit dashboard, a redesigned multi-page UI with a SQLite user store, and finally a leak-free labelling pipeline, a frozen benchmark and diagnostic experiments on the classifier. (Chapter 1 has the dated timeline.)

A5 [Intermediate] What did you actually build versus plan?
Everything in the original plan exists: the three engines, the evaluation suite and an interface. What we did not achieve is a high-accuracy strategy classifier: we planned a classifier, built it, and then showed that its accuracy is limited by label reliability.

A6 [Advanced] Why did you choose movie recommendation?
It has an open, standard dataset (MovieLens) with enough users and ratings for collaborative filtering and for training a per-user model, a rich metadata source (TMDB) for content-based methods, and clearly measurable offline evaluation. It also lets us demonstrate each recommender family and compare them fairly.

A7 [Expert] If you were told to remove one of the three engines, which would you remove?
The classical IR engine's Trie autocomplete is not used by the recommendation path, but TF-IDF similarity is used for content candidates and for the diversity objective, so I would remove the Trie first. Removing the strategy selector would still leave a working recommender with the fixed prior blend, but we would lose the per-user idea that is the project's core.

A8 [Critical] What is the biggest weakness of the project?
The strategy classifier does not beat the majority-class rate on plain accuracy (0.617 vs 0.644) because its labels are noisy. We measured this rather than hiding it, and showed that the strategy choice still gives a statistically significant hit-rate gain over always-popularity.

B. Problem statement and research gap

B1 [Basic] State the problem statement.
Given a user's possibly short rating history and a movie catalog with text and metadata, choose the most suitable strategy for that user, generate candidates accordingly, and rank them on six objectives to produce a top-K list that is accurate on held-out interactions while remaining diverse, novel and explainable.

B2 [Basic] What is information overload?
Too many items for a person to browse. MovieLens has 27,278 movies and TMDB about 1.5 million, so users need a short, relevant list.

B3 [Intermediate] Why is one strategy not enough for everyone?
Evidence differs per user: collaborative filtering needs history and neighbours, content-based needs a coherent taste profile, popularity needs nothing. In our data the best strategy on a hidden holdout was popularity for 64% of users, collaborative for 33% and content-based for 2%, and the mix varies with history size (the selector gains most for users with 21-40 ratings).

B4 [Intermediate] What is the research gap?
Many hybrid systems blend strategies with one global weighting, and many re-rankers optimise one or two objectives. CineRankX studies user-specific strategy selection together with a six-objective re-ranker in one explainable pipeline and, importantly, measures how reliable strategy labels actually are.

B5 [Advanced] Why not just use the best strategy overall (popularity)?
Popularity is strong offline, which is why it is our frozen baseline, but it is not personalised, has very low catalog coverage (0.38% in our benchmark) and cannot serve users with distinct tastes. The point is to beat it where evidence allows. On the benchmark CineRankX reaches Hit Rate@10 of 0.877 versus 0.601.

B6 [Expert] What if every user's best strategy were popularity?
Then a selector would add nothing and a classifier would equal the majority rule. We see something close to this: 64% of labels are popularity. That is why we compare against the majority baseline and report hit-rate gains: the classifier is useful only to the extent that it identifies users who gain from a different strategy.

B7 [Critical] Is your research contribution really new?
Not as individual techniques; each is established and we say so. The contribution is the combination, the leak-free labelling and frozen-benchmark methodology, and the documented negative result on label reliability. Related work on predicting recommender failure and per-user algorithm selection exists (for example Ekstrand and Riedl, RecSys 2012).

C. Architecture and system design

C1 [Basic] Describe the architecture.
Five layers: raw data and SQLite; ingestion producing `movies_merged.csv`; three recommendation engines; an evaluation layer (23-KPI suite plus frozen benchmark); and a Streamlit presentation layer. (Figure in section 3.5.)

C2 [Basic] Where does a recommendation request flow?
`runner.py` collects inputs; `app.recommend` builds features and predicts a strategy; `MultiObjectiveRanker.rank` generates candidates and ranks greedily; results are shown and logged to SQLite.

C3 [Intermediate] Why is there both src/evaluation and src/benchmark?

`src/evaluation` is the original suite that computes all 23 named KPIs on a sample (per-user random splits). `src/benchmark` was added later because we wanted a stricter, frozen protocol: a hash-based split that cannot change, identical cohorts for every strategy, and a baseline that cannot drift.

C4 [Intermediate] Why SQLite for the user store?

We need one user's ratings out of 20 million rows in milliseconds, with server-side filtering and pagination. SQLite with an index on `(userId, ts)` does that in a single file without a server. A multi-user production system would use PostgreSQL.

C5 [Advanced] Why Streamlit rather than Flask plus React?

Time and scope: Streamlit lets Python models become a working interface with little code, which fitted a mini-project. The costs are less layout control and a script re-run model; we managed these with caching (`st.cache_resource`) and custom CSS.

C6 [Advanced] How does caching work and why is it needed?

Streamlit re-executes the script on every interaction. `load_engine()` is decorated with `@st.cache_resource`, so the catalog, ratings sample and ranker load once per server process. The KPI JSON uses `@st.cache_data` keyed by file modification time so the page shows new results as soon as a report is regenerated.

C7 [Expert] What happens if the classifier file is missing?

`StrategyClassifier.load()` returns False, `recommend` leaves `strategy = None`, and `strategy_source_weights` returns the fixed prior (collaborative 0.60, content 0.20, popularity 0.20). The app keeps working; there is a unit test for the missing-strategy path.

C8 [Critical] What are the scalability limits?

The live app loads only 400,000 ratings (about 2,900 users) and the collaborative source uses a 5,000-user by 2,000-movie window, because the user-user similarity is quadratic. Profiles use the database and cover all 138,493 users, but live recommendations are limited to the sample. For millions of users we would need approximate nearest neighbours or a learned model, a feature store and a serving layer.

D. Dataset and preprocessing

D1 [Basic] What datasets do you use?

MovieLens 20M (20,000,263 ratings, 138,493 users, 27,278 movies, 465,564 tags) and the TMDB Movies Dataset 2023 (about 1.5 million movies with overviews, genres, keywords, vote statistics and poster paths).

D2 [Basic] Why MovieLens 20M?

It is a standard benchmark, large enough for collaborative filtering and classifier training, small enough for a laptop, and it ships `link.csv` mapping movies to TMDB ids. We also considered `ml-latest-small` (too small) and `ml-32m` (the join rate to TMDB 5000 would fall to about 5%).

D3 [Basic] Why integrate TMDB?

MovieLens has only titles, genres and tags. TMDB adds plot overviews, keywords and vote statistics, which give the content-based engine real text and the quality objective a vote count and average.

D4 [Intermediate] How do you join them?

`link.csv` gives `tmdbId` for each MovieLens movie; we inner-join on `tmdbId == TMDB id`. 26,743 of 27,278 movies (98.04%) match; the rest have no TMDB id or are missing from the export.

D5 [Intermediate] How is text prepared?

We concatenate overview, genre names and keywords into `text_chunk`, then `clean_text`: lowercase, keep alphabetic tokens, remove English stop-words and Porter-stem. Cast and crew are not available in this TMDB export, and we deliberately did not backfill them.

D6 [Intermediate] What data problems did you find?

Missing overviews (89 movies), 252 movies without a TMDB id, singular versus plural filenames across MovieLens releases, datetime-string timestamps instead of integers, and TMDB genres/keywords as comma-separated strings instead of JSON. No duplicate movie ids and no missing values in the classifier feature table.

D7 [Advanced] How do you split data and why?

Three splits serve different purposes: a per-user profile/holdout split to create strategy labels; a user-level 80/20 stratified split plus 5-fold CV to evaluate the classifier; and a hash-based train/validation/test split of ratings (70/10/20) for the recommendation benchmark. Splitting by user for the classifier prevents the same user appearing in train and test.

D8 [Expert] Why a hash split instead of random shuffling?

A hash of (user, movie, seed) gives every rating a fixed uniform number, so the split is identical on every run and machine, independent of row order and sample size, and needs no stored file. That is what lets us call the test set "frozen".

D9 [Critical] What are the weaknesses of your data?

MovieLens ratings stop in 2015 (so newer movies and tastes are absent); users have at least 20 ratings so there are no natural cold-start users; ratings come from one site's volunteers; the join drops 2% of movies; content is limited to overview, genres and keywords.

D10 [Critical] How do you handle cold start in evaluation?

Since no natural cold users exist, we simulate them: take real users, keep only their first five training ratings (by a deterministic order) and score them on their untouched held-out items. The code records `mode = "simulated"` so it is never presented as natural.

E. Collaborative filtering

E1 [Basic] What is collaborative filtering?

Recommending items that similar users liked, using only the pattern of who rated what.

E2 [Basic] User-based versus item-based?

User-based finds similar users and recommends what they liked; item-based finds items similar (by co-ratings) to what the user liked. Our `candidates.py` is user-based nearest-neighbour; our EASE model in the benchmark is item-item.

E3 [Intermediate] Explain the user-item matrix and sparsity.

Rows are users, columns are movies, entries ratings. MovieLens 20M has 20,000,263 ratings among 138,493 x 27,278 possible cells, about 0.53% filled, so most pairs of users share few rated movies.

E4 [Intermediate] How is similarity computed?

Cosine similarity between rating vectors: dot product divided by the product of norms, with unrated entries as zero. Worked example in 6.A.4: for user U1 the similarity with U2 is 0.80.

E5 [Intermediate] How does your CF candidate generator work?

We find the 25 most similar users in a pool of 5,000 users and 2,000 most-rated movies, take each neighbour's 20 highest-rated movies, and score a candidate by the sum of the similarities of the neighbours who propose it.

E6 [Advanced] Why did you not use matrix factorisation?

We kept the explainable classical scope. Matrix factorisation (SVD) was explored informally and discarded in favour of EASE, which is also classical, closed-form and strong on top-N tasks. The repository contains no SVD implementation.

E7 [Advanced] What is EASE and why did you choose it?

EASE is a linear item-item model: ridge regression of every item on all other items with a zero diagonal, in closed form `B = -P / diag(P)` with `P = (X'X + lambda I)^-1`. We chose it because it trains in seconds on the 6,000 most-rated movies, scores a cold user from only five ratings by fold-in, and beat popularity clearly on our benchmark (Hit Rate@10 0.877 vs 0.601).

E8 [Expert] What happens if you use Pearson correlation instead of cosine?

Pearson mean-centres each user, removing rating-scale bias (harsh vs generous raters) and usually helps on dense data, but on very sparse vectors the centred overlap is small and noisy. It is a reasonable improvement we have not tested.

E9 [Critical] Weaknesses of CF here?

Cold start; popularity-biased neighbours; the 5,000-user window means most users are not neighbours of each other; no normalisation of ratings; quadratic similarity cost.

F. Content-based filtering

F1 [Basic] What is content-based filtering?

Recommend items similar in content (text, genres) to items the user liked, without using other users.

F2 [Basic] What content do you use?

`text_chunk_clean`: stemmed overview, genre names and keywords from TMDB; plus genre sets for gating and diversity.

F3 [Intermediate] How is a user profile built?

In the label backtest, as the rating-weighted sum of the TF-IDF rows of the user's 20 best-rated movies; the movies most cosine-similar to that vector (excluding rated ones) are recommended. In the ranking engine, the user's top-10 rated movies seed `similar_to` queries.

F4 [Intermediate] What is the genre gate?

A multiplier `floor + (1 - floor) x Jaccard(genres)` applied to IR candidates so that keyword-only matches (such as toy-themed horror films for *Toy Story*) are suppressed. Default floor 0.25.

F5 [Advanced] Why is content-based your weakest strategy?

Mean hit-rate 0.016 against about 0.115 for collaborative and popularity. Reasons: the text is short and generic; the profile averages many different movies; random-split offline evaluation rewards methods that capture popularity and co-rating; and TF-IDF over overview words says little about quality.

F6 [Expert] How did you try to improve it?

We tested representations on 2,000 users with a development/confirmation split. Bigram TF-IDF with 20,000 terms raised hit-rate from 0.0156 to 0.0246; adding a quality prior (similarity times a quality percentile to the power 2) raised it to 0.0452, about 2.9 times better (95% CI of the gain +0.027 to +0.033). Mean-centred profiles and appended genre blocks made it worse. It is still far below popularity (0.119).

F7 [Critical] Is the quality prior still "content-based"?

Partly. It makes the strategy lean towards well-rated similar movies, so some gain is a generic quality effect. We say so explicitly and keep it as a separate, labelled variant.

F8 [Critical] Why is cast/director similarity missing?

The TMDB export in use has no cast or crew; backfilling it for only a minority of movies would make the feature inconsistent across the catalog.

G. Popularity-based recommendation

G1 [Basic] What is popularity-based recommendation?

Recommend the items that most users rated. No personal data required.

G2 [Basic] How do you define popularity?

By observed rating count in the data (the 200 most-rated movies for the candidate pool, all movies sorted by count for the frozen baseline), not by TMDB's `popularity` column.

G3 [Intermediate] Why not TMDB popularity?

TMDB popularity is a live, recency-weighted number, while MovieLens ratings stop in 2015. The TMDB top-10 was touched by only 39% of users against 87% for the most-rated top-10, which made the popularity strategy look artificially weak and the cold-start baseline artificially easy to beat. We found and fixed this (commit `c273ba3`).

G4 [Intermediate] What is the Bayesian weighted rating?

`WR = v/(v+m) R + m/(v+m) C`, with `R` the movie's average, `v` its votes, `m` the median vote count and `C` the mean average. It pulls ratings of rarely voted movies towards the mean so three 10/10 votes cannot beat a film rated 8.4 by 20,000 people. Used in the popularity/quality objective, then min-max scaled.

G5 [Advanced] Why is popularity such a strong baseline?

In offline evaluation with random held-out ratings, many users' future ratings fall on already popular movies, so recommending the most-rated ones scores well. This is a known effect in the literature on top-N evaluation (for example Cremonesi, Koren and Turrin, RecSys 2010). In our benchmark it reaches Hit Rate@10 of 0.601, which is why we freeze it as the reference.

G6 [Expert] What is popularity bias and how do you counter it?

Popular items collect more exposure and ratings and so appear more popular; recommendations concentrate on few titles. We counter it with novelty (self-information), coverage and the quality objective, and we measure catalog coverage (baseline 0.38%, CineRankX 5.7%).

G7 [Critical] Isn't a baseline that wins 64% of labels a sign the project adds little?

It is a sign that popularity is hard to beat *for many users*, which we report openly. Where personalisation can help, it does: Hit Rate@10 0.877 vs 0.601 overall, and the selector adds +0.0155 over always-popularity (CI +0.011 to +0.020).

H. Hybrid recommendation

H1 [Basic] What is a hybrid recommender?

One that combines several recommendation methods.

H2 [Basic] Is CineRankX hybrid?

Yes in two senses: it blends three candidate sources with weights, and it combines six objectives in the ranker.

H3 [Intermediate] Hybrid versus strategy selection versus switching?

Hybrid blends scores from several methods; strategy selection predicts which method suits a user; switching uses one method chosen by a rule. In CineRankX the prediction is a weighted prior over sources (so it is selection feeding a blend), not a hard switch.

H4 [Intermediate] Exactly how does the prediction change the blend?

`strategy_source_weights`: strength `s = clip((confidence - 1/3)/(2/3), 0, 1)`; weights `(1 - s) x prior + s x one_hot(strategy)` with prior (collaborative 0.60, content 0.20, popularity 0.20). At confidence 0.7 and strategy collaborative: collaborative 0.82, content 0.09, popularity 0.09.

H5 [Advanced] Why a weighted prior instead of a hard switch?
The classifier's probabilities are uncertain and its labels noisy, so a hard decision would amplify errors. A prior that smoothly moves towards the predicted strategy as confidence grows degrades gracefully; at chance confidence it reverts to a sensible default.

H6 [Expert] Why not average the three ranked lists directly (list fusion)?
It is a good idea and listed as future work. It needs per-strategy ranked lists and a fusion rule (for example reciprocal rank fusion); we did not implement it, and the repository contains no such code.

H7 [Critical] Does the sources' sum favour popularity?
Each source's scores are min-max normalised first, so raw magnitudes (popularity counts) do not swamp cosine scores. An item proposed by two sources outranks one proposed by a single source at equal strength. Normalisation is a design choice that could be varied.

I. TF-IDF and cosine similarity

I1 [Basic] What is TF-IDF?
A weighting that scores a word highly if it appears often in a document (term frequency) but rarely across the collection (inverse document frequency).

I2 [Basic] What is cosine similarity?
The cosine of the angle between two vectors: 1 for the same direction, 0 for orthogonal. For L2-normalised TF-IDF vectors it equals the dot product.

I3 [Intermediate] Give the formulas.
`idf(t) = ln((1+N)/(1+df)) + 1` (scikit-learn smoothing), `tfidf = tf x idf`, then L2-normalise. `cos(a,b) = a.b / (|a||b|)`. Worked example in 6.B.2: d1 = "toy story toy adventure" gives toy 0.771, story 0.507, adventure 0.386.

I4 [Intermediate] Which TF-IDF settings do you use and why?
`max_features=5000, min_df=2, max_df=0.8, ngram_range=(1,2)`. `min_df=2` removes one-off terms, `max_df=0.8` removes near-universal terms, bigrams capture phrases, 5,000 terms limit memory.

I5 [Advanced] Why cosine instead of Euclidean distance?
Document length changes vector magnitude but not direction; cosine measures topical similarity independent of length. Euclidean distance would penalise longer overviews.

I6 [Advanced] Why stemming?
It merges inflections (*toys*, *toy*) so the same concept is one term, shrinking the vocabulary. The cost is non-words (*woodi*) and occasional over-merging.

I7 [Expert] Would word embeddings be better?
Likely for semantic similarity (synonyms), but they would break the explainable, no-deep-learning scope and need a pretrained model; TF-IDF is transparent and cheap. It would be a natural extension.

I8 [Critical] What does TF-IDF miss?
Synonyms and context, plot structure, tone, cast and director (not in our data), and quality. That is why the genre gate was necessary.

J. Multi-objective recommendation

J1 [Basic] List the six objectives.
Relevance, diversity, novelty, coverage, popularity/quality, predicted rating.

J2 [Basic] How are they combined?

A weighted sum with weights normalised to 1 (defaults 0.30, 0.15, 0.15, 0.10, 0.15, 0.15), computed greedily: each step picks the candidate with the highest total given the list so far.

J3 [Intermediate] Why greedy and not sort once?

Diversity and coverage depend on what is already chosen; the best second item depends on the first. Greedy selection is the standard MMR-style approach.

J4 [Intermediate] Define diversity.

`1 - max similarity to already selected items`; the first item gets 1. Similarity is TF-IDF cosine, with genre Jaccard as a fallback.

J5 [Intermediate] Define novelty.

Self-information `-log2 p(item)` of observed engagement (rating count plus TMDB votes), min-max scaled: rarely seen items score near 1.

J6 [Advanced] Why use max rather than mean similarity for diversity?

One near-duplicate in the list makes a candidate redundant even if it differs from the other nine; a mean would hide that.

J7 [Advanced] Is coverage a real implementation of catalog coverage?

It is a per-item genre-coverage proxy (fraction of the item's genres not yet in the list). True catalog coverage is a property of all users' lists together and is reported as a KPI; we state clearly that the objective is only a proxy.

J8 [Expert] What happens if you set all weight on relevance?

The list becomes the most relevant but most similar and popular movies. In the benchmark, relevance-only ranking scored a slightly higher NDCG@10 than the tuned multi-objective version (0.373 vs 0.362 overall) but lower diversity (0.789 vs 0.809): the objectives cost a little accuracy and buy variety.

J9 [Expert] How did you choose the weights?

Defaults come from `eval_config.yaml`. For the benchmark we searched random weight vectors on the validation split, maximising NDCG@10 subject to diversity not falling below the baseline's. The tuned relevance weight is 0.576.

J10 [Critical] Weighted sums assume trade-offs are linear. Is that a problem?

Yes: a linear scalarisation cannot reach points on non-convex parts of the Pareto front and weights are not interpretable as exchange rates. Pareto-based or constrained methods are alternatives; we chose the weighted sum because it is transparent and adjustable by sliders.

K. The strategy classifier

K1 [Basic] What does it predict?

For each user, which of three strategies (collaborative, content-based, popularity) is most likely to work best, with a probability for each.

K2 [Basic] How are training labels created?

By backtesting: split each user's ratings 70/30 into profile and holdout; run each strategy on the profile; count how many of its recommendations land in the holdout; the best hit-rate is the label (ties go to popularity).

K3 [Intermediate] What features does it use?

47 numbers per user: activity, rating statistics, genre concentration (entropy, top-genre share, 19 genre affinities), popularity of taste, time span, engine-strength scores and three "inner backtest" hit-rates computed on the visible profile (section 8.4).

K4 [Intermediate] Why "inner backtest" features?

The label measures how each strategy performs on a hidden holdout; the best available predictor of that is how the same strategies perform when the *visible* history is split again. It uses only ratings the system can see at prediction time.

K5 [Advanced] How do you prevent leakage?

Features come from the profile ratings only; labels from the holdout only; the user-level train/test split is made before any model fitting; SMOTE (disabled) only sees the training fold; CV and tuning use development users only; the held-out users are scored once.

K6 [Advanced] Why LightGBM?

It handles mixed numeric features, needs little preprocessing, trains fast and was among the best of fourteen candidates (CV macro-F1 0.467), though several models were statistically tied. We chose it for performance, speed and simplicity, not because it won decisively.

K7 [Advanced] How is the confidence used?

The maximum predicted probability becomes the `confidence` in `strategy_source_weights`, deciding how strongly the predicted strategy shifts the source blend.

K8 [Expert] What if you drop the inner-backtest features?

In an ablation (5-fold CV, balanced random forest, on the cleaned-label subset used at that stage) macro-F1 fell from about 0.47-0.48 with them to 0.43 without them, so they carry real information. (That ablation predates the full-population evaluation.)

K9 [Expert] What if you drop all ambiguous users from the data?

Accuracy rises (it was 0.636 on a cleaned subset) but only because we removed hard cases; it does not describe real users. We report the full-population number (0.617).

K10 [Critical] Why is your classifier worse than the majority baseline?

On plain accuracy it scores 0.617 against 0.644 for always predicting popularity. The labels are only about 65-73% reproducible and a perfect model would reach about 0.76; with a dominant class, a model that tries to recover minority classes pays in accuracy. It beats the baseline on balanced accuracy (0.469 vs 0.333), AUC (0.735 vs 0.5) and recommendation value (+0.0155 hit-rate).

K11 [Critical] If accuracy is only 61.7%, why should we trust the system?

Because the system's value is not the classifier's accuracy but the quality of the recommendations. The ranking pipeline reaches Hit Rate@10 0.877 against 0.601 for the frozen baseline, and following the selector gives a significant gain over always using popularity. The classifier's weakness is measured, explained and bounded by label noise, not hidden.

K12 [Critical] Why not use a neural network for the classifier?

The data is tabular with about 5,000 labelled users, where gradient-boosted trees are usually at least as good as neural networks, and the bottleneck is label noise rather than model capacity (fourteen models landed in a narrow band). A neural network would reduce explainability and add cost without addressing the bottleneck.

L. LightGBM and machine learning

L1 [Basic] What is a decision tree?

A model that splits data by feature thresholds to reach leaf predictions.

L2 [Basic] What is gradient boosting?

Building trees sequentially, each one fitted to the errors (gradients) of the ensemble so far, and adding it with a small learning rate.

L3 [Intermediate] What are gradient and Hessian here?

For softmax cross-entropy, per class `g = p - y` and `h = p(1-p)`: the first and second derivatives of the loss with respect to the raw score. Leaf value `w = -G/(H + lambda)`.

L4 [Intermediate] What is leaf-wise growth?

LightGBM always splits the leaf with the largest loss reduction rather than growing all leaves at one depth, giving more accurate but more overfit-prone trees; `num_leaves` and `min_child_samples` control it.

L5 [Intermediate] Which hyperparameters did you tune and how?

`n_estimators` (357), `learning_rate` (0.022), `num_leaves` (27), `min_child_samples` (50), row and column subsampling (0.60, 0.46) and `reg_lambda`, by Optuna's TPE sampler with 3-fold CV macro-F1 on development users only, plus a sample-weight multiplier based on label clarity.

L6 [Advanced] What is overfitting and how do you see it here?

Memorising training noise: training accuracy 0.77-0.88 against 0.61-0.63 on held-out users. Remedies: small learning rate, limited leaves, minimum leaf size, subsampling, L2 regularisation, CV.

L7 [Advanced] Why macro-F1 for model selection?

Because accuracy would reward predicting the majority class. Macro-F1 averages per-class F1 equally, so poor minority-class performance is visible. It is noisy with only 22 content-based test users, so we also look at balanced accuracy, AUC and hit-rate gain.

L8 [Expert] What does SMOTE do and why is it disabled?

It creates synthetic minority samples by interpolating between real neighbours. Across ten seeds it raised content-based F1 by about 0.038 but left AUC flat and lowered accuracy in 10 of 10 seeds, meaning it only moved the decision threshold. We kept the code, tested for leakage, and disabled it.

L9 [Expert] Why did probability calibration make it worse?

Sigmoid calibration (3-fold) flattened the class probabilities on a small, noisy dataset; held-out accuracy fell from 0.617 to 0.495 and its hit-rate gain over popularity vanished (+0.0001). Calibration needs more data than we have.

L10 [Critical] How do you know the model is not overfitting?

We do not claim it is immune: the train-test gap shows overfitting. What we control is that evaluation is on users never used for fitting or selection (stratified 80/20 split, CV inside the 80%), and that conclusions rest on held-out numbers with confidence intervals.

M. Evaluation metrics

M1 [Basic] Define precision and recall.

Precision: of what we predicted/recommended, how much was right. Recall: of what was truly relevant, how much we found. For the collaborative class: precision 191/418 = 0.457, recall 191/334 = 0.572.

M2 [Basic] What is F1?

The harmonic mean `2PR/(P+R)`, high only when both precision and recall are high.

M3 [Basic] What is a confusion matrix?

Counts of true class versus predicted class; the diagonal is correct. It shows which mistakes happen, such as 216 true-popularity users predicted collaborative.

M4 [Intermediate] Macro versus weighted F1?

Macro averages classes equally (0.472), weighted by class size (0.623). Macro exposes weak minority classes; weighted looks like accuracy.

M5 [Intermediate] What is ROC-AUC?

The probability that a random positive receives a higher score than a random negative; 0.5 is chance. For three classes we use one-vs-rest and average (0.735). Example: scores positives {0.9, 0.7, 0.4}, negatives {0.6, 0.3, 0.2}: 8 of 9 pairs ordered correctly, AUC 0.889.

M6 [Intermediate] Define Precision@K, Recall@K and Hit Rate@K.

P@K = hits in top K divided by K; R@K = hits divided by the number of relevant items; HR@K = share of users with at least one hit in the top K.

M7 [Intermediate] Define NDCG and MAP.

NDCG sums `1/log2(rank+1)` over hits and divides by the ideal sum, rewarding early hits. MAP averages, over users, the precision at each hit rank divided by min(K, #relevant).

M8 [Advanced] What is a "relevant" movie in your evaluation?

A held-out rating of at least 4.0 (`positive_threshold`). Movies the user rated poorly do not count as hits.

M9 [Advanced] What is the difference between accuracy and hit rate?

Accuracy (classification) is the share of users whose predicted strategy equals the labelled strategy. Hit Rate@10 (recommendation) is the share of users for whom at least one of the ten recommended movies was a movie they later liked. The dashboard's "accuracy" in the strategy comparison is Hit Rate@10, which is different from the classifier accuracy.

M10 [Expert] Why can Recall@10 be near zero for search?

There is no hand-labelled relevance, so relevance is a proxy (shares a genre or keyword). On average 1,812 of 26,743 movies are relevant per query, so recall is bounded by 10 divided by the number of relevant movies and measured 0.010. It says more about the relevant pool size than about the engine.

M11 [Expert] When is RMSE misleading?

A model can predict ratings accurately (low RMSE) and still rank poorly, since ranking depends on the order among high-rated items. We therefore report both (RMSE 0.846, MAE 0.649 for the bias predictor, and the ranking metrics).

M12 [Critical] Are the hit-rate numbers comparable across your experiments?

Only within the same protocol: hit-rate depends on holdout size, K and the number of positives. The benchmark's 0.877 (K=10, test split) and the classifier labelling's 0.128 (30 recommendations vs a 30% holdout) are different quantities.

N. Accuracy and label noise

N1 [Basic] What is label noise?

Labels that are not reliably correct. Here, a user's "best strategy" is the maximum of three noisy sample hit-rates computed from a small holdout (median 26 movies).

N2 [Intermediate] What is a standard error and how do you use it?

The typical sampling error of an estimate; for a hit-rate `p` from `m` holdout items it is about `sqrt(p(1-p)/m)`. We compared the gap between the top two strategies with their combined standard error: for 60.3% of users the gap is within one standard error, so the winner is not reliably identified.

N3 [Intermediate] What is label agreement?

Label the same users twice with two independent random holdouts and measure how often the labels match: 65.3% in our setting (73.3% with a larger holdout and budget).

N4 [Intermediate] What is the accuracy ceiling and how was it estimated?
We simulated an oracle that knows each user's true hit-rates (taking the observed values as truth), drew fresh noisy labels from the holdout sizes and computed how often the oracle's choice matches them: about 0.76.

N5 [Advanced] Why is training accuracy higher than test accuracy?
Boosted trees fit patterns, including noise, in the training labels; noise does not repeat in new data. Train 0.77-0.88 versus held-out 0.62.

N6 [Advanced] Why did changing algorithms not help?
Fourteen models and formulations all landed between 0.41 and 0.47 CV macro-F1 and 0.61-0.65 accuracy. When the target is only about two-thirds reproducible, model choice matters little; improving the label (budget, holdout size) and the information (features) matters more.

N7 [Advanced] What did improving the label definition do?
A 50-recommendation / 40%-holdout definition raised label agreement from 0.65 to 0.73. Combined with the improved content strategy, the experiment (not deployed) reached held-out accuracy 0.677 against a 0.564 majority rate, macro-F1 0.586 and ROC-AUC 0.807.

N8 [Expert] Is it legitimate to change the label definition to get better accuracy?
Only if the change makes the labels *more reliable and meaningful* and not merely easier. We justified it by agreement and uncertainty measures, we report both settings with their own baselines, and we never changed test labels to flatter results.

N9 [Expert] What is an "indistinct" user and how is it handled?
A user whose best strategy does not beat the runner-up by a one-sided 95% margin. 72-85% of users are indistinct depending on the setting. Instead of inventing a confident label for them, this representation records the uncertainty; among users with a clear winner in both labellings, agreement is 92-99%.

N10 [Critical] Why not remove ambiguous users to raise accuracy?
That would measure performance only on easy cases and overstate how well the system works for real users. We did it at one stage (accuracy 0.636) and then reverted to the full population because removing difficult samples is not honest.

N11 [Critical] Why can the classifier have a low accuracy yet improve recommendations?
Misclassifying a user whose strategies perform almost equally costs almost nothing; the value comes from the cases where the strategies differ clearly. The selector's hit-rate is 0.128 vs 0.112 (popularity), with a significant paired gain.

N12 [Critical] Is the oracle hit-rate of 0.148 a fair target?
No: it is the mean of each user's best of three noisy numbers, so it is biased upward and unreachable. We use it only as a rough upper bound; the selector captures about 43% of the gap between popularity and that bound.

O. Software architecture and implementation

O1 [Basic] How is the code organised?
By package: `ingestion`, `ir_engine`, `strategy_selector`, `ranking_engine`, `evaluation`, `benchmark`, `storage`, `ui`; scripts and tests alongside. Tunable settings live in `eval_config.yaml`.

O2 [Intermediate] How does MultiObjectiveRanker.rank work?
It generates or accepts a candidate pool, precomputes the four per-item objectives, then loops K times: score every remaining candidate (adding diversity and coverage relative to the selected set), pick the best, update the selected list and covered genres. It returns a frame with a column per objective.

O3 [Intermediate] How is the UI connected to the model?
`app.py` wraps loading and prediction; `runner.py` collects input, calls `recommend`, and renders results; `profile.py` reuses the same runner and result display for per-user recommendations.

O4 [Advanced] Why must training and serving use the same feature function?
Any difference (a column computed differently) silently shifts inputs and degrades predictions. `build_features` is used both to build the training table and in `app.recommend`, and a test checks they produce identical columns.

O5 [Advanced] How do you serialise models and what is the risk?
Pickle for scikit-learn/LightGBM objects, `save_npz` for sparse matrices, `np.savez` for EASE. Pickle can execute code on load, so files must come from a trusted source; pickled classes must live in importable modules.

O6 [Expert] What would you change for production?
Pin dependencies; add authentication; replace pickle with a safer format or ONNX; move features to a feature store; serve from a REST API with a model registry; add monitoring and online experiments; replace the in-process cache with a service.

O7 [Critical] What is the weakest part of the engineering?
Unpinned requirements, no authentication, no automated browser tests, and a ranking pipeline whose heavy parts run in one Streamlit process.

O8 [Critical] Where does the frozen baseline live and how is it protected from changes?
`src/benchmark/baseline.py` is self-contained (imports only numpy and pandas; a test checks this). `scripts/run_benchmark.py` stores a SHA-256 hash of the baseline, metrics and split code plus the baseline's results; if either the code or the numbers change, the run aborts.

P. Testing and debugging

P1 [Basic] How many tests do you have?
272 test cases in 12 files; all passed at the last full run.

P2 [Intermediate] Give an example of a test that prevents leakage.
`test_holdout_labels_never_use_holdout_ratings` and `test_features_use_only_the_ratings_given`; SMOTE tests assert synthetic rows are never derived from test users.

P3 [Intermediate] How do you test a probabilistic model?
With planted-signal tests (the model must find a known pattern) and noise tests (it must not beat chance on random labels).

P4 [Advanced] Describe a bug you found and how.
The popularity definition: both engines used TMDB popularity; we noticed the backtest popularity score was zero for most users and that the TMDB top-10 touched only 39% of users, so labels were assigned by tie-break. Switching to observed rating counts fixed the labels and the baseline.

P5 [Advanced] Another bug?
A saved classifier that could not be loaded because its estimator factory had been defined in a script and pickled as `__main__.make`; we moved it to `src/strategy_selector/estimators.py`.

P6 [Critical] What is not tested?
End-to-end browser behaviour, load/performance tests, and the correctness of external data. Pages were exercised with Streamlit's `AppTest` and manually.

Q. Project differentiation

Q1 [Basic] How is CineRankX different from a standard recommender website?
It chooses and weights methods per user, ranks on six objectives, explains each recommendation with scores, lets the operator change trade-offs live, logs runs, and reports its own measurement limits.

Q2 [Intermediate] What is your actual contribution?
The combination (per-user strategy prior plus six-objective explainable ranking), a leak-free and frozen evaluation methodology, and the documented finding that per-user best-strategy labels are only weakly reproducible. We do not claim a new algorithm.

Q3 [Intermediate] Why six objectives?
Each one counters a specific failure of relevance-only lists: repetition (diversity), filter bubbles (novelty), concentration (coverage), unreliable ratings (quality), and ignoring the user's likely score (predicted rating).

Q4 [Advanced] Why is per-user strategy selection useful?
Evidence differs by user. The selector gains most for light users (0.185 vs 0.152 for popularity among users with 21-40 ratings) and almost nothing for heavy users (0.070 vs 0.069).

Q5 [Advanced] How do you compare against modern platforms?
We do not claim equivalence; public descriptions (for example Gomez-Uribe and Hunt 2015) describe many algorithms, personalised ranking and online experiments. CineRankX is an offline, explainable, research-scale system.

Q6 [Expert] What would make it publishable?
Multi-dataset evaluation, comparison with learned hybrids and published algorithm-selection methods, online or counterfactual evaluation, a formal uncertainty-aware label protocol, and the label-reliability analysis as the central finding.

Q7 [Critical] Isn't this just a hybrid recommender?
It contains hybrid elements, but a standard hybrid uses one blend for everyone. Here the blend depends on a per-user prediction, and the final list is the product of a multi-objective ranking step, both measured.

R. Limitations and future scope

R1 [Basic] Name three limitations.
Weak content-based strategy, noisy strategy labels limiting classifier accuracy, and offline-only evaluation with no user feedback.

R2 [Intermediate] How would you improve content-based recommendation?
Richer text representation (bigrams helped), embeddings, metadata such as cast and director, a user taste model that weights liked and disliked items, and a quality prior; we measured the first and last.

R3 [Intermediate] How would you handle users with no history?
Fall back to popularity or quality-weighted popularity, ask for genre preferences, and use metadata-based content recommendations; EASE can score a user from even five ratings.

R4 [Advanced] How would you scale to millions of users?
Approximate nearest-neighbour search or learned embeddings instead of dense user-user similarity, a precomputed item-item model (EASE restricted to top items), a feature store, batch scoring and a serving service, plus incremental retraining.

R5 [Advanced] What is a contextual bandit and would it help?

A bandit learns online which action (here, strategy) pays off in a given context (user features) using observed rewards (clicks or watches). It would remove the label-noise problem by learning from real feedback, but needs live users; our system collects none.

R6 [Expert] Could reinforcement learning help?

Possibly for sequential session optimisation if interaction logs existed; with static ratings it adds complexity without clear benefit.

R7 [Critical] If given six more months?

Deploy the improved label definition and content strategy; add real feedback collection and a bandit; evaluate on several datasets with temporal splits; add embeddings and cast/crew metadata; productionise (API, authentication, monitoring).

S. Advanced research questions

S1 [Expert] How would you formally estimate the benefit of strategy selection?

Per-user off-policy comparison: for each user, estimate each strategy's expected utility with uncertainty, then compare the selector's choices against always-popularity with a paired bootstrap on per-user utilities (what we did) or with doubly robust estimators on logged data.

S2 [Expert] What is regression to the mean in the labels?

Picking the maximum of three noisy estimates overstates the winner's true advantage and makes labels unstable. It is why the oracle number is optimistic and why labels agree only 65-73%.

S3 [Expert] Why not use the continuous hit-rates as soft labels?

We tried predicting each strategy's expected hit-rate directly (utility regression with a fallback margin); it matched but did not beat the classifier. Soft targets reduce noise sensitivity but cannot add information the features lack.

S4 [Expert] How would you account for multiple comparisons across fourteen models?

Differences among top models were inside run-to-run noise; selection by CV macro-F1 on development users and a single held-out evaluation limit optimism. A strict approach would use nested CV or a pre-registered single comparison.

S5 [Expert] Could the benchmark's 0.877 be inflated by popularity?

Popularity is in the pool through the blend, but the frozen baseline gets 0.601 on the same ground truth; the gain is substantial in NDCG@10 (0.163 to 0.362) and precision (0.131 to 0.291). Coverage 5.7% vs 0.38% shows items beyond the head are being recommended.

S6 [Expert] What are the threats to validity?

One dataset; random (not temporal) splits that favour popularity; simulated cold start; proxy relevance for search; the classifier's simplified strategies in the backtest differ from the production sources; small held-out sets for rare classes.

S7 [Critical] Where could the evaluation still leak?

Candidate generators for the old pipeline use ratings of other users in the training pool; hyper-parameters of EASE were tuned on a validation cohort drawn from the same user population; the 5,000-user classifier labelling sample was drawn independently of the benchmark cohorts, so some users may appear in both (not checked). None uses the test interactions of the evaluated users, but these are points to mention honestly.

S8 [Critical] Defend the project in one sentence.

CineRankX shows that per-user strategy selection plus explainable multi-objective ranking can beat a frozen popularity baseline on held-out interactions (Hit Rate@10 0.877 vs 0.601), while honestly demonstrating that the strategy labels, not the algorithms, limit classifier accuracy.

Chapter 17. Practical Demonstration Preparation

17.1 Before the examiner arrives (10-minute checklist)

Check	Command or action	Expected
Environment	`cd CineRankX` then `pip install -r requirements.txt` (once)	no errors
Data present	`data/processed/movies_merged.csv`, `cinerankx.db`, `strategy_classifier.pkl`, `strategy_context.pkl`, `reports/kpi_report.json`, `reports/strategy_comparison.json` exist	files listed
Start the app	`streamlit run app.py`	browser opens at `http://localhost:8501`
Warm the cache	open **Run Engine** once and press Generate for user 5	first load takes several seconds; later runs about 0.2-0.7 s
Sample file	`demo/demo_ranked_user5.csv` (a real exported ranked list for user 5)	present
Tests (optional)	`python -m pytest -q`	all pass

17.2 Demonstration script (about 10 minutes)

0:00 Introduce (30 s). *"CineRankX is a movie recommender that predicts, per user, which strategy fits best and then ranks movies on six objectives. It uses MovieLens 20M, 20 million ratings, with TMDB metadata."*

0:30 Overview page. Show the four tiles (catalog titles, ratings in sample, strategy model Ready, search MRR), the candidate funnel and objective weights. *Say:* the weights come from `eval_config.yaml`; the funnel is real counts: catalog, rated titles, 200-candidate pool, latest slate. Mention strategy routing and telemetry come from the saved KPI report.

2:00 Recommendation Engines. Three cards: IR engine (Precision@10 0.41, MRR 0.44, search about 29 ms), strategy selector (accuracy, F1, AUC), ranker (MAP, diversity, novelty, coverage). *Say:* all numbers are read from `reports/kpi_report.json`.

3:00 Run Engine (the main demo).

1. Source: Live engine. User ID **5**, K **10**. The badge "User 5 found" appears; mention validation (an unknown ID disables the button).
2. Point at the six weight sliders and the "Raw total normalised to 1.00" pill. Explain each objective in one sentence (Chapter 7).
3. Press **Generate recommendations**. Note the elapsed time in the status line.
4. Read the first card: rank, title, year, genre chips, rank score, the *reason* ("ranked on strong relevance and diversity, sourced from collaborative+popularity") and the six bars.
5. **Change one thing**: raise the Diversity slider to 0.6, regenerate, show how the list changes. *Say:* the ranker is greedy and rescored against what is already chosen.
6. Press **Export ranked CSV**. Then switch Source to **Upload ranked CSV**, upload `demo/demo_ranked_user5.csv`: the same cards reappear (validated).
7. Show the **Open full profile** button.

6:00 User Profile. Overview tab (ratings count, average, genres, first/latest activity, distribution), History tab (search a title, filter by genre, sort, paginate: done in SQL), Preferences (favourite genres, decades, languages),

Past runs (the run you just made, with strategy, weights, time), Current recommendations. Mention MovieLens has no demographics, so none are shown.

8:00 KPI Benchmarks. Top cards: *Overall accuracy 0.877* and *Cold-start accuracy 0.698* with the frozen baselines 0.601 and 0.609; stress that this "accuracy" is **Hit Rate@10**, not classifier accuracy. Show the Strategy comparison panels (baseline vs old pipeline vs now), then the classifier card: accuracy 0.617, majority 0.644, macro-F1, balanced accuracy, AUC, confusion matrix, per-class table, and the caption about the label-noise ceiling.

9:30 Conclude (30 s). *"CineRankX beats a frozen popularity baseline on held-out interactions, explains each recommendation, and we have measured exactly why the strategy classifier is limited: label noise. Next steps are better labels, a stronger content model and real feedback."*

17.3 Questions an examiner may ask while the app is running

- *"What does this slider do?"* It sets the weight of one objective; weights are normalised to sum to 1.
- *"Why did the list change when you raised diversity?"* The greedy ranker penalises candidates similar to those already chosen more heavily.
- *"Which strategy was chosen for this user and how sure was it?"* Shown in the run context line and in Past runs (strategy and confidence).
- *"Is this number accuracy?"* On the KPI page "Overall accuracy" is Hit Rate@10; the classifier accuracy is in the classifier card.
- *"What if I type a user who is not there?"* The Generate button is disabled and a badge says the user is not in the sample.
- *"Show me a cold user."* The cold-start numbers are simulated (five ratings); explain why (MovieLens has none).
- *"Can you show the code?"* `src/ranking_engine/ranker.py::rank` (greedy loop), `src/ranking_engine/candidates.py::strategy_source_weights`, `src/strategy_selector/strategy_context.py::build_features`.

17.4 Troubleshooting during the demo

Problem	Quick fix
Page stuck on "CONNECTING..." or blank	The server stopped or restarted: rerun `streamlit run app.py` and refresh
"Local datasets are not ready"	Run the ingestion (`python -m src.ingestion.build_dataset`) and `scripts/verify_ir_engine.py`; or demonstrate with Upload ranked CSV
"User N is not in the loaded ratings sample"	Use a small user ID (1-2,900)
Profile page asks to build the user store	`PYTHONPATH=. python scripts/build_user_db.py` (long) or skip profiles in the demo
KPI page shows the "No offline report" note	Run `PYTHONPATH=. python scripts/generate_kpi_report.py 400000 1000`
Slow first run	The first Generate loads and caches the engine; warm it before the demo
Posters not showing	The History poster column loads images from TMDB's CDN and needs internet
A number looks different from this guide	Reports were regenerated; check the date of `reports/*.json`; the deployed classifier numbers are in `data/processed/strategy_eval.json`

Chapter 18. Ten-Day Study Roadmap

Day	Study	Key concepts	Files to open	Practise	You should be able to
1	Overview and architecture	three engines, user journey, data flow, hybrid vs selection	`README.md`, `CLAUDE.md`, `app.py`, Chapters 1-3	Draw the architecture from memory; viva A, B, C	Explain the project in two minutes and trace one request
2	Datasets and preprocessing	MovieLens files, TMDB join, text cleaning, splits, cold-start simulation	`src/ingestion/*`, `DATASET_MIGRATION.md`, Chapter 5	Viva D; compute the join rate; explain `clean_text` on a sentence	Justify every data decision and the three splits
3	Recommendation algorithms	CF, TF-IDF, cosine, popularity, Bayesian rating, EASE	`candidates.py`, `tfidf_vectorizer.py`, `ease.py`, Chapter 6	Redo the numerical examples by hand; viva E-I	Derive cosine, TF-IDF, weighted prediction
4	The six objectives	greedy MMR, each formula, trade-offs, weight tuning	`objectives.py`, `ranker.py`, `config.py`, Chapter 7	Redo the 3-candidate greedy example; viva J	Explain why each objective exists and how they conflict
5	Strategy classifier and LightGBM	label creation, features, boosting, gradients, overfitting	`holdout_labeler.py`, `strategy_context.py`, `formulations.py`, Chapter 8	Explain labels without notes; viva K, L	Describe training, validation, prediction and confidence use
6	Evaluation and mathematics	all metrics, CV, leakage, bootstrap, significance	`src/evaluation/*`, `src/benchmark/*`, Chapter 10	Recompute precision, recall, F1, AUC from the confusion matrix; viva M	Compute every metric by hand and say when each misleads
7	Codebase walkthrough	package responsibilities, key functions	Chapter 11, Appendix D, run a debugger on `rank()`	Open each file and state its input and output	Navigate the repository confidently
8	Differentiation and contribution	comparison table, innovation, limitations, publishability	Chapters 13, 15	Viva Q, R, S; write your own contribution statement	Say what is new, what is established, what is weak
9	Mock viva	accuracy investigation, label noise, honest reporting	Chapter 9, Chapter 16	Answer 40 random questions aloud; record yourself; peer mock	Answer "why is your classifier worse than baseline?" calmly
10	Demonstration and revision	demo flow, troubleshooting, key numbers	Chapter 17, Appendix B	Run the full demo twice with a timer; revise Appendix B	Deliver a 10-minute demo and handle failures

Appendix A. Glossary

Term	Meaning
Accuracy	share of correct predictions (classification)
AUC / ROC-AUC	probability a random positive outranks a random negative; 0.5 = chance
Backtest	evaluating a strategy on data held back from it (here: the user's hidden holdout)
Balanced accuracy	mean per-class recall; chance = 1/(number of classes)
Bayesian weighted rating	rating shrunk towards a global mean in proportion to how few votes it has
Bias (predictor)	a user's or item's systematic offset from the global mean rating
Bootstrap	resampling users with replacement to estimate uncertainty
Candidate generation	producing a pool of possible recommendations before ranking
Class imbalance	classes with very different frequencies
Cold start	a user or item with little or no history
Collaborative filtering	recommending from the behaviour of similar users or co-rated items
Confidence (selector)	maximum predicted class probability; steers the source blend
Confusion matrix	table of true versus predicted classes
Content-based filtering	recommending items similar in content to what the user liked
Cosine similarity	cosine of the angle between two vectors
Coverage	share of the catalog that appears in recommendations
Cross-validation	repeated train/test splits inside the training data to estimate generalisation
Data leakage	information from the evaluation target reaching training
DCG / NDCG	position-discounted gain of hits; NDCG normalises by the ideal ordering
Diversity	how dissimilar the items in one list are
EASE	closed-form linear item-item recommender (Steck 2019)
F1 score	harmonic mean of precision and recall
Feature	a numeric input describing a user or item
Frozen baseline	a reference method that is never retrained or changed
Gradient boosting	building trees sequentially, each fitting the previous errors
Greedy selection	choosing items one at a time, always the best given the current list
Hessian	second derivative of the loss; scales boosting steps
Hit rate@K	share of users with at least one relevant item in the top K
Holdout	data hidden from the model, used as the test
Hyperparameter	a setting chosen before training (learning rate, leaves)
IDF	inverse document frequency; down-weights common terms
Jaccard similarity	size of intersection divided by size of union of two sets
Label noise	labels that are not reliably correct
Label agreement	how often two independent labelings give the same label
LightGBM	gradient-boosting library with leaf-wise trees and histogram binning
MAP	mean (over users) of average precision
Macro / weighted average	unweighted / size-weighted mean over classes
MMR	maximal marginal relevance: balance relevance and novelty to already chosen items
MRR	mean reciprocal rank of the first relevant result
Majority-class baseline	accuracy of always predicting the most common class
Novelty	how unexpected (rare) an item is; self-information
Oracle	an idealised selector that always picks the best option (an upper bound)

Overfitting	fitting training noise; high training, lower test performance
Popularity bias	tendency to over-recommend already popular items
Precision@K / Recall@K	share of the top K that is relevant / share of relevant items found
Profile / holdout	the visible and hidden parts of a user's ratings
Regularisation	penalties that limit model complexity
RMSE / MAE	root mean squared / mean absolute error of rating predictions
SMOTE	oversampling by interpolating between minority samples
Sparsity	the fraction of empty cells in the user-item matrix
Standard error	typical sampling error of an estimate
Stratified sampling	keeping class proportions in each split
TF-IDF	term frequency times inverse document frequency weighting
Trie	prefix tree for fast prefix lookup
TreeSHAP	exact Shapley-value feature attributions for tree models
Utility (strategy)	expected recommendation quality of a strategy for a user

Appendix B. Quick Revision Notes (one page)

The idea. Predict the best strategy per user (content / collaborative / popularity), blend candidate sources by that prediction, re-rank on six objectives.

Key numbers (verified).

- Data: 20,000,263 ratings, 138,493 users, 27,278 movies; joined catalog 26,743 (98.04%).
- IR engine: Precision@10 0.41, MRR 0.44, NDCG 0.41, latency about 29 ms.
- Frozen benchmark (test): Hit Rate@10 baseline 0.601, old CineRankX 0.751, now **0.877** (cold 0.609, 0.658, 0.698); NDCG@10 0.163 -> 0.362; RMSE 0.936 -> 0.846.
- Classifier (999 held-out users, none dropped): accuracy **0.617**, majority 0.644, macro-F1 **0.472**, balanced accuracy **0.469**, ROC-AUC **0.735**.
- Selector value: hit-rate **0.128** vs popularity **0.112** vs oracle 0.148; gain +0.0155 [0.011, 0.020].
- Noise: winner within 1 SE for 60%; label agreement 65-73%; oracle ceiling about 0.76.
- Content-based hit-rate 0.016 (improved variant 0.045, not deployed).
- Best experiment: improved content + 50 recs / 40% holdout: accuracy 0.677 vs 0.564 majority; macro-F1 0.586; AUC 0.807; gain +0.037.
- System: about 23 ms per instrumented operation, about 46 ops/s, 398 MB.

Formulas.

- cosine: a.b / (||a||b||); idf = ln((1+N)/(1+df)) + 1.
- strategy weights: s = clip((conf - 1/3)/(2/3), 0, 1); w = (1-s) prior + s one-hot; prior (0.6, 0.2, 0.2).
- objectives: diversity = 1 - max sim; novelty = -log2 p; coverage = new genres / genres; quality = Bayesian WR; predicted = (r^ - 0.5)/4.5; score = sum of weight x objective (greedy).
- EASE: B = -P/diag(P), P = (X'X + lambda I)^-1.
- F1 = 2PR/(P+R); NDCG = DCG/IDCG; AUC = P(score+ > score-).
- LightGBM: g = p - y, h = p(1-p), leaf = -G/(H + lambda).

Say these sentences. "Accuracy is label-limited, not model-limited." "Hit rate and classification accuracy are different." "The baseline is frozen and self-contained." "Prediction is a weighted prior, not a hard switch." "Cold start is simulated because MovieLens has no cold users."

Never say. That the classifier is highly accurate; that CineRankX invented TF-IDF, CF or MMR; that the improved content model/labels are deployed; that cold-start results are natural.

Appendix C. References and Source Paths

C.1 Repository sources used for this guide

`README.md`, `CLAUDE.md`, `DATASET_MIGRATION.md`, `eval_config.yaml`, `requirements.txt`, `app.py`, all of `src/`, `scripts/`, `tests/`, `git log`, `demo/weekly_reports_week7-12.md`, and the saved artifacts: `data/processed/strategy_eval.json`, `selector_phase2_base.json`, `selector_phase2_rq_t30_h0.3.json`, `selector_phase2_rq_t50_h0.4.json`, `selector_analysis.json`, `selector_experiments_inner3.json`, `label_experiments.json`, `content_experiments.json`, `data/benchmark/baseline_frozen.json`, `data/benchmark/cinerankx_params.json`, `reports/kpi_report.json`, `reports/strategy_comparison.json`.

C.2 Literature (recalled from memory; verify details before citing in a report)

- F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context", ACM TiiS, 2015.
- K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval", Journal of Documentation, 1972 (IDF); G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval", 1988 (TF-IDF).
- J. Carbonell and J. Goldstein, "The use of MMR, diversity-based reranking...", SIGIR 1998.
- H. Steck, "Embarrassingly Shallow Autoencoders for Sparse Data", WWW 2019 (EASE).
- P. Cremonesi, Y. Koren, R. Turrin, "Performance of recommender algorithms on top-N recommendation tasks", RecSys 2010.
- M. D. Ekstrand and J. Riedl, "When Recommenders Fail: Predicting Recommender Failure for Algorithm Selection and Combination", RecSys 2012.
- C. A. Gomez-Uribe and N. Hunt, "The Netflix Recommender System: Algorithms, Business Value, and Innovation", ACM TMIS, 2015.
- G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", NeurIPS 2017.
- N. V. Chawla et al., "SMOTE: Synthetic Minority Over-sampling Technique", JAIR 2002.
- S. Lundberg and S. Lee, "A unified approach to interpreting model predictions", NeurIPS 2017 (SHAP); LightGBM's `pred_contrib` gives TreeSHAP values.

Appendix D. Complete Code Inventory (auto-extracted)

Generated from the repository by `docs/_build/extract_code.py` (AST parse). Each row: path, line count, the first line of the module docstring, then classes (methods) and functions. Use it to confirm that no module was left out of Chapter 11.

File	Lines	Summary	Symbols
app.py	157	Streamlit UI for live or exported Module 4 recommendation output.	load_engine, strategy_context, engine_parts, _log_run, recommend, db_connection, load_report
src/benchmark/baseline.py	44	FROZEN baseline strategy. Do not edit: any change creates a different benchmark.	FrozenBaseline (recommend, predict)
src/benchmark/cinerankx.py	69	CineRankX as a benchmark strategy: EASE relevance -> six-objective re-ranking.	CineRankXStrategy (fit, pools, recommend_many, predict) ; weights_from
src/benchmark/config.py	29	Benchmark settings from eval_config.yaml (the benchmark: block).	load_benchmark_config
src/benchmark/evaluate.py	44	Strategy-agnostic evaluation: every strategy is scored on identical users and ground truth	exclusions, evaluate_strategy
src/benchmark/metrics.py	67	Ranking and rating-prediction metrics for the frozen benchmark.	hit_rate, precision, recall, ndcg, average_precision, rating_errors, summarize
src/benchmark/split.py	141	Frozen benchmark: deterministic train/validation/test split and fixed evaluation cohorts.	Benchmark ; uniform_hash, assign_splits, load_interactions, _eligible, build_benchmark, fingerprint, get_benchmark
src/benchmark/tuning.py	83	Hyperparameter tuning on the VALIDATION split only (test interactions are never read).	_score, tune_relevance, _weight_candidates, tune_weights
src/evaluation/metric_names.py	89	The exact KPI name strings from eval_config.yaml.	load_configured_metrics, load_k_values, missing_from
src/evaluation/ml_metrics.py	132	Classifier metrics — the "machine_learning" KPI group.	majority_baseline_accuracy, roc_auc, classification_metrics, to_report_names
src/evaluation/recommendation_metrics.py	331	Recommendation KPIs, driven off Module 4's final ranked output.	precision_at_k, recall_at_k, f1_score_at_k, average_precision, mean_average_precision, coverage, intra_list_diversity, novelty, split_user_ratings, popularity_baseline_top_k, evaluate_cold_start, evaluate_recommendations
src/evaluation/report.py	230	Assembles and renders the full KPI report.	build_report, _fmt, _rule, _header, render
src/evaluation/search_metrics.py	180	Search KPIs — Precision@10, Recall@10, MRR, NDCG, Search Latency.	_tokenize, build_relevance_index, is_relevant, precision_at_k,

			recall_at_k, reciprocal_rank, ndcg_at_k, evaluate_search
src/evaluation/system_metrics.py	132	System KPIs — Response Time, Memory Usage, CPU Usage, Throughput.	measure, aggregate
src/ingestion/build_dataset.py	38	CLI entrypoint: loads raw MovieLens + TMDB data, joins them, cleans text,	build
src/ingestion/joiner.py	70	Joins MovieLens movies to TMDB metadata via links.csv, and builds the	_split_names, join_movielens_tmdb
src/ingestion/loaders.py	31	Loaders for the raw MovieLens 20M and TMDB 930K CSVs.	load_movielens, load_tmdb
src/ingestion/text_cleaner.py	31	NLTK-based text cleaning: lowercase, stop-word removal, stemming.	_ensure_nltk_data, clean_text
src/ir_engine/ir_engine.py	42	Unified IR engine combining TF-IDF search and Trie autocomplete.	IREngine (fit, save, load, search, autocomplete, similar_to)
src/ir_engine/tfidf_vectorizer.py	123	TF-IDF vectorization and cosine similarity search over movie texts.	TFIDFEngine (fit, save, load, search, similar_to)
src/ir_engine/trie.py	54	Trie for prefix-based autocomplete over movie titles.	TrieNode (); Trie (insert, search)
src/ranking_engine/candidates.py	307	Candidate generation: blends Module 2 (IR) and Module 3 (strategy) output.	CandidateGenerator (_build_popularity, _build_collaborative, _content_candidates, _collaborative_can) ; strategy_source_weights, genre_overlap_gate, _normalize
src/ranking_engine/config.py	105	Tunable weights for the six ranking objectives.	RankingWeights (items, with_overrides, from_config) ; load_genre_gate_floor
src/ranking_engine/ease.py	79	EASE item-item linear model: CineRankX's personalised relevance source.	EaseModel (fit, user_matrix, blended_scores, save, load)
src/ranking_engine/objectives.py	185	The six ranking objectives, plus the catalog statistics they read from.	CatalogStats (_build_genres, _build_popularity_quality, _build_novelty, genres, popularity_qua) ; _clean_genre, _minmax, diversity_score, coverage_score
src/ranking_engine/predictor.py	68	Baseline rating predictor — the source for the predicted_rating objective.	BaselinePredictor (fit, predict, predict_normalized)
src/ranking_engine/ranker.py	171	Multi-Objective Hybrid Ranking Engine (Module 4).	MultiObjectiveRanker (_similarity_lookup, rank, recommend) ; _jaccard
src/storage/db.py	84	SQLite schema and bulk ingestion for ratings, movies, per-user stats and run history.	connect, db_ready, _movie_frame, build_database
src/storage/runs.py	37	Persisted recommendation runs: what was generated, when, with which engine and weights.	record_run, list_runs
src/storage/users.py	104	Server-side user queries: profile summary, paginated	user_summary, _filters, history_page, _joined,

		history, preferences, timelines.	rating_distribution, genre_preferences, user_timeline, active_users
src/strategy_selector/classifier.py	248	Strategy classifier: predict best-fit strategy per user.	StrategyClassifier (fit, fit_full, _resample_training_fold, predict, save, load)
src/strategy_selector/config.py	33	Strategy-selector settings, read from eval_config.yaml (never hardcoded in the logic).	load_selector_config
src/strategy_selector/content_model.py	90	Content-based recommender with configurable representation, user profile and quality prior	ContentModel (profile_vector, top_items, hit_rate) ; _quality_percentile
src/strategy_selector/dataset.py	77	Build (and cache) the labelled training frame: sampled users -> features + backtest scores	sample_user_ids, load_ratings, cache_path, build_labelled_frame
src/strategy_selector/estimators.py	27	Importable estimator factory: saved models reference it by module path, so it must not liv	make
src/strategy_selector/formulations.py	186	Five ways to turn user features into a strategy choice, behind one sklearn-style interface	_Constant (predict_proba); _Base (predict); MulticlassForm (fit, predict_proba); OneVsRestForm (fit, predict_proba); PairwiseForm (fit, predict_proba); RegressionForm (fit, predict_scores, predict_proba); RankingForm (_expand, fit, predict_scores, predict_proba) ; winner, margin, _fit, _fit_binary, _softmax, decision_quality
src/strategy_selector/holdout_labeler.py	71	Leak-free strategy labels: backtest on a per-user holdout the profile never sees.	split_profile_holdout, label_holdout, clean_labels
src/strategy_selector/model_selection.py	165	Cross-validated model comparison and one-shot held-out evaluation for the selector.	_lgbm, _scaled, build_estimator, fit_candidate, tune_candidate, _proba, _auc, cross_validate, compare_candidates, evaluate_holdout
src/strategy_selector/profile_features.py	152	Extract user profile features from rating history (vectorized over all users).	_parse_time, _catalog_tables, _popularity_percentile, _genre_features, _time_features, extract_profile_features
src/strategy_selector/stats.py	46	Small statistics helpers: bootstrap confidence intervals and paired significance tests.	bootstrap_ci, paired_bootstrap, mcnemar_p, proportion_ci
src/strategy_selector/strategy_context.py	207	Strategy-aware features: a cheap backtest of each strategy on the user's OWN profile.	LiteContext (_build_cf_lite, cb_hit_rate, engine_scores, cf_hit_rate) ; inner_scores, build_features, save_context, load_context
src/strategy_selector/strategy_labeler.py	295	Backtest strategies to determine the best-fit strategy per user.	_BacktestContext (_build_popularity, _build_content_based, _build_collaborative) ; select_best_strategy, _build_cb_profile_vector,

			_content_based_hit_rate, _collaborative_hit_rate, _popularity_hit_rate, label_users_with_best_strategy
src/strategy_selector/utility.py	61	Utility-based strategy selection with an uncertainty-aware fallback.	UtilitySelector (fit, utilities, adjusted, predict_proba, predict) ; choose_tau
src/ui/comparison_view.py	66	Strategy Comparison: frozen baseline vs CineRankX on the fixed benchmark (reports/strategy	_delta_badge, _panel, render_comparison
src/ui/icons.py	28	Feather icons (https://feathericons.com, MIT) as inline SVG strings.	icon
src/ui/kpi_view.py	133	Evaluation workspace rendered from reports/kpi_report.json (generate_kpi_report.py).	_num, _comparison, render_report
src/ui/nav.py	39	Navigation state shared by screens: section labels, deep links and the selected user.	selected_user, set_user, open_profile, sync_view
src/ui/pages.py	146	Overview and engines screens. Everything shown comes from loaded data, config or reports.	_pool_size, _funnel, _routing, render_overview, render_engines
src/ui/profile.py	158	User 360 screen: summary, history, preferences, past runs and current recommendations.	_picker, _weights_from_session, _overview, _preferences, _current, render_profile
src/ui/profile_history.py	91	Profile tabs backed by the store: paginated rating history and past recommendation runs.	render_history, render_runs
src/ui/results.py	87	Validate ranked output and summarize metrics available from that output.	prepare_results, output_kpis, objective_label, split_title, reason_for, genres_lookup
src/ui/runner.py	192	Recommendation runner screen: source, user, K, objective weights, ranked results.	_default_weights, _reset_weights, _weights_panel, _live_controls, show_results, _csv_source, render_runner
src/ui/theme.py	160	CineRankX visual system: porcelain canvas, orange/cobalt accents, hairline cards.	apply_theme, header, card_title, card, card_html, kpi, kpi_row, meter
scripts/analyze_selector.py	128	Diagnose why the strategy classifier plateaus: labels, features, errors and attainable acc	margins, noise_ceiling, main
scripts/build_movielens_only.py	68	Build data/processed/ from MovieLens alone (no TMDB), for local demos.	main
scripts/build_user_db.py	29	Build data/processed/cinerankx.db (ratings, movies, user stats, run history).	main
scripts/compare_smote.py	139	Before/after comparison: Module 3's classifier with and without SMOTE.	
scripts/content_experiments.py	102	Does a better content representation actually raise	main

		held-out hit rate?	
scripts/generate_kpi_report.py	186	End-to-end KPI report — the project's final results table.	log, _plain
scripts/label_experiments.py	96	Compare label-generation methods: reliability, class mix and what a perfect model could sc	labels_for, classify, oracle_ceiling, main
scripts/run_benchmark.py	161	Reproducible strategy comparison: frozen baseline vs. CineRankX on one fixed benchmark.	code_hash, run_baseline, freeze_baseline, before_pipeline, main
scripts/selector_experiments.py	266	Systematic strategy-selector experiments on the FULL labelled population (no user dropped)	space, evaluate, cv_oof, tune, deploy_winner, main
scripts/selector_phase2.py	221	Strategy selection, evaluated on TWO separate objectives, with uncertainty and significanc	Reweighted (predict_proba, predict) ; per_user_value, oof_utilities, tune_thresholds, main
scripts/smote_seed_robustness.py	119	Multi-seed robustness check for the SMOTE result.	run_arm
scripts/smote_vs_class_weight.py	112	Does SMOTE do anything that cheaper class_weight tuning does not?	run
scripts/train_strategy_selector.py	147	Train and honestly evaluate the Adaptive Strategy Selector.	label_reliability, inference_latency, main
scripts/verify_ir_engine.py	52	Verification script for IR engine on real dataset.	
scripts/verify_ranking_engine.py	191	Verification script for the Multi-Objective Hybrid Ranking Engine (Module 4).	show, spread, horror_like
scripts/verify_strategy_selector.py	116	Verification script for Strategy Selector on real MovieLens data.	
tests/test_benchmark.py	138	Tests for the frozen benchmark: deterministic split, metrics, frozen baseline, EASE, evalu	8 test functions
tests/test_evaluation.py	604	Tests for the evaluation suite (all four KPI groups from eval_config.yaml).	71 test functions
tests/test_formulations.py	87	Tests for the strategy-choice formulations and the decision-quality metric.	6 test functions
tests/test_ingestion.py	103		6 test functions
tests/test_ir_engine.py	192	Tests for the IR engine: TF-IDF, Trie, and similarity search.	18 test functions
tests/test_ranking_engine.py	539	Tests for the Multi-Objective Hybrid Ranking Engine (Module 4).	72 test functions
tests/test_selector_pipeline.py	156	Tests for the richer features, leak-free labelling, margin cleaning and CV evaluation.	11 test functions
tests/test_strategy_context.py	84	Tests for the strategy-aware features (inner backtest) and their train/serve consistency.	6 test functions
tests/test_strategy_selector.py	741	Tests for the Adaptive ML	43 test functions

		Strategy Selection Engine.	
tests/test_ui_results.py	81	Unit tests for output displayed by the recommendation UI.	9 test functions
tests/test_user_store.py	80	Tests for the SQLite user store: lookup, history paging/filtering, run history.	6 test functions
tests/test_utility_stats.py	77	Tests for bootstrap statistics, the utility selector with fallback, and the content model.	6 test functions